

**IMPLEMENTASI VARIATIONAL AUTOENCODER DAN
REGULARIZED SINGULAR VALUE DECOMPOSITION PADA SISTEM
REKOMENDASI FILM**

SKRIPSI



IDA BAGUS GEDE BASUDEWA WEDA

NIM.2208561077

**PROGRAM STUDI INFORMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS UDAYANA
JIMBARAN**

2026

**IMPLEMENTASI VARIATIONAL AUTOENCODER DAN
REGULARIZED SINGULAR VALUE DECOMPOSITION PADA SISTEM
REKOMENDASI FILM**

SKRIPSI



**IDA BAGUS GEDE BASUDEWA WEDA
NIM.2208561077**

**PROGRAM STUDI INFORMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS UDAYANA
JIMBARAN
2026**

SURAT PERNYATAAN KEASLIAN KARYA ILMIAH

Yang bertanda tangan dibawah ini menyatakan bahwa naskah skripsi dngan judul:
**IMPLEMENTASI VARIATIONAL AUTOENCODER DAN
REGULARIZED SINGULAR VALUE DECOMPOSITION PADA SISTEM
REKOMENDASI FILM**

Nama : Ida Bagus Gede Basudewa Weda
NIM : 2208561077
Program Studi : Informatika
E – mail : basudewaweda@gmail.com
Nomor telp/HP : 08179328323
Alamat : Jln Sekar Tunjung XVIIIIB No. 3, Kesiman Kertalangu,
Denpasar, Bali

Belum pernah dipublikasikan dalam dokumen skripsi, jurnal maupun internasional atau dalam prosiding manapun, dan tidak sedang atau akan diajukan untuk publikasi di jurnal atau prosiding manapun. Apabila di kemudian hari terbukti terdapat pelanggaran kaidah-kaidah akademik pada karya ilmiah saya, maka saya bersedia menanggung sanksi-sanksi yang dijatuhkan karena kesalahan tersebut, sebagaimana diatur oleh Peraturan Menteri Pendidikan Nasional Nomor Tahun 17 Tahun 2010 tentang Pencegahan dan Penanggulangan Plagiat di Perguruan Tinggi. Demikian Surat Pernyataan ini saya buat dengan sesungguhnya untuk dapat digunakan bilaman diperlukan.

Jimbaran, 12 Juni 2026

Yang membuat pernyataan,

Ida Bagus Gede Basudewa Weda
NIM.2208561077

LEMBAR PENGESAHAN TUGAS AKHIR

Judul : Implementasi *Variational Autoencoder* dan *Regularized Singular Value Decomposition* Pada Sistem Rekomendasi Film

Nama : Ida Bagus Gede Basudewa Weda

NIM : 2208561077

Tanggal Seminar : 12 Juni 2026

Disetujui oleh:

Pembimbing I

Penguji I

Dr. Ir. Ngurah Agus Sanjaya ER, S.Kom.,
M.Kom.
NIP. 197803212005011001

Cokorda Rai Adi Pramatha, ST, MM.PhD
NIP. 197806212006041002

Penguji II

Ir. Gst. Ayu Vida Mastrika Giri, S.Kom.,
M.Cs.
NIP. 199006062022032009

Mengetahui,
Program Studi Informatika
FMIPA UNUD
KPS

Dr. I Made Widiartha, S.Si., M.Kom
NIP. 198212202008011008

Judul : Implementasi *Variational Autoencoder* dan *Regularized Singular Value Decomposition* Pada Sistem Rekomendasi Film
Nama : Ida Bagus Gede Basudewa Weda
Pembimbing : Dr. Ir. Ngurah Agus Sanjaya ER, S.Kom, M.Kom

ABSTRAK

Melimpahnya pilihan film yang tersedia menimbulkan tantangan bagi penonton untuk menemukan konten yang sesuai dengan preferensi mereka. Penelitian ini mengusulkan sistem rekomendasi film berbasis *hybrid ensemble* yang menggabungkan *Variational Autoencoder* (VAE) dengan *KL annealing* dan *Regularized Singular Value Decomposition* (RSVD) menggunakan *weighted average* pada dataset MovieLens 100K. Evaluasi dilakukan menggunakan *10-fold cross-validation*, pengujian *final* pada *hold-out test set*, dan uji signifikansi statistik melalui *paired t-test*. Hasil penelitian menunjukkan bahwa *hybrid ensemble* dengan bobot $\alpha = 0,25$ untuk VAE dan $\beta = 0,75$ untuk RSVD mencapai RMSE sebesar $0,9307 \pm 0,0043$ pada *cross-validation* dan RMSE sebesar 0,9365 pada *test set*, mengungguli RSVD *standalone* (RMSE 0,9506) dan VAE *standalone* (RMSE 1,0876). Seluruh perbedaan performa dikonfirmasi signifikan secara statistik melalui *paired t-test* dengan *p-value* jauh di bawah 0,05.

Kata Kunci: *Variational Autoencoder*, *Regularized Singular Value Decomposition*, Sistem Rekomendasi Film, *Hybrid Ensemble*, *Collaborative Filtering*, *KL Annealing*

Judul : *Implementation Of Variational Autoencoder and Regularized Singular Value Decomposition In Film Recommendation System*
Nama : Ida Bagus Gede Basudewa Weda
Pembimbing : Dr. Ir. Ngurah Agus Sanjaya ER, S.Kom, M.Kom

ABSTRACT

The abundance of available films presents a challenge for viewers in finding content that matches their preferences. This study proposes a hybrid ensemble movie recommendation system combining a Variational Autoencoder (VAE) with KL annealing and Regularized Singular Value Decomposition (RSVD) using a weighted average on the MovieLens 100K dataset. Model performance is evaluated using 10-fold cross-validation, final testing on a hold-out test set, and statistical significance testing via paired t-test. Results show that the hybrid ensemble with weights $\alpha = 0.25$ for VAE and $\beta = 0.75$ for RSVD achieves an RMSE of 0.9307 ± 0.0043 on cross-validation and 0.9365 on the test set, outperforming standalone RSVD (RMSE 0.9506) and standalone VAE (RMSE 1.0876). All performance differences are confirmed statistically significant by paired t-test with p-values well below 0.05.

Keywords: *Variational Autoencoder, Regularized Singular Value Decomposition, Movie Recommendation System, Hybrid Ensemble, Collaborative Filtering, KL Annealing*

KATA PENGANTAR

Puji syukur penulis panjatkan kehadapan Tuhan Yang Maha Esa atas berkat dan rahmat-Nya sehingga penulis dapat menyelesaikan Tugas Akhir yang berjudul "Implementasi *Variational Autoencoder* dan *Regularized Singular Value Decomposition* pada Sistem Rekomendasi Film" tepat pada waktunya. Tugas Akhir ini disusun sebagai salah satu syarat untuk memperoleh gelar Sarjana Komputer pada Program Studi Informatika, Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Udayana.

Penulis menyadari bahwa penyelesaian Tugas Akhir ini tidak terlepas dari dukungan dan bantuan berbagai pihak. Untuk itu, penulis mengucapkan terima kasih kepada:

1. Bapak Dr. Ir. Ngurah Agus Sanjaya ER, S.Kom., M.Kom. selaku Pembimbing.
2. Bapak Cokorda Rai Adi Pramatha, ST, MM.PhD dan Ibu Gst. Ayu Vida Mastika Giri, S.Kom., M.Cs. selaku Dosen Penguji.
3. Bapak Dr. I Made Widiartha, S.Si., M.Kom. selaku Koordinator Program Studi Informatika Fakultas Matematika dan Ilmu Pengetahuan Alam Universitas Udayana.
4. Orang tua, keluarga, dan teman-teman penulis yang senantiasa memberikan dukungan moral dan doa dalam penyelesaian Tugas Akhir ini.

Penulis menyadari bahwa Tugas Akhir ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat penulis harapkan demi perbaikan di masa mendatang. Semoga Tugas Akhir ini dapat memberikan manfaat bagi pembaca dan menjadi kontribusi bagi perkembangan penelitian di bidang sistem rekomendasi.

Jimbaran, 12 Juni 2026

Penulis

Ida Bagus Gede Basudewa Weda

NIM. 2208561077

DAFTAR ISI

HALAMAN JUDUL.....	i
SURAT PERNYATAAN KEASLIAN KARYA ILMIAH	ii
LEMBAR PENGESAHAN TUGAS AKHIR.....	iii
ABSTRAK	iv
<i>ABSTRACT</i>	v
KATA PENGANTAR.....	vi
DAFTAR ISI	vii
DAFTAR TABEL.....	x
DAFTAR GAMBAR	xiii
DAFTAR LAMPIRAN	xiv
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	4
1.3 Batasan Masalah.....	5
1.4 Tujuan Penelitian.....	5
1.5 Manfaat Penelitian.....	5
1.6 Sistematika Penulisan.....	5
BAB II TINJAUAN PUSTAKA	5
2.1 Tinjauan Teori.....	5
2.1.1 Sistem Rekomendasi	5
2.1.2 Collaborative Filtering	5
2.1.3 Variational Autoencoder (VAE)	6
2.1.4 Regularized Singular Value Decomposition (RSVD)	8
2.1.5 Sistem Rekomendasi Hibrid.....	10
2.1.6 Mean Absolute Error, Mean Squared Error, Root Mean Squared Error	
11	
2.1.7 K-Fold Cross Validation.....	12
2.1.8 Uji-t Berpasangan (<i>Paired t-test</i>)	13
2.2 Tinjauan Empiris	14

2.2.1	Deep Variational Models for Collaborative Filtering-Based Recommender Systems (Bobadilla et al., 2021).....	14
2.2.2	A Deep Autoencoder-Based Hybrid Recommender System (Bougteb et al., 2022)	15
2.2.3	Implementasi Regularized Singular Value Decomposition dalam Sistem Rekomendasi Buku Collaborative Filtering (Putra & Santiyasa, 2025)	16
2.2.4	Hybrid Recommendation System for Movies Using Artificial Neural Network (Sharma & Kumar Shakya, 2024).....	16
2.2.5	A Deep Learning Based Hybrid Recommendation Model for Internet Users (Sami et al., 2024).....	17
BAB III ANALISIS DAN PERANCANGAN SISTEM		18
3.1	Desain Sistem	18
3.2	Dataset	18
3.3	Preprocessing Data	19
3.4	Perancangan Model	21
3.4.1	Variational Autoencoder (VAE)	21
3.4.2	Regularized Singular Value Decomposition (RSVD)	22
3.4.3	Model <i>Hybrid</i> dengan <i>Weighted Average Ensemble</i>	23
3.5	<i>Hyperparameter Tuning</i>	23
3.5.1	<i>Hyperparameter Tuning</i> VAE	24
3.5.2	<i>Hyperparameter Tuning</i> RSVD	25
3.6	Evaluasi Model.....	26
3.6.1	Metrik Evaluasi	26
3.6.2	<i>10-Fold Cross-Validation</i>	27
3.6.3	Uji Signifikansi Statistik (<i>Paired t-Test</i>)	28
BAB IV HASIL DAN PEMBAHASAN		30
4.1	Implementasi Sistem	30
4.1.1	Program Definisi Model.....	32
4.1.2	Program Preprocessing Data	45
4.1.3	Program <i>Hyperparameter Tuning</i>	50

4.1.4	Program <i>Training</i>	61
4.1.5	Program <i>Cross Validation</i>	67
4.1.6	Program <i>Testing</i>	85
4.2	Hasil <i>Preprocessing Data</i>	96
4.3	Hasil <i>Hyperparameter Tuning</i>	97
4.3.1	Hasil <i>Hyperparameter Tuning VAE</i>	97
4.3.2	Hasil <i>Hyperparameter Tuning RSVD</i>	99
4.4	Performa Sistem Rekomendasi.....	99
4.4.1	Hasil <i>Cross-Validation</i>	100
4.4.2	Hasil Uji Signifikansi Statistk (<i>Paired t-Test</i>).....	103
4.4.3	Hasil Pelatihan Model	104
4.4.4	Hasil Pengujian Model.....	104
4.5	Model Optimal	106
4.5.1	Bobot Ensemble Optimal	106
4.5.2	Perbandingan <i>Hybrid</i> dan <i>Standalone</i>	107
4.6	Pembahasan	107
BAB V KESIMPULAN DAN SARAN.....		110
5.1	Kesimpulan.....	110
5.2	Saran	111
DAFTAR PUSTAKA.....		112
LAMPIRAN.....		115

DAFTAR TABEL

Tabel 3.1. Contoh Data ratings.csv	19
Tabel 3.2. Contoh Matriks Interaksi.....	20
Tabel 3.3. Ruang Pencarian <i>Hyperparameter</i> VAE	24
Tabel 3.4. Ruang Pencarian <i>Hyperparameter</i> RSVD	25
Tabel 4.1. Deskripsi Komponen Implementasi Sistem	31
Tabel 4.2. Kelas <i>Sampling</i>	32
Tabel 4.3. Kelas <i>Encoder</i>	33
Tabel 4.4. Kelas <i>Decoder</i>	34
Tabel 4.5. Kelas VAE	34
Tabel 4.6. Kelas <i>KLAnnealingCallback</i>	38
Tabel 4.7. Kelas RSVD	39
Tabel 4.8. Cell <i>File Paths</i>	45
Tabel 4.9. Cell <i>Load Data</i>	46
Tabel 4.10. Cell Matriks Interaksi <i>User-Item</i>	46
Tabel 4.11. Cell Normalisasi Rating	47
Tabel 4.12. Cell Konfigurasi Rasio <i>Split</i>	48
Tabel 4.13. <i>Cell Shuffle</i> dan Potong Indeks	48
Tabel 4.14. Cell Bangun Matriks <i>Train, Validation, dan Test</i>	48
Tabel 4.15. Cell <i>Save Data</i>	49
Tabel 4.16. Cell <i>Import Library</i>	50
Tabel 4.17. Cell <i>Path</i> Folder	51
Tabel 4.18. Cell <i>Import</i> Data & Model	51
Tabel 4.19. Cell Definisi Ruang Pencarian VAE	53
Tabel 4.20. Cell Fungsi Objektif VAE	54
Tabel 4.21. Cell Inisialisasi <i>Study</i> Optuna	56
Tabel 4.22. Cell Eksekusi <i>Tuning</i> VAE.....	56
Tabel 4.23. Cell Simpan Hasil Terbaik <i>Tuning</i> VAE.....	57
Tabel 4.24. Cell Definisi Ruang Pencarian RSVD	58
Tabel 4.25. Cell Load Progress <i>Tuning</i> RSVD	58

Tabel 4.26. Cell Eksekusi <i>Tuning</i> RSVD	59
Tabel 4.27. Cell <i>Import Library</i> Pada Training.ipynb.....	61
Tabel 4.28. Cell Konfigurasi Direktori Pada Training.ipynb	62
Tabel 4.29. Cell Load Data Latih dan Import Kelas Pada Training.ipynb.....	62
Tabel 4.30. Cell Import <i>Hyperparameter</i> dan Konstruksi VAE Pada Training.ipynb 63	
Tabel 4.31. Cell <i>Training</i> VAE Pada Training.ipynb	65
Tabel 4.32. Cell Menyimpan Bobot dan <i>Loss History</i> VAE Pada Training.ipynb	65
Tabel 4.33. Cell Load <i>Hyperparameter</i> dan Konstruksi Model RSVD Pada Training.ipynb	66
Tabel 4.34. Cell <i>Training</i> Model RSVD Pada Training.ipynb.....	66
Tabel 4.35. Cell Menyimpan Bobot dan <i>Loss History</i> RSVD Pada Training.ipynb 67	
Tabel 4.36. Cell <i>Import Library</i> Pada CrossValidation.ipynb.....	68
Tabel 4.37. Cell Konfigurasi Direktori Pada CrossValidation.ipynb	68
Tabel 4.38. Cell <i>Import</i> Model, <i>Load</i> Data dan <i>Hyperparameter</i> Pada CrossValidation.ipynb	69
Tabel 4.39. Cell Konfigurasi <i>Cross-Validation</i> Pada CrossValidation.ipynb.....	70
Tabel 4.40. Cell Ekstraksi Koordinat Rating dan Pembagian <i>Fold</i> Pada CrossValidation.ipynb	71
Tabel 4.41. Cell Fungsi <i>create_fold_matrices</i> Pada CrossValidation.ipynb	72
Tabel 4.42. Cell Fungsi <i>train_vae_fold</i> Pada CrossValidation.ipynb.....	73
Tabel 4.43. Cell Fungsi <i>train_rsvd_fold</i> Pada CrossValidation.ipynb	76
Tabel 4.44. Cell Fungsi <i>find_best_ensemble_alpha</i> Pada CrossValidation.ipynb	77
Tabel 4.45. Cell Fungsi <i>calculate_metrics</i> Pada CrossValidation.ipynb.....	78
Tabel 4.46. Cell <i>Loop</i> Utama <i>Cross-Validation</i> Pada CrossValidation.ipynb.....	78
Tabel 4.47. Cell Perhitungan Ringkasan dan Penyimpanan Hasil Pada CrossValidation.ipynb	81
Tabel 4.48. Cell Perhitungan Ringkasan dan Penyimpanan Hasil Pada CrossValidation.ipynb	83
Tabel 4.49. Cell <i>Import Library</i> Pada Testing.ipynb.....	85

Tabel 4.50. Cell Konfigurasi Direktori Pada Testing.ipynb	86
Tabel 4.51. Cell <i>Load</i> Data dan Model Pada Testing.ipynb.....	87
Tabel 4.52. Cell Konstruksi dan <i>Load</i> Bobot VAE Pada Testing.ipynb.....	88
Tabel 4.53. Cell <i>Load</i> Komponen Bobot RSVD Pada Testing.ipynb	89
Tabel 4.54. Cell Prediksi VAE Pada Testing.ipynb	90
Tabel 4.55. Cell Prediksi RSVD Pada Testing.ipynb	91
Tabel 4.56. Cell Pencarian Bobot Alpha <i>Ensemble</i> Pada Testing.ipynb	92
Tabel 4.57. Cell Penerapan Alpha ke Test Set Pada Testing.ipynb	93
Tabel 4.58. Fungsi <i>calculate_metrics</i> Pada Testing.ipynb	94
Tabel 4.59. Cell Perhitungan Metrik Pada Testing.ipynb	94
Tabel 4.60. Cell Penyimpanan Hasil Evaluasi Pada Testing.ipynb.....	95
Tabel 4.61. Atribut Dataset Setelah <i>Preprocessing</i>	96
Tabel 4.62. <i>Hyperparameter</i> Terbaik VAE.....	98
Tabel 4.63. <i>Hyperparameter</i> Terbaik RSVD	99
Tabel 4.64. Ringkasan Statistik RMSE dan MAE 10-Fold <i>Cross-Validation</i>	102
Tabel 4.65. Hasil <i>Paired t-Test</i> Antar Model.....	103
Tabel 4.66. Hasil Pengujian Pada <i>Test Set</i>	105

DAFTAR GAMBAR

Gambar 3.1. Metode Penelitian.....	18
Gambar 3.2. Alur <i>Preprocessing</i> Data	19
Gambar 3.3. Ilustrasi Pembagian Data.....	20
Gambar 3.4. Struktur Cross Validation	27
Gambar 4.1. Hubungan Antar Komponen.....	30
Gambar 4.2. Perbandingan RMSE Per <i>Fold</i>	100
Gambar 4.3. Perbandingan MAE Per <i>Fold</i>	101
Gambar 4.4. Perbandingan MAE Per <i>Fold</i>	102
Gambar 4.5. Kurva Konvergensi VAE dan RSVD	104
Gambar 4.6. Kurva Tuning Bobot <i>Ensemble Alpha</i> pada <i>Validation Set</i>	105

DAFTAR LAMPIRAN

Lampiran 1. Program DataPreprocessing.ipynb	115
Lampiran 2. Program HyperParameterTuningOptuna.ipynb	118
Lampiran 3. Program Training.ipynb.....	127
Lampiran 4. Program Testing.ipynb.....	133
Lampiran 5. Program CrossValidation.ipynb.....	140
Lampiran 6. Program model.py	163

BAB I

PENDAHULUAN

1.1 Latar Belakang

Industri perfilman mengalami peningkatan yang signifikan pada jumlah film yang diproduksi setelah terjadinya penurunan produksi yang disebabkan oleh terjadinya pandemi COVID-19 pada tahun 2020. Pada tahun 2021 produksi film meningkat sebesar 40% dan pada tahun 2022 meningkat sebesar 15%. Keuntungan yang diraup dari Global box office juga meningkat pada tahun 2022 meningkat sebesar 21,8% dan pada tahun 2023 meningkat sebesar 29,4% (Hancock et al., 2024). Peningkatan kuantitas film yang diproduksi dan keuntungan yang didapatkan akan memperbanyak film yang tersedia. Banyaknya film yang tersedia akan memberikan pilihan yang banyak kepada penonton, tetapi juga memberikan tantangan yaitu menemukan film yang sesuai dengan preferensi mereka. Sistem rekomendasi dapat digunakan untuk mengatasi tantangan ini.

Sistem rekomendasi merupakan sistem yang berfungsi untuk memperkirakan dan memberikan informasi yang sekiranya menarik untuk seorang pengguna. Sistem rekomendasi menggunakan berbagai metode untuk memberikan saran produk berdasarkan preferensi dan kebutuhan pengguna seperti content-based filtering, collaborative filtering, dan metode hybrid. Metode content-based filtering bekerja dengan menganalisis atribut atau fitur dari item-item yang disukai oleh pengguna dan kemudian merekomendasikan item lain yang memiliki kemiripan atribut (Ayu et al., 2021). Metode collaborative filtering dapat dibagi menjadi dua kategori yaitu memory-based dan model-based. Kategori memory-based menggunakan tingkat similaritas untuk memberikan prediksi dengan mencari asosiasi antara pengguna dengan item (Rajput et al., 2024). Kategori memory-based dapat dikategorikan lagi menjadi item-based dan user-based. Item-based menganalisis bagaimana pengguna menilai suatu item dan mencari item dengan penilain yang mirip. User-based mencari pengguna yang mirip berdasarkan penilaian yang diberikan dan merekomendasikan item yang pengguna mirip sudah sukai ke pengguna target. Model-based menggunakan matriks pengguna-item

sebagai data untuk membangun dan melatih sebuah model yang dapat memprediksi penilaian pengguna terhadap suatu item.

Salah satu tantangan utama dalam sistem rekomendasi adalah data sparsity. Data sparsity merujuk pada situasi di mana interaksi antara pengguna dan item sangat sedikit atau jarang. Dalam sistem rekomendasi berbasis matriks, ini berarti matriks pengguna-item memiliki banyak nilai kosong atau tidak diketahui. Hal ini menyulitkan model untuk menemukan pola yang cukup kuat untuk memberikan rekomendasi yang akurat. Misalnya, dalam layanan streaming dengan ribuan film, kebanyakan pengguna mungkin hanya menonton atau memberikan rating pada sebagian kecil dari film yang tersedia, menyebabkan matriks menjadi sangat jarang terisi.

Penelitian terkait sistem rekomendasi telah banyak dilakukan untuk mengatasi masalah data sparsity dan meningkatkan akurasi prediksi. (Rajput et al., 2024) menggunakan penggabungan standard autoencoder dan faktorisasi matriks dengan singular value decomposition (SVD) untuk mengatasi masalah sparsity dalam sistem rekomendasi multi-kriteria, dengan hasil MAE sebesar 0.3350 dan RMSE sebesar 0.4897. (Wibisono et al., 2021) mengembangkan sistem rekomendasi menggunakan item-based collaborative filtering dengan SVD yang terbukti mampu mereduksi dimensi matriks rating dan mengurangi noise pada data sparse, dengan hasil MAE sebesar 1.2752 dan RMSE sebesar 1.4882. Selain itu, (Mu'afa & Baizal, 2022) menunjukkan bahwa penggabungan SVD dengan metode lain mampu meningkatkan akurasi prediksi rating dengan RMSE sebesar 0.835 dibandingkan 0.896 tanpa SVD. Penelitian-penelitian ini secara konsisten menunjukkan potensi penggabungan faktorisasi matriks berbasis SVD dengan metode lain untuk meningkatkan performa sistem rekomendasi. Namun, seluruh penelitian tersebut menggunakan standard autoencoder, bukan Variational Autoencoder (VAE), dan SVD tanpa regularisasi sehingga masih terdapat gap penelitian yang belum dieksplorasi.

Berdasarkan gap tersebut, penelitian ini mengajukan pendekatan yang menggabungkan Variational Autoencoder (VAE) dan Regularized Singular Value Decomposition (RSVD). VAE dipilih karena memiliki karakteristik yang sesuai

untuk sistem rekomendasi, sifat Bayesian-nya memungkinkan VAE menangani data sparse dan kompleks seperti data interaksi pengguna-item secara lebih tepat karena ketidakpastian diperhitungkan dalam proses inferensi, dan VAE mempelajari representasi laten sebagai distribusi probabilistik sehingga menghasilkan latent space yang lebih terstruktur dibandingkan standard autoencoder (Bobadilla et al., 2021; Liang et al., 2024). Namun, pelatihan VAE pada data collaborative filtering berisiko mengalami posterior collapse, yaitu kondisi di mana model mengabaikan struktur ruang laten dan hanya mengandalkan decoder sehingga representasi laten menjadi tidak informatif (Wang et al., 2025). Untuk mengatasi hal ini, penelitian ini menerapkan KL annealing yaitu sebuah teknik yang menaikkan bobot β pada KL divergence secara bertahap selama pelatihan agar model terlebih dahulu mempelajari rekonstruksi sebelum tekanan regularisasi diberikan sepenuhnya (Liang et al., 2024). Di sisi lain, SVD standar pada sistem rekomendasi sering kali menemui kendala ketika dihadapkan pada data yang sangat sparse. SVD standar cenderung mengalami overfitting karena model berusaha menghafal data latih sehingga menghasilkan generalisasi yang buruk pada data yang belum pernah dilihat sebelumnya. Penerapan regularisasi pada RSVD mengatasi kelemahan ini dengan memberikan penalti pada parameter latent factor pengguna dan item, yang secara signifikan mengurangi risiko overfitting dan membuat model lebih stabil serta akurat dalam memprediksi rating pada matriks yang jarang terisi (Putra & Santiyasa, 2025). Kombinasi spesifik VAE dan RSVD ini belum pernah dieksplorasi dalam penelitian sebelumnya, sehingga penelitian ini berkontribusi dalam mengisi gap tersebut.

Pendekatan hybrid yang menggabungkan dua model dengan karakteristik berbeda terbukti mampu mengatasi keterbatasan masing-masing model secara individual dalam sistem rekomendasi, karena setiap model dapat saling melengkapi kelemahan yang lain (Sami et al., 2024). Dalam penelitian ini, VAE dan RSVD memiliki karakteristik yang saling komplementer, VAE unggul dalam menangkap pola preferensi pengguna yang kompleks dan non-linear melalui representasi probabilistik di ruang laten (Bobadilla et al., 2021; Liang et al., 2024), namun prediksinya dapat kurang stabil pada data yang sangat sparse karena bergantung

pada sampling dari distribusi laten. Sebaliknya, RSVD menghasilkan prediksi yang lebih stabil melalui dekomposisi bias dan faktor laten yang terregularisasi (Putra & Santiyasa, 2025), namun terbatas dalam menangkap hubungan non-linear antar pengguna dan item. Dengan menggabungkan prediksi keduanya melalui weighted average ensemble penelitian ini bertujuan memanfaatkan kekuatan representasi probabilistik VAE sekaligus kestabilan prediksi RSVD, sehingga menghasilkan sistem rekomendasi yang lebih akurat dibandingkan masing-masing model secara mandiri, sebagaimana dikonfirmasi oleh hasil evaluasi pada dataset MovieLens 100K dengan RMSE *hybrid ensemble* sebesar 0,9365 yang mengungguli RSVD *standalone* (RMSE 0,9506) dan VAE *standalone* (RMSE 1,0876).

Dataset *MovieLens* 100K dipilih sebagai data evaluasi karena dataset ini merupakan salah satu *benchmark* yang paling banyak digunakan dalam penelitian sistem rekomendasi berbasis *collaborative filtering*, sehingga hasil yang diperoleh dapat dibandingkan secara indikatif dengan penelitian-penelitian terdahulu pada domain yang sama (Sami et al., 2024). Selain itu, ukurannya yang mencakup 100.000 *rating* dari 943 pengguna terhadap 1.682 film menjadikan dataset ini cukup representatif untuk eksperimen tanpa membutuhkan sumber daya komputasi yang berlebihan. Berdasarkan hal tersebut, penelitian ini bertujuan untuk mengembangkan dan mengevaluasi sistem rekomendasi film yang menggabungkan VAE dan RSVD menggunakan dataset MovieLens 100k, serta mengukur performanya menggunakan metrik *mean squared error* (MSE), *root mean squared error* (RMSE), dan *mean absolute error* (MAE).

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dijabarkan, rumusan masalah pada penelitian ini adalah sebagai berikut:

- a. Bagaimana performa sistem rekomendasi dengan *variational autoencoder* dan *regularized singular value decomposition* dalam menghasilkan rekomendasi?
- b. Bagaimana model yang optimal untuk sistem rekomendasi dengan *variational autoencoder* dan *regularized singular value decomposition*?

1.3 Batasan Masalah

Terdapat beberapa batasan permasalahan dalam penelitian ini:

- a. Data didapatkan secara sekunder dari situs web grouplens.org yang berjudul “movielens 100k”.

1.4 Tujuan Penelitian

Berdasarkan latar belakang dan rumusan masalah yang telah dijabarkan, tujuan dari penelitian ini adalah sebagai berikut:

- a. Mengetahui performa sistem rekomendasi yang menggunakan *variational autoencoder* dan *regularized singular value decomposition* dalam menghasilkan rekomendasi.
- b. Mengetahui model yang optimal untuk sistem rekomendasi dengan *variational autoencoder* dan *regularized singular value decomposition*.

1.5 Manfaat Penelitian

Manfaat dari hasil penelitian ini adalah:

- a. Bagi penonton, dapat dengan mudah mendapatkan rekomendasi yang sesuai dengan preferensi mereka.
- b. Bagi pembuat film, Film yang dibuat dapat direkomendasikan kepada pengguna sesuai dengan preferensi mereka.

1.6 Sistematika Penulisan

Untuk memberikan gambaran yang jelas dan menyeluruh mengenai penyusunan Tugas Akhir ini, maka penulisan dibagi menjadi lima bab dengan sistematika sebagai berikut:

BAB I PENDAHULUAN

Bab ini menguraikan kerangka dasar penelitian yang meliputi latar belakang masalah mengenai data sparsity pada sistem rekomendasi film, rumusan masalah, tujuan penelitian, batasan masalah, manfaat penelitian, serta sistematika penulisan yang digunakan dalam penyusunan tugas akhir.

BAB II TINJAUAN PUSTAKA

Bab ini membahas landasan teori dan tinjauan empiris dari penelitian-penelitian terdahulu yang relevan. Landasan teori yang diuraikan mencakup konsep dasar sistem rekomendasi, *Collaborative Filtering*, *Variational*

Autoencoder (VAE), Regularized Singular Value Decomposition (RSVD), serta metrik evaluasi pengujian (seperti MSE, RMSE, dan MAE). Bab ini menjadi dasar pijakan untuk perancangan model yang diusulkan.

BAB III ANALISIS DAN PERANCANGAN SISTEM

Bab ini menjelaskan tahapan, analisis, dan perancangan yang dilakukan dalam penelitian. Pembahasan di dalamnya meliputi desain sistem secara umum, teknik pengumpulan data sekunder (dataset MovieLens 100k), perancangan model (preprocessing data, pengembangan model VAE, dan dekomposisi dengan RSVD), penggabungan model melalui pendekatan *weighted average ensemble*, serta rancangan skenario evaluasi kinerja model menggunakan 10-fold cross-validation dan pengujian pada data uji akhir.

BAB IV HASIL DAN PEMBAHASAN

Bab ini memaparkan secara detail hasil dari pengembangan dan pengujian model. Pembahasan difokuskan pada analisis performa sistem rekomendasi hasil penggabungan arsitektur VAE dan RSVD, hasil hyperparameter tuning, perbandingan metrik evaluasi (RMSE dan MAE) antar model untuk mengukur akurasi prediksi, serta hasil evaluasi stabilitas model melalui 10-fold cross-validation dan pengujian akhir pada data uji.

BAB V KESIMPULAN DAN SARAN

Bab ini berisi kesimpulan akhir yang ditarik dari hasil analisis dan pembahasan untuk menjawab rumusan masalah penelitian. Selain itu, pada bab ini juga dikemukakan saran-saran yang konstruktif untuk mengatasi keterbatasan yang ada serta sebagai bahan pertimbangan bagi pengembangan sistem atau penelitian sejenis di masa yang akan datang.

BAB II

TINJAUAN PUSTAKA

2.1 Tinjauan Teori

2.1.1 Sistem Rekomendasi

Sistem rekomendasi merupakan sistem yang berfungsi untuk memperkirakan dan memberikan informasi yang sekiranya menarik untuk seorang pengguna (Nugroho & Rahayu, 2020). Sistem rekomendasi bekerja dengan menganalisis data pengguna, seperti riwayat pembelian, tontonan, dan interaksi mereka dengan platform digital. Berdasarkan data tersebut, sistem rekomendasi dapat memprediksi apa yang disukai dan dibutuhkan pengguna, dan kemudian memberikan rekomendasi yang dipersonalisasi. Rekomendasi yang diberikan oleh sistem dapat digunakan oleh pengguna untuk mempertimbangkan apa yang harus mereka lakukan, seperti membeli barang atau menonton pertunjukan. Sistem ini banyak digunakan dalam berbagai industri, seperti *e-commerce*, *media streaming*, media sosial, dan lainnya.

2.1.2 Collaborative Filtering

Collaborative filtering adalah teknik yang digunakan dalam sistem rekomendasi untuk memberikan saran item kepada pengguna berdasarkan preferensi pengguna lain yang serupa (Sutjiningtyas et al., 2022). Sederhananya, sistem ini mencari pengguna yang memiliki selera yang sama dengan pengguna yang ingin mendapatkan rekomendasi, dan kemudian merekomendasikan item yang disukai oleh pengguna-pengguna tersebut. Contohnya, di situs web *e-commerce*, *collaborative filtering* dapat digunakan untuk merekomendasikan produk kepada pengguna berdasarkan produk yang telah mereka beli atau lihat sebelumnya, serta produk yang dibeli oleh pengguna lain yang memiliki profil serupa.

Collaborative filtering dapat dibagi menjadi dua kategori yaitu *memory-based* dan *model-based*. Kategori *memory-based* menggunakan tingkat similaritas untuk memberikan prediksi dengan mencari asosiasi antara pengguna dengan *item* (Rajput et al., 2024). Kategori *memory-based* dapat dikategorikan lagi menjadi *item-based* dan *user-based*. *Item-based* menganalisis bagaimana pengguna menilai

suatu *item* dan mencari *item* dengan penilai yang mirip. *User-based* mencari pengguna yang mirip berdasarkan penilaian yang diberikan dan merekomendasikan item yang pengguna mirip sudah sukai ke pengguna target. *Model-based* menggunakan matriks pengguna-item sebagai data untuk membangun dan melatih sebuah model yang dapat memprediksi penilaian pengguna terhadap suatu item.

2.1.3 Variational Autoencoder (VAE)

Variational Autoencoder (VAE) adalah model generatif berbasis jaringan saraf tiruan yang mempelajari representasi laten dari data masukan sebagai distribusi probabilistik, bukan sebagai titik tetap di ruang laten (Liang et al., 2024). Berbeda dengan autoencoder standar yang memetakan setiap input ke satu titik laten, VAE memetakan input ke parameter distribusi Gaussian, yaitu rata-rata μ dan log-varians $\log(\sigma^2)$, sehingga ruang laten menjadi kontinu dan terstruktur. VAE memodelkan data sebagai:

$$p(x, z) = p(x|z)p(z) \quad (1)$$

Keterangan:

- x = data masukan
- z = representasi laten
- $p(x|z)$ = probabilitas x diberikan z
- $p(z)$ = prior dari z , diasumsi sebagai distribusi Gaussian standar $N(0,1)$

Karena posterior sejati $p(z|x)$ bersifat *intractable*, VAE mengaproksimasi posterior tersebut dengan distribusi variasional:

$$q(z|x) = N(z|\mu(x), \sigma(x)) \quad (2)$$

Keterangan:

- $\mu(x)$ = rata-rata yang dihasilkan *encoder*
- $\sigma(x)$ = varians yang dihasilkan *encoder*

Fungsi objektif VAE adalah *Evidence Lower Bound* (ELBO):

$$\mathcal{L} = E_{q(z|x)}[\log p(x|z)] - \beta \cdot KL(q(z|x)||p(z)) \quad (3)$$

Keterangan:

- $E_{q(z|x)}[\log p(x|z)]$ = *reconstruction loss*

- $\beta \cdot KL(q(z|x)||p(z)) = \text{KL divergence}$

Bagian pertama adalah *reconstruction loss* yang mengukur seberapa baik *decoder* merekonstruksi data dari z , dan bagian kedua adalah *KL divergence* yang meregularisasi distribusi laten agar mendekati prior Gaussian. *Hyperparameter* β mengontrol bobot antara dua komponen ini (Wang et al., 2025).

Untuk kasus *prior* Gaussian standar $\mathcal{N}(0, I)$, *KL divergence* antara distribusi posterior $q(z | x)$ dan prior $p(z)$ dapat dihitung sebagai:

$$L_{KL} = -\frac{1}{2} \sum_{j=1}^d (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2) \quad (4)$$

Keterangan:

- d = dimensi ruang laten
- μ_j = rata-rata dimensi ke- j dari distribusi *posterior*
- σ_j^2 = varians dimensi ke- j dari distribusi *posterior*

Pada penelitian ini, *reconstruction loss* dihitung hanya pada *rating* yang tersedia, mengabaikan sel kosong dalam matriks interaksi yang belum diberi *rating*. Pendekatan ini disebut *masked reconstruction loss* dan diformulasikan sebagai:

$$L_{recon} = (\sum_{\{(u,i) \in \Omega\}} (r_{ui} - \hat{r}_{ui})^2) / |\Omega| \quad (5)$$

Keterangan:

- Ω = himpunan pasangan *user item* yang memiliki *rating*
- r_{ui} = *rating* asli
- $\hat{r}_{ui}$ = prediksi rekonstruksi *decoder*

Pendekatan *masked loss* mencegah model belajar dari nilai kosong yang bukan merupakan preferensi nyata pengguna, sehingga representasi laten yang dihasilkan lebih akurat mencerminkan pola interaksi yang sebenarnya. Total loss VAE pada penelitian ini adalah:

$$L_{VAE} = L_{recon} + \beta \cdot KL(q(z|x)||p(z)) \quad (6)$$

Untuk mencegah *posterior collapse* yaitu kondisi di mana model mengabaikan z dan hanya mengandalkan decoder (Wang et al., 2025), nilai β dinaikkan secara bertahap dari 0 menuju nilai target selama pelatihan menggunakan teknik KL annealing (Liang et al., 2024).

Sama seperti *autoencoder* pada umumnya, VAE memiliki tiga komponen utama (Liang et al., 2024):

- Lapisan *Encoder*

Lapisan *Encoder* berfungsi mengubah data masukan menjadi parameter distribusi laten. *Encoder* menerima vektor rating pengguna dan menghasilkan μ dan $\log(\sigma^2)$ dari distribusi Gaussian laten.

- Lapisan *Sampling*

Lapisan *Sampling* memungkinkan *backpropagation* melalui proses sampling menggunakan *reparameterization trick*. Sampel z diambil dari distribusi Gaussian menggunakan rumus:

$$z = \mu + \sigma \cdot \varepsilon \quad (7)$$

Keterangan:

- $\varepsilon \sim N(0, I)$ = sampel acak dari distribusi normal standar
- μ = rata-rata distribusi
- σ = standar deviasi distribusi

- Lapisan *Decoder*

Lapisan Decoder merekonstruksi data dari representasi laten z menggunakan lapisan *dense* dengan fungsi aktivasi *sigmoid* untuk menghasilkan prediksi rating dalam rentang 0 - 1.

2.1.4 Regularized Singular Value Decomposition (RSVD)

Singular Value Decomposition (SVD) merupakan teknik faktorisasi matriks yang memecah sebuah matriks menjadi komponen-komponen yang lebih sederhana. Dalam konteks sistem rekomendasi, SVD digunakan untuk mendekomposisi matriks rating pengguna-item menjadi representasi laten berdimensi rendah yang menangkap pola preferensi tersembunyi (Alshbanat et al., 2025). Matriks A berukuran $m \times n$ yang didekomposisi dengan SVD dinyatakan sebagai:

$$A = U\Sigma V^T \quad (8)$$

Keterangan:

- U = matriks ortogonal berukuran $m \times r$ yang memuat vektor eigen dari $M^T M$

- Σ = matriks diagonal berukuran $r \times r$ berisi nilai-nilai singular non-negatif
- V^T = matriks ortogonal berukuran $r \times n$ yang memuat vektor eigen dari MM^T

Namun, SVD klasik tidak dapat diterapkan langsung pada matriks rating yang sparse karena memerlukan matriks lengkap. Untuk mengatasi hal ini sekaligus mencegah *overfitting*, diterapkan regularisasi L2 sehingga menghasilkan *Regularized Singular Value Decomposition* (RSVD). Selain itu, implementasi RSVD pada sistem rekomendasi diperkaya dengan *bias terms* berupa rata-rata global (μ), bias pengguna (b_u), dan bias item (b_i) untuk menangkap kecenderungan rating yang bersifat sistematis di luar faktor laten (Alshbanat et al., 2025). Prediksi rating dari pengguna u terhadap item i diformulasikan sebagai:

$$\hat{r} = \mu + b_u + b_i + (U\Sigma V^T)_{ui} \quad (9)$$

Keterangan:

- μ = rata-rata *rating* global
- b_u = bias pengguna u
- b_i = bias *item* i
- $(U\Sigma V^T)_{ui}$ = komponen faktor laten

Fungsi *loss* yang dioptimalkan menggunakan regularisasi L2 atau *Ridge Regularization* adalah:

$$Loss = \sum_{i,j} (Z_{ij} - \hat{r}_{ij})^2 + \lambda (\|U\|_F^2 + \|V\|_F^2) \quad (10)$$

Keterangan:

- Z = matriks rating asli
- λ = bobot penalti regularisasi
- $\|X\|_F^2$ = kuadrat norma *Frobenius* dari x

Norma Frobenius dari matriks A berukuran $m \times n$ dihitung sebagai:

$$\|A\|_F = \sqrt{(\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2)} \quad (11)$$

Keterangan:

- a_{ij} = elemen baris ke- i kolom ke- j dari matriks A

Untuk mengoptimalkan fungsi *loss*, digunakan metode *Stochastic Gradient Descent* (SGD) yang memperbarui parameter secara iteratif menggunakan informasi gradien dari satu data rating pada setiap langkah pembaruan (Alshbanat et al., 2025). Berbeda dengan *gradient descent* biasa yang menghitung gradien dari seluruh data sekaligus, SGD melakukan pembaruan hanya pada pasangan (u, i) yang memiliki rating aktual ($Z[u, i] > 0$), sehingga sel-sel kosong dalam matriks *sparse* tidak mempengaruhi gradien. Aturan pembaruan untuk setiap parameter adalah:

$$U[u, k] \leftarrow U[u, k] + \eta \cdot (e_{ui} \cdot \Sigma[k, k] \cdot V[i, k] - \lambda \cdot U[u, k]) \quad (12)$$

$$V[i, k] \leftarrow V[i, k] + \eta \cdot (e_{ui} \cdot \Sigma[k, k] \cdot U[u, k] - \lambda \cdot V[i, k]) \quad (13)$$

$$\Sigma[k, k] \leftarrow \Sigma[k, k] + \eta \cdot (e_{ui} \cdot U[u, k] \cdot V[i, k]) \quad (14)$$

dimana η adalah *learning rate* dan error rekonstruksi e_{ui} didefinisikan sebagai:

$$e_{ui} = Z[u, i] - \hat{r}_{ui} \quad (15)$$

Terlihat pada rumus (12) dan (13) bahwa pembaruan U dan V mengandung suku penalti regularisasi λ , sedangkan pada rumus (14) pembaruan Σ tidak mengandung suku penalti tersebut. Keputusan mengenai penerapan regularisasi pada Σ serta strategi inisialisasinya dibahas lebih lanjut pada bagian metodologi.

2.1.5 Sistem Rekomendasi Hibrid

Sistem rekomendasi hibrid merupakan pendekatan yang menggabungkan dua atau lebih teknik rekomendasi untuk memanfaatkan kelebihan masing-masing teknik secara bersamaan (Sami et al., 2024). Pendekatan ini dilatarbelakangi oleh kenyataan bahwa setiap teknik rekomendasi tunggal memiliki keterbatasan tersendiri. Collaborative filtering rentan terhadap masalah cold-start dan data sparsity, sedangkan metode berbasis konten bergantung pada ketersediaan atribut item yang lengkap. Dengan menggabungkan beberapa teknik, sistem rekomendasi hibrid dapat saling menutupi kelemahan masing-masing sehingga menghasilkan prediksi yang lebih akurat (Sami et al., 2024).

Salah satu strategi hibridisasi yang umum digunakan adalah weighted hybridization, di mana prediksi akhir diperoleh dari kombinasi linear tertimbang (weighted linear combination) dari prediksi yang dihasilkan oleh masing-masing model penyusun. Pada strategi ini, setiap model diberi bobot yang mencerminkan

proporsi kontribusinya terhadap prediksi akhir, dengan total seluruh bobot bernilai satu. (Sami et al., 2024) menggunakan strategi ini untuk menggabungkan prediksi dari beberapa model rekomendasi, dengan prediksi akhir dirumuskan sebagai kombinasi linear dari skor prediksi masing-masing model yang dikalikan dengan bobot yang sesuai. Untuk sistem hibrid yang menggabungkan dua model, formula prediksi *weighted hybridization* dapat dituliskan sebagai:

$$\hat{r}_{ui} = \alpha \cdot \hat{r}_{ui}^A + \beta \cdot \hat{r}_{ui}^B \quad (16)$$

dengan *constraint*:

$$\alpha + \beta = 1, \alpha, \beta \in [0,1] \quad (17)$$

Keterangan:

- $\hat{r}_{ui}$ = prediksi *rating* pengguna u terhadap item i
- $\hat{r}_{ui}^A$ = prediksi *rating* model A untuk pengguna u terhadap item i
- $\hat{r}_{ui}^B$ = prediksi *rating* model B untuk pengguna u terhadap item i
- α = bobot model A
- β = bobot model B

Nilai bobot α dan β dapat ditentukan secara empiris melalui pencarian pada rentang 0 – 1 yang meminimalkan metrik kesalahan prediksi pada data validasi. Pendekatan ini memastikan proporsi kontribusi masing-masing model ditentukan berdasarkan performa aktual pada data, sehingga kombinasi yang dihasilkan bersifat adaptif terhadap karakteristik dataset yang digunakan.

2.1.6 Mean Absolute Error, Mean Squared Error, Root Mean Squared Error

Evaluasi performa sistem rekomendasi berbasis prediksi rating dilakukan menggunakan metrik kesalahan yang mengukur selisih antara nilai rating aktual dengan nilai rating yang diprediksi oleh model. Tiga metrik yang digunakan dalam penelitian ini adalah *Mean Absolute Error* (MAE), *Mean Squared Error* (MSE), dan *Root Mean Squared Error* (RMSE) (Bobadilla et al., 2021).

Mean Absolute Error (MAE) mengukur rata-rata nilai absolut dari selisih antara *rating* aktual dan *rating* prediksi. MAE memberikan gambaran langsung seberapa jauh prediksi model menyimpang dari nilai sebenarnya dalam satuan yang sama dengan skala *rating*. MAE diformulasikan sebagai:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (18)$$

Mean Squared Error (MSE) mengukur rata-rata kuadrat selisih antara *rating* aktual dan *rating* prediksi. Pengkuadratan selisih menyebabkan MSE memberikan penalti yang lebih besar terhadap kesalahan prediksi yang jauh dari nilai aktual dibandingkan MAE. MSE diformulasikan sebagai:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (19)$$

Root Mean Squared Error (RMSE) merupakan akar kuadrat dari MSE. RMSE mengembalikan nilai kesalahan ke satuan yang sama dengan skala *rating* asli sehingga lebih mudah diinterpretasikan, sekaligus mempertahankan sifat MSE yang memberikan penalti lebih besar pada kesalahan besar. RMSE diformulasikan sebagai:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (20)$$

Keterangan:

- n = jumlah pasangan *rating* aktual dan prediksi yang dievaluasi
- y_i = nilai *rating* aktual k-i
- $\hat{y}_i$ = nilai *rating* prediksi k-i

Ketiga metrik ini bernilai non-negatif dan semakin kecil nilainya semakin baik performa model. RMSE lebih sensitif terhadap kesalahan besar karena sifat pengkuadratan yang diwarisi dari MSE, sedangkan MAE memberikan gambaran kesalahan rata-rata yang lebih merata tanpa memberikan penalti berlebih pada outlier (Bobadilla et al., 2021).

2.1.7 K-Fold Cross Validation

K-fold cross validation merupakan teknik evaluasi model yang membagi seluruh data menjadi k bagian yang berukuran sama, kemudian melatih dan menguji model sebanyak k kali secara bergantian (Abedin et al., 2026). Teknik ini mengatasi keterbatasan metode *holdout* biasa yang hanya melakukan satu kali pembagian data latih dan data uji, sehingga hasil evaluasinya bergantung pada bagaimana data kebetulan terbagi. Dengan menggunakan seluruh data untuk pelatihan dan

pengujian secara bergantian, *k-fold cross validation* menghasilkan estimasi performa model yang lebih stabil dan dapat direplikasi.

Prosedur *k-fold cross validation* diawali dengan mengacak seluruh data, kemudian membaginya menjadi *k fold* yang berukuran sama. Pada setiap iterasi ke-*i*, *fold* ke-*i* digunakan sebagai data uji sementara *k – 1 fold* sisanya digunakan sebagai data latih. Model dilatih dari awal pada setiap iterasi menggunakan data latih yang berbeda, kemudian dievaluasi pada data uji *fold* ke-*i*. Proses ini diulang sebanyak *k* kali hingga setiap *fold* telah digunakan tepat satu kali sebagai data uji. Metrik evaluasi akhir dihitung sebagai rata-rata dari metrik yang diperoleh di setiap *fold*, yaitu:

$$\bar{M} = \frac{1}{k} \sum_{i=1}^k M_i \quad (21)$$

Keterangan:

- $\bar{M}$ = nilai rata-rata metrik evaluasi dari *k fold*
- M_i = nilai metrik evaluasi pada *fold* ke-*i*
- K = jumlah *fold*

Pemilihan nilai *k* berpengaruh pada keseimbangan antara *bias* dan *variance* dari estimasi performa. Nilai *k* yang terlalu kecil menghasilkan data latih yang lebih sedikit sehingga estimasi cenderung *bias*, sedangkan nilai *k* yang terlalu besar dapat meningkatkan *variance* antar *fold* (Abedin et al., 2026). Nilai *k* = 10 merupakan salah satu pilihan yang umum digunakan dalam penelitian *machine learning* (Abedin et al., 2026), meskipun pemilihan nilai *k* yang optimal bergantung pada karakteristik data dan model yang digunakan.

2.1.8 Uji-t Berpasangan (*Paired t-test*)

Uji-t berpasangan (*paired t-test*) merupakan salah satu metode pengujian hipotesis yang digunakan ketika data yang dibandingkan berasal dari subjek yang sama namun diukur dalam dua kondisi yang berbeda (Rahmani et al., 2025). Metode ini membandingkan rata-rata dua kelompok data yang berpasangan untuk menentukan apakah perbedaan yang teramati signifikan secara statistik atau hanya merupakan variasi acak. Dua pengukuran dikatakan berpasangan apabila setiap nilai pada kelompok pertama memiliki pasangan yang bersesuaian pada kelompok kedua dari subjek yang sama.

Uji-t berpasangan dirumuskan dengan hipotesis nol (H_0) dan hipotesis alternatif (H_1) sebagai berikut (Rahmani et al., 2025):

- $H_0: \mu_1 - \mu_2 = 0$ (tidak terdapat perbedaan signifikan antara rata-rata kedua kelompok)
- $H_1: \mu_1 - \mu_2 \neq 0$ (terdapat perbedaan signifikan antara rata-rata kedua kelompok)

Statistik uji t dihitung menggunakan selisih antar pasangan data dengan rumus (Rahmani et al., 2025):

$$t = \frac{\bar{d}}{s_d/\sqrt{n}} \quad (22)$$

Dengan derajat kebebasan

$$df = n - 1 \quad (23)$$

Keterangan:

- $\bar{d}$ = rata-rata selisih antara dua kelompok pengukuran berpasangan
- s_d = simpang baku dari selisih
- n = jumlah pasangan pengukuran
- df = derajat kebebasan

2.2 Tinjauan Empiris

2.2.1 Deep Variational Models for Collaborative Filtering-Based Recommender Systems (Bobadilla et al., 2021)

Pada penelitian ini, penulis mengembangkan sistem rekomendasi berbasis *collaborative filtering* menggunakan beberapa varian model *variational autoencoder* (VAE), termasuk VAE standar, *Bernoulli VAE*, dan *Gaussian VAE*. Tujuan penelitian ini adalah untuk mengevaluasi apakah penambahan tahap variasional pada model *autoencoder* yang sudah ada dapat meningkatkan akurasi prediksi rating dibandingkan model *deep matrix factorization* konvensional. Penulis berargumen bahwa VAE menghasilkan ruang laten yang lebih teratur dan kontinu dibandingkan *autoencoder* biasa, sehingga lebih mampu menangkap preferensi pengguna secara probabilistik. Eksperimen dilakukan menggunakan beberapa dataset *benchmark* sistem rekomendasi. Evaluasi dilakukan menggunakan

metrik MAE dan RMSE dengan skema pengujian *train-test split*. Hasil penelitian menunjukkan bahwa model-model berbasis VAE secara konsisten mengungguli model *deep matrix factorization* pada seluruh dataset yang diuji. Penelitian ini menjadi landasan empiris penggunaan VAE dalam konteks *collaborative filtering* pada penelitian yang dilakukan. Namun, penelitian ini tidak mengeksplorasi penggabungan VAE dengan teknik dekomposisi matriks seperti RSVD, dan tidak menerapkan *cross-validation* dalam proses evaluasinya.

2.2.2 A Deep Autoencoder-Based Hybrid Recommender System (Bougteb et al., 2022)

Pada penelitian ini, penulis membangun sebuah sistem rekomendasi hibrid yang menggabungkan *deep autoencoder* dengan SVD++ untuk mengatasi masalah *data sparsity* dan skalabilitas pada matriks rating pengguna-item. *Deep autoencoder* dengan 5 *hidden layer* digunakan untuk mempelajari representasi laten berdimensi rendah dari matriks rating yang *sparse* dan merekonstruksi *rating* yang hilang, sementara SVD++ dijalankan secara paralel untuk mempertahankan informasi korelasi antara faktor laten pengguna dan item, termasuk *implicit feedback*. Prediksi akhir dihasilkan melalui *weighted average* dari keluaran kedua komponen tersebut, dengan bobot optimal $w = 0,9$ untuk SVD++ dan $1 - w = 0,1$ untuk *deep autoencoder*. Eksperimen dilakukan menggunakan tiga dataset, yaitu MovieLens 100K, MovieLens 1M, dan Book-Crossing. Evaluasi dilakukan menggunakan metrik RMSE dan MAE dengan skema *train-test split* 75:25. Pada dataset MovieLens 100K, hasil yang diperoleh adalah RMSE sebesar 0,992 dan MAE sebesar 0,777, mengungguli dua model *baseline* hibrid yang diuji. Penelitian ini secara langsung memotivasi pendekatan yang digunakan dalam penelitian yang dilakukan, yaitu mengombinasikan model berbasis *autoencoder* dengan teknik dekomposisi matriks melalui *weighted average*. Meskipun demikian, terdapat perbedaan mendasar yaitu penelitian Bougteb et al. menggunakan *autoencoder* deterministik dan SVD++ yang memperhitungkan *implicit feedback*, sedangkan penelitian yang dilakukan menggunakan *variational autoencoder* dengan regularisasi KL *annealing* dan RSVD dengan *bias terms* yang dioptimasi

menggunakan SGD pada data *explicit feedback*. Selain itu, penelitian ini tidak menerapkan *cross-validation* dalam proses evaluasinya.

2.2.3 Implementasi Regularized Singular Value Decomposition dalam Sistem Rekomendasi Buku Collaborative Filtering (Putra & Santiyasa, 2025)

Pada penelitian ini, penulis membangun sistem rekomendasi buku menggunakan metode *collaborative filtering* dengan RSVD sebagai algoritma prediksi rating. RSVD digunakan untuk mendekomposisi matriks rating pengguna-item ke dalam representasi faktor laten dengan meminimalkan *regularized squared error* melalui SGD. Penulis juga memanfaatkan nilai faktor laten item yang dihasilkan RSVD untuk menghitung *cosine similarity* antar buku sebagai dasar rekomendasi non-personal. Data yang digunakan adalah data primer yang dikumpulkan langsung dari siswa SMP Negeri 3 Kediri, terdiri dari 886 buku, 332 pengguna, dan 124.841 data rating dengan skala 1–5. Evaluasi dilakukan menggunakan metrik MAE dan RMSE dengan skema *holdout* 80:20. Hasil terbaik yang diperoleh adalah MAE sebesar 0,478 dan RMSE sebesar 0,686 dengan kombinasi 100 faktor laten dan nilai regularisasi 0,05. Penelitian ini juga menunjukkan bahwa penambahan regularisasi secara signifikan meningkatkan akurasi prediksi dibandingkan RSVD tanpa regularisasi. Penelitian ini memperkuat justifikasi penggunaan RSVD dalam penelitian yang dilakukan, khususnya dalam hal efektivitas regularisasi terhadap akurasi prediksi. Namun, penelitian ini berfokus pada domain rekomendasi buku dengan data primer berskala kecil, tidak mengeksplorasi penggabungan RSVD dengan model *deep learning* seperti VAE, dan tidak menerapkan *cross-validation* dalam evaluasinya.

2.2.4 Hybrid Recommendation System for Movies Using Artificial Neural Network (Sharma & Kumar Shakya, 2024)

Pada penelitian ini, penulis mengembangkan sistem rekomendasi film hibrid yang mengintegrasikan empat komponen: SVD yang telah ditingkatkan untuk prediksi rating berbasis *collaborative filtering*, *Content Driven* KNN berbasis *cosine similarity* untuk deteksi kemiripan film, IKSOM berbasis *EISEN cosine correlation distance* untuk pengklusteran, dan K-Means++ dengan metode *silhouette* untuk kategorisasi data film. Prediksi akhir dihasilkan melalui kombinasi

skor dari keempat komponen tersebut. Data yang digunakan adalah MovieLens 25M yang berisi 25 juta rating dari 162.541 pengguna untuk 62.423 film. Evaluasi dilakukan menggunakan metrik RMSE dan MAE dengan skema *train-test split* 75:25. Hasil yang diperoleh menunjukkan RMSE sebesar 0,414 dan MAE sebesar 0,279, mengungguli berbagai metode *baseline*. Penelitian ini menunjukkan bahwa pendekatan hibrid yang menggabungkan SVD dengan metode lain dapat meningkatkan akurasi prediksi secara signifikan pada domain rekomendasi film.

2.2.5 A Deep Learning Based Hybrid Recommendation Model for Internet Users (Sami et al., 2024)

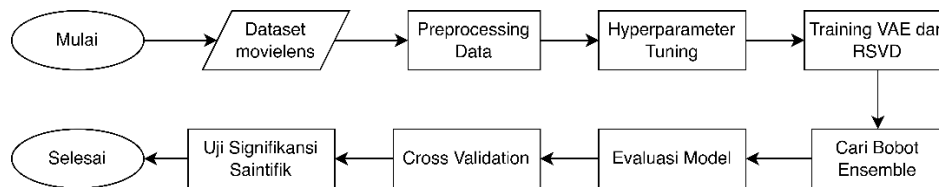
Pada penelitian ini, penulis mengembangkan model sistem rekomendasi hibrid yang mengintegrasikan *user-based* dan *item-based collaborative filtering*, *neural collaborative filtering* (NCF), *recurrent neural network* (RNN), dan *content-based filtering* berbasis TF-IDF. Prediksi akhir dihasilkan melalui kombinasi tertimbang dari keluaran seluruh komponen tersebut. Data yang digunakan adalah MovieLens 100K yang berisi 100.000 rating dari 943 pengguna untuk 1.682 film, yang merupakan dataset yang sama dengan penelitian yang dilakukan. Evaluasi dilakukan menggunakan metrik RMSE, MAE, Precision, dan Recall dengan skema *train-test split* 80:20. Hasil yang diperoleh setelah *hyperparameter tuning* adalah RMSE sebesar 0,7723 dan MAE sebesar 0,6018, mengungguli berbagai model *baseline* yang diuji pada dataset yang sama.

BAB III

ANALISIS DAN PERANCANGAN SISTEM

3.1 Desain Sistem

Penelitian ini mengembangkan sistem rekomendasi film berbasis prediksi rating menggunakan pendekatan hybrid ensemble yang menggabungkan model Variational Autoencoder (VAE) dan Regularized Singular Value Decomposition (RSVD). Prediksi dari kedua model digabungkan melalui mekanisme weighted average ensemble untuk menghasilkan prediksi rating akhir. Secara keseluruhan, penelitian ini mencakup delapan tahap utama sebagaimana diilustrasikan pada Gambar 3.1.



Gambar 3.1. Metode Penelitian

Penelitian dimulai dengan pengumpulan dataset MovieLens 100K, dilanjutkan dengan *preprocessing* data yang mencakup normalisasi rating dan pembagian data menjadi partisi latih, validasi, dan uji. Selanjutnya dilakukan *hyperparameter tuning* untuk memperoleh konfigurasi optimal bagi kedua model, yang kemudian digunakan pada tahap pelatihan. Bobot *ensemble* dicari pada data validasi dan diterapkan saat evaluasi pada data uji. Untuk mengukur stabilitas dan generalisasi model secara lebih komprehensif, evaluasi dilanjutkan dengan *10-fold cross-validation* pada keseluruhan dataset, diikuti uji signifikansi statistik menggunakan *paired t-test*.

3.2 Dataset

Data yang digunakan dalam penelitian ini adalah dataset MovieLens 100K yang diperoleh secara sekunder dari situs *grouplens.org*. Dataset ini merupakan salah satu dataset *benchmark* yang umum digunakan dalam penelitian sistem rekomendasi (Sami et al., 2024). MovieLens 100K terdiri dari 100.000 data rating yang diberikan oleh 943 pengguna terhadap 1.682 film, dengan skala rating 1

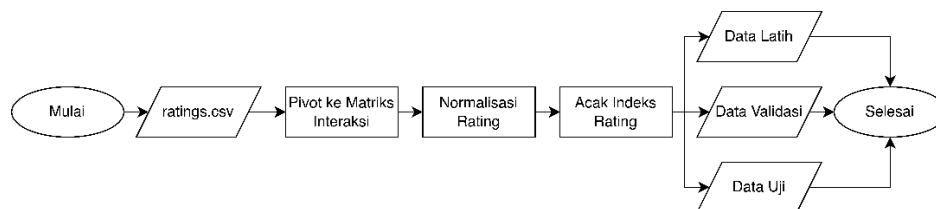
hingga 5. File yang digunakan dalam penelitian ini adalah *ratings.csv* yang memuat kolom *userId*, *movieId*, *rating*, dan *timestamp* sebagaimana ditunjukkan pada Tabel 3.1. Kolom *timestamp* tidak digunakan dalam penelitian ini karena model yang dikembangkan tidak mempertimbangkan dimensi waktu dalam prediksi rating.

Tabel 3.1. Contoh Data ratings.csv

userId	movieId	rating	timestamp
1	1	4.0	964982703
1	3	4.0	964981247
1	6	4.0	964982224
1	47	5.0	964983815
1	50	5.0	964982931

3.3 Preprocessing Data

Preprocessing data dilakukan dalam tiga tahap utama, yaitu pembangunan matriks interaksi pengguna-film, normalisasi rating, dan pembagian data. Alur keseluruhan tahap *preprocessing* diilustrasikan pada Gambar 3.2.



Gambar 3.2. Alur Preprocessing Data

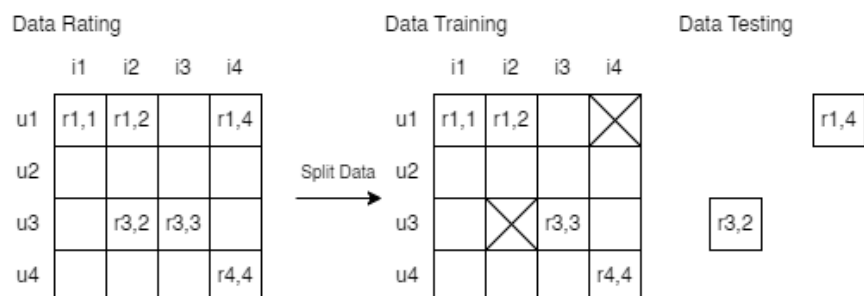
Tahap pertama adalah pembangunan matriks interaksi pengguna-film. Data pada *ratings.csv* dipivot menjadi matriks berukuran 943×1.682 , di mana baris merepresentasikan pengguna dan kolom merepresentasikan film. Setiap sel pada matriks berisi nilai rating yang diberikan pengguna terhadap film yang bersangkutan, sedangkan sel yang tidak memiliki rating diisi dengan nilai 0. Contoh matriks interaksi ditunjukkan pada Tabel 3.2 dimana indeks baris adalah pengguna dan kolom adalah *film*.

Tabel 3.2. Contoh Matriks Interaksi

	1	2	3	4	5	6	...
1	4.0	0.0	4.0	0.0	0.0	4.0	...
2	0.0	0.0	0.0	0.0	0.0	0.0	...
3	0.0	0.0	0.0	0.0	0.0	0.0	...
4	0.0	0.0	0.0	0.0	0.0	0.0	...
...	...	...	...	...	...	...	...

Tahap kedua adalah normalisasi rating. Seluruh nilai rating pada matriks dinormalisasi ke rentang $[0, 1]$ dengan membagi setiap nilai dengan nilai rating maksimum (5.0). Normalisasi ini diperlukan agar nilai masukan sesuai dengan fungsi aktivasi *sigmoid* pada lapisan *output decoder* VAE yang menghasilkan keluaran dalam rentang yang sama. Sel kosong yang bernilai 0 tetap bernilai 0 setelah normalisasi sehingga dapat dibedakan dari rating terendah yang bernilai 0.2.

Tahap ketiga adalah pembagian data menggunakan metode *hold-out* pada level rating individual. Seluruh 100.000 rating diacak dengan *random seed* 42 untuk memastikan reproduksibilitas, kemudian dibagi menjadi tiga partisi: 70% data latih (70.000 rating) untuk melatih VAE dan RSVD, 10% data validasi (10.000 rating) untuk menentukan bobot *ensemble* optimal, dan 20% data uji (20.000 rating) untuk evaluasi performa akhir model. Pembagian dilakukan pada level rating individual, bukan pada level pengguna, sehingga setiap pengguna dapat muncul di ketiga partisi sekaligus. Ilustrasi pembagian data ditunjukkan pada Gambar 3.3.

**Gambar 3.3. Ilustrasi Pembagian Data**

3.4 Perancangan Model

3.4.1 Variational Autoencoder (VAE)

VAE yang dikembangkan dalam penelitian ini terdiri dari tiga komponen utama: *encoder*, *sampling*, dan *decoder*. *Encoder* menerima masukan berupa vektor rating pengguna dari matriks interaksi berukuran 1.682 dimensi. Sebelum melewati lapisan tersembunyi, masukan dikenai *dropout* untuk mencegah *overfitting*. Lapisan tersembunyi *encoder* terdiri dari dua lapisan *dense* dengan fungsi aktivasi ReLU, diikuti dua lapisan keluaran yang masing-masing menghasilkan vektor μ (z_mean) dan $\log \sigma^2$ (z_log_var) yang merepresentasikan rata-rata dan log varians distribusi ruang laten.

Komponen *sampling* mengimplementasikan *reparameterization trick* agar proses pengambilan sampel dari ruang laten tetap dapat diturunkan secara matematis (*differentiable*) sehingga *backpropagation* dapat berjalan. Titik sampel laten z diperoleh melalui persamaan (7).

Decoder menerima vektor laten z dan merekonstruksinya kembali menjadi vektor prediksi rating berukuran 1.682 dimensi. Lapisan tersembunyi *decoder* merupakan kebalikan simetris dari *encoder*, yaitu dua lapisan *dense* dengan fungsi aktivasi ReLU. Lapisan keluaran *decoder* menggunakan fungsi aktivasi *sigmoid* yang menghasilkan nilai dalam rentang $[0, 1]$, sesuai dengan skala rating yang telah dinormalisasi.

Fungsi *loss* yang digunakan adalah fungsi *loss* Beta-VAE yang merupakan kombinasi dari *reconstruction loss* dan *KL divergence loss* menggunakan persamaan (6). *Reconstruction loss* dihitung menggunakan *masked MSE* yang hanya mempertimbangkan sel matriks yang memiliki nilai rating, sehingga model tidak belajar dari nilai 0 yang merepresentasikan rating yang tidak ada menggunakan persamaan (5). *KL divergence loss* meregulasi distribusi ruang laten agar mendekati distribusi normal standar menggunakan persamaan (4).

Hyperparameter β dikontrol secara dinamis melalui mekanisme *KL annealing* untuk mencegah *posterior collapse*, yaitu kondisi di mana *encoder* mengabaikan masukan dan menghasilkan distribusi laten yang identik untuk semua pengguna (Wang et al., 2025). Pada awal pelatihan β ditetapkan sebesar 0 sehingga

model fokus meminimalkan *reconstruction loss* terlebih dahulu. Nilai β kemudian dinaikkan secara linear hingga mencapai nilai target yang diperoleh dari hasil *hyperparameter tuning*. Model dioptimasi menggunakan algoritma Adam. Nilai *learning rate*, *batch size*, *dropout rate*, dimensi ruang laten, dan jumlah unit pada setiap lapisan tersembunyi ditentukan melalui proses *hyperparameter tuning*.

3.4.2 Regularized Singular Value Decomposition (RSVD)

RSVD yang dikembangkan dalam penelitian ini mengimplementasikan faktorisasi matriks dengan tiga komponen utama yaitu bias global (μ), bias pengguna (b_u), dan bias item (b_i), di samping matriks faktor laten U , Σ , dan V . Prediksi rating pengguna u terhadap item i diformulasikan dengan persamaan (9). Parameter model diinisialisasi sebagai berikut:

- μ dihitung sebagai rata-rata global dari seluruh rating pada data latih
- b_u dan b_i diinisialisasi dengan nilai 0
- matriks U dan V diinisialisasi dengan nilai acak dari distribusi normal dengan skala 0.1
- matriks Σ diinisialisasi sebagai matriks identitas.

Inisialisasi Σ sebagai matriks identitas dipilih untuk mencegah *vanishing gradient* pada epoch-epoch awal pelatihan, karena inisialisasi dengan nilai mendekati 0 menyebabkan gradien U dan V ikut terhambat secara kaskade.

Model dilatih menggunakan *Stochastic Gradient Descent* (SGD) dengan fungsi *loss* yang merupakan kombinasi *reconstruction loss* dan penalti regularisasi L2 menggunakan persamaan (10). Pembaruan parameter dilakukan hanya pada pasangan (u, i) yang memiliki rating aktual, sehingga sel kosong pada matriks interaksi tidak mempengaruhi proses pelatihan. Aturan pembaruan untuk setiap parameter adalah dilakukan melalui persamaan (12) – (15). Perlu diperhatikan bahwa regularisasi L2 diterapkan pada U , V , b_u , dan b_i , namun tidak diterapkan pada Σ . Keputusan desain ini diambil berdasarkan eksperimen awal yang menunjukkan bahwa regularisasi pada Σ menyebabkan nilai-nilainya mendekati 0 secara kaskade sehingga menghambat gradien U dan V .

Untuk mencegah *overfitting*, RSVD menggunakan mekanisme *early stopping* berbasis *validation MSE*. Sebelum pelatihan dimulai, sebanyak 10% dari

rating data latih disisihkan sebagai *validation set* internal menggunakan metode *hold-out*. Pada setiap akhir epoch, model mengevaluasi *validation MSE* dan menyimpan *snapshot* bobot terbaik. Pelatihan dihentikan apabila tidak terdapat perbaikan *validation MSE* selama sejumlah epoch yang ditentukan oleh parameter *patience*. Setelah pelatihan selesai, bobot dikembalikan ke *snapshot* terbaik. *Validation MSE* digunakan sebagai kriteria *early stopping* karena *training MSE* selalu menurun secara monoton sehingga tidak mencerminkan performa model pada data yang belum dilihat. Nilai *learning rate*, koefisien regularisasi λ , jumlah faktor laten, dan *patience* ditentukan melalui proses *hyperparameter tuning*.

3.4.3 Model Hybrid dengan Weighted Average Ensemble

Penelitian ini mengembangkan model *hybrid* yang menggabungkan prediksi VAE dan RSVD melalui mekanisme *weighted average ensemble*. Pendekatan *hybrid* dipilih karena VAE dan RSVD memodelkan preferensi pengguna dari sudut pandang yang berbeda, VAE mempelajari representasi laten melalui distribusi probabilistik, sedangkan RSVD mengdekomposisi matriks rating melalui faktorisasi laten deterministik dengan bias terms.

Sebelum digabungkan, prediksi dari kedua model didenormalisasi ke skala rating asli (1–5) dan dibatasi (*clipping*) agar tidak melampaui rentang tersebut. Prediksi *hybrid* diformulasikan dengan persamaan (16) dan (17). Nilai α optimal ditentukan melalui *grid search* pada data validasi (10%) yang telah disisihkan pada tahap *preprocessing*. Pencarian dilakukan dengan mencoba seluruh nilai α dari 0.00 hingga 1.00 dengan langkah 0.01, menghasilkan 101 kandidat nilai. Untuk setiap nilai α , RMSE dihitung antara prediksi *hybrid* dan rating aktual pada data validasi. Nilai α yang menghasilkan RMSE terendah di data validasi dipilih sebagai bobot final dan digunakan untuk menghitung prediksi *hybrid* pada data uji. Dengan pendekatan ini, pemilihan α sepenuhnya didasarkan pada data validasi sehingga evaluasi pada data uji tetap tidak bias.

3.5 Hyperparameter Tuning

Hyperparameter tuning dilakukan secara terpisah untuk VAE dan RSVD menggunakan data latih (70%). Tujuan dari tahap ini adalah menemukan

konfigurasi hyperparameter yang menghasilkan performa terbaik sebelum model dilatih secara final dan dievaluasi.

3.5.1 *Hyperparameter Tuning VAE*

Tuning VAE dilakukan menggunakan *framework* Optuna dengan algoritma *Tree-structured Parzen Estimator* (TPE). TPE membangun model probabilistik dari hasil *trial* sebelumnya untuk menentukan kombinasi hyperparameter berikutnya yang paling menjanjikan agar pencarian menjadi lebih efisien. Setiap *trial* melatih VAE selama 30 *epoch* dan menggunakan *reconstruction loss* pada epoch terakhir sebagai kriteria seleksi. *Reconstruction loss* dipilih sebagai kriteria bukan *total loss* agar hasil tidak bias terhadap nilai β yang kecil, karena *total loss* yang rendah dapat dicapai sekadar dengan meminimalkan kontribusi KL divergence melalui β kecil tanpa benar-benar meningkatkan kualitas rekonstruksi. Total *trial* yang dijalankan adalah 200. Ruang pencarian hyperparameter VAE ditunjukkan pada Tabel 3.3.

Tabel 3.3. Ruang Pencarian *Hyperparameter VAE*

<i>Hyperparameter</i>	Nama Variabel	Ruang Pencarian
Dimensi ruang laten	latent_dim	20, 50, 100, 200
Kecepatan belajar	learning_rate	0.0001, 0.0005, 0.001, 0.005, 0.01
Jumlah sampel per iterasi	batch_size	64, 128, 256
Laju <i>dropout</i> pada lapisan tersembunyi	dropout_rate	0.1, 0.2, 0.3, 0.4
Jumlah unit pada setiap lapisan tersembunyi	hidden_dims	[512, 256], [512, 256, 128], [1024, 512], [1024, 512, 256]
Bobot KL divergence pada fungsi <i>loss</i> Beta-VAE	beta	0.01, 0.05, 0.1, 0.3, 0.5, 0.8, 1.0

Pemilihan rentang kandidat untuk setiap hyperparameter VAE didasarkan pada karakteristik dataset dan arsitektur model.

- `latent_dim` divariasikan dari 20 hingga 200 untuk mengeksplorasi trade-off antara kapasitas representasi ruang laten dan risiko *overfitting*. Dimensi terlalu kecil tidak mampu menangkap kompleksitas preferensi pengguna, sedangkan dimensi terlalu besar berpotensi mempelajari *noise*.
- `learning_rate` mencakup rentang lebar dari 0.0001 hingga 0.01 karena sensitivitas konvergensi VAE terhadap nilai ini cukup tinggi, terutama akibat interaksi antara *reconstruction loss* dan KL divergence.
- `batch_size` dibatasi pada nilai 64, 128, dan 256 mengingat ukuran dataset (943 pengguna). *Batch size* yang terlalu besar akan menghasilkan terlalu sedikit iterasi per epoch.
- `dropout_rate` diuji pada rentang 0.1 - 0.4 karena nilai di bawah 0.1 memberikan regularisasi yang tidak signifikan, sedangkan nilai di atas 0.4 berisiko menghilangkan terlalu banyak informasi dari vektor rating yang sudah sangat *sparse*.
- `hidden_dims` mencakup empat variasi arsitektur dua dan tiga lapisan dengan jumlah unit yang berbeda untuk mengeksplorasi kedalaman dan lebar jaringan secara bersamaan.
- `beta` mencakup rentang 0.01 hingga 1.0 karena nilai β yang terlalu besar cenderung menyebabkan *posterior collapse*, sedangkan nilai yang terlalu kecil membuat regularisasi ruang laten tidak efektif.

3.5.2 Hyperparameter Tuning RSVD

Tuning RSVD dilakukan menggunakan *exhaustive grid search* yang mencoba seluruh kombinasi dari ruang pencarian secara sistematis. Total kombinasi yang dievaluasi adalah $3 \times 4 \times 4 = 48$ kombinasi. Setiap kombinasi melatih RSVD selama 15 *epoch* dan menggunakan *total loss* (MSE + penalti L2) pada epoch terakhir sebagai kriteria seleksi. Ruang pencarian hyperparameter RSVD ditunjukkan pada Tabel 3.4.

Tabel 3.4. Ruang Pencarian Hyperparameter RSVD

<i>Hyperparameter</i>	Nama Variabel	Ruang Pencarian
Jumlah dimensi faktor laten	<code>n_factors</code>	10, 20, 50

Kecepatan belajar SGD	learning_rate	0.001, 0.005, 0.01, 0.02
Koefisien penalti regularisasi L2	lambda_reg	0.001, 0.005, 0.01, 0.02

Pemilihan rentang kandidat untuk setiap hyperparameter RSVD mempertimbangkan karakteristik dataset dan kompleksitas komputasi.

- `n_factors` diuji pada nilai 10, 20, dan 50 karena RSVD dengan jumlah faktor laten yang besar membutuhkan waktu komputasi yang jauh lebih lama akibat iterasi SGD per pasangan (u, i).
- `learning_rate` diuji pada rentang 0.001 hingga 0.02 karena SGD pada RSVD lebih sensitif terhadap nilai ini dibandingkan optimizer Adam pada VAE. Nilai terlalu besar menyebabkan divergensi, sedangkan nilai terlalu kecil memperlambat konvergensi secara signifikan.
- `lambda_reg` diuji pada rentang 0.001 hingga 0.02 untuk menyeimbangkan antara regularisasi yang cukup untuk mencegah *overfitting* dan tidak terlalu agresif sehingga menghambat kemampuan model mempelajari pola rating.

3.6 Evaluasi Model

Setelah hyperparameter optimal diperoleh melalui proses *hyperparameter tuning*, VAE dan RSVD dilatih secara final menggunakan data latih (70%). Bobot *ensemble α* kemudian ditentukan pada data validasi (10%) melalui prosedur *grid search*. Model yang telah dilatih selanjutnya dievaluasi melalui dua tahap. Tahap pertama adalah evaluasi *hold-out* pada data uji (20%) yang tidak pernah digunakan pada proses pelatihan maupun penentuan bobot *ensemble*. Tahap kedua adalah *10-fold cross-validation* pada keseluruhan dataset untuk mengukur stabilitas dan kemampuan generalisasi model secara lebih komprehensif. Signifikansi statistik perbedaan performa antar model kemudian diuji menggunakan *paired t-test*.

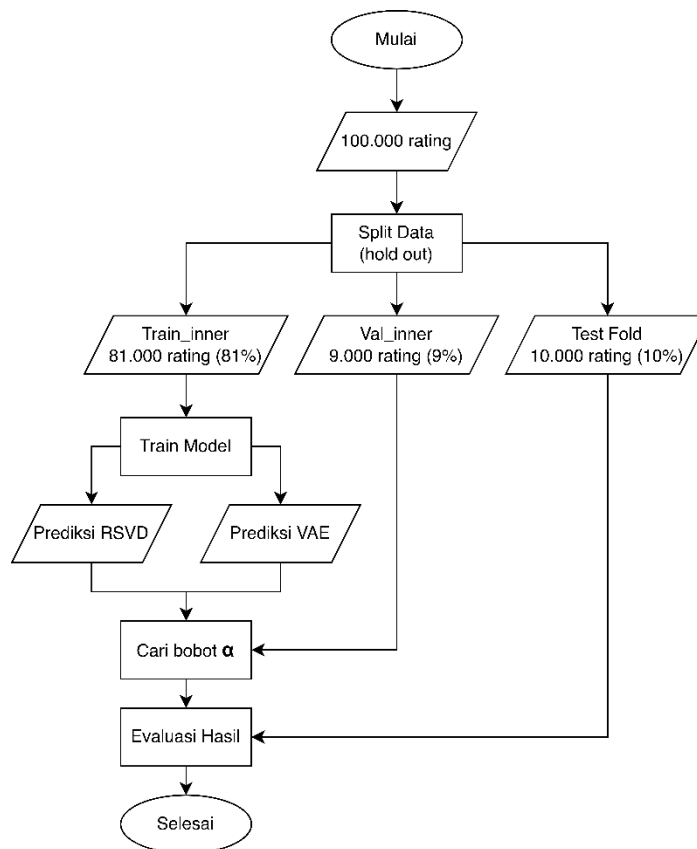
3.6.1 Metrik Evaluasi

Evaluasi performa model diukur menggunakan dua metrik, yaitu *Mean Absolute Error* (MAE) dan *Root Mean Squared Error* (RMSE) sebagaimana diformulasikan pada persamaan (18) dan (20). Kedua metrik ini mengukur selisih antara nilai rating aktual dengan nilai rating yang diprediksi oleh model. MAE memberikan gambaran rata-rata kesalahan prediksi secara merata, sedangkan

RMSE memberikan penalti yang lebih besar terhadap kesalahan prediksi yang jauh dari nilai aktual karena sifat pengkuadratannya. Nilai kedua metrik bersifat non-negatif dan semakin kecil nilainya semakin baik performa model.

3.6.2 10-Fold Cross-Validation

10-fold cross-validation dilakukan pada keseluruhan dataset untuk mengukur stabilitas dan kemampuan generalisasi model secara lebih komprehensif dibandingkan evaluasi hold-out tunggal. Nilai $k = 10$ dipilih berdasarkan pertimbangan keseimbangan antara bias dan variance dari estimasi performa sebagaimana dijelaskan pada tinjauan teori.



Gambar 3.4. Struktur Cross Validation

Pembagian fold dilakukan pada level rating individual, bukan pada level pengguna. Seluruh rating dikelompokkan ke dalam 10 fold secara acak, sehingga setiap fold berisi sekitar 10% dari total rating. Struktur partisi data pada setiap fold diilustrasikan pada Gambar 3.4. Pada setiap iterasi fold ke- i , sekitar 10.000 *rating*

pada fold ke- i digunakan sebagai data uji (*test fold*), sedangkan 90.000 *rating* sisanya dibagi lebih lanjut menjadi data latih-dalam (*train_inner*, ± 81.000 *rating*) dan data validasi-dalam (*val_inner*, ± 9.000 *rating*) menggunakan skema *hold-out* 90/10.

Dari masing-masing partisi ini, dibangun dua matriks interaksi pengguna-item yang terpisah, matriks *train_inner* yang hanya berisi *rating* pada posisi *train_inner*, dan matriks uji yang hanya berisi *rating* pada posisi *test fold*. Posisi yang tidak termasuk dalam partisi masing-masing matriks dibiarkan bernilai nol dan dikecualikan dari perhitungan metrik melalui *masked loss* pada VAE dan seleksi indeks eksplisit pada RSVD.

Pada setiap iterasi, VAE dan RSVD dilatih secara paralel dari awal menggunakan *train_inner* dengan hyperparameter optimal yang diperoleh dari Sub-bab 3.5. Prediksi dari kedua model kemudian dihasilkan pada posisi *rating val_inner* untuk keperluan pencarian bobot ensemble. Pencarian bobot α dilakukan melalui *grid search* pada *val_inner* menggunakan prosedur yang sama dengan Sub-bab 3.4.3. Bobot α terbaik yang diperoleh dari *val_inner* selanjutnya digunakan untuk menghitung prediksi hybrid pada *test fold*, di mana RMSE dan MAE dihitung untuk ketiga model (VAE, RSVD, dan hybrid) menggunakan *rating* aktual *test fold*. Hasil setiap *fold* disimpan dan proses ini diulang sebanyak 10 kali hingga seluruh *fold* telah digunakan sebagai data uji tepat satu kali. Metrik akhir dilaporkan sebagai rata-rata $\pm$ standar deviasi dari 10 *fold* menggunakan persamaan (21).

3.6.3 Uji Signifikansi Statistik (*Paired t-Test*)

Uji signifikansi statistik dilakukan untuk membuktikan bahwa perbedaan performa adalah perbedaan yang bermakna secara statistik. Pengujian *paired t-test* menggunakan persamaan (22) dan (23), dengan data berpasangan berupa metrik per *fold* dari *10-fold cross-validation*. Setiap pasangan nilai metrik dari dua model yang dibandingkan diperlakukan sebagai pengamatan berpasangan karena keduanya dievaluasi pada partisi data uji yang identik di setiap *fold*.

Pengujian dilakukan pada metrik RMSE dan MAE, mencakup tiga pasangan perbandingan yaitu *Hybrid* vs. RSVD, *Hybrid* vs. VAE, dan RSVD vs. VAE. Hipotesis yang diuji adalah sebagai berikut:

- H_0 : Tidak terdapat perbedaan yang signifikan antara rata-rata metrik kedua model yang dibandingkan
- H_1 : Terdapat perbedaan yang signifikan antara rata-rata metrik kedua model yang dibandingkan

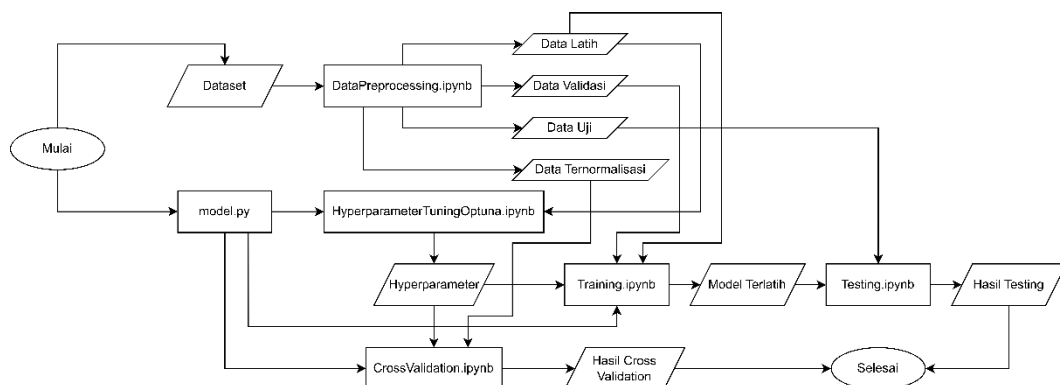
Tingkat signifikansi yang digunakan adalah $\alpha = 0,05$. H_0 ditolak apabila $p\text{-value} < 0,05$, yang berarti perbedaan performa antar model signifikan secara statistik pada tingkat kepercayaan 95%.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Implementasi Sistem

Sistem rekomendasi film yang dikembangkan dalam penelitian ini diimplementasikan menggunakan bahasa pemrograman Python dengan memanfaatkan beberapa *library* utama, yaitu NumPy dan Pandas untuk manipulasi data, TensorFlow dan Keras untuk pembangunan dan pelatihan model *Variational Autoencoder* (VAE), serta SciPy untuk pengujian statistik. Implementasi sistem diorganisasi ke dalam enam komponen utama yang saling terhubung, yaitu satu *script* definisi model (*model.py*) dan lima *notebook* Jupyter yang masing-masing menangani tahapan pipeline yang berbeda. Hubungan antar komponen tersebut diilustrasikan pada Gambar 4.1.



Gambar 4.1. Hubungan Antar Komponen

Gambar 4.1 menunjukkan bahwa pipeline sistem dimulai dari Dataset MovieLens 100K yang diproses oleh *DataPreprocessing.ipynb* menghasilkan empat keluaran yaitu Data Latih, Data Validasi, Data Uji, dan Data Ternormalisasi. *HyperParameterTuningOptuna.ipynb* mengonsumsi *model.py* dan Data Latih untuk mencari konfigurasi hyperparameter optimal, yang hasilnya disimpan sebagai Hyperparameter dan digunakan bersama oleh *Training.ipynb* dan *CrossValidation.ipynb*. *Training.ipynb* menerima Data Latih, Data Validasi, dan Hyperparameter, kemudian menghasilkan Model Terlatih yang menjadi masukan bagi *Testing.ipynb*. *CrossValidation.ipynb* menerima Data Ternormalisasi, Hyperparameter, dan definisi kelas dari *model.py* untuk menjalankan evaluasi 10-

fold secara independen dan menghasilkan Hasil Cross Validation. *Testing.ipynb* menggunakan Model Terlatih dan Data Uji untuk menghasilkan Hasil Testing. Deskripsi masing-masing komponen beserta fungsinya dirangkum pada Tabel 4.1.

Tabel 4.1. Deskripsi Komponen Implementasi Sistem

Komponen	Fungsi
<i>model.py</i>	Mendefinisikan seluruh kelas model (Sampling, Encoder, Decoder, VAE, KLANnealingCallback, dan RSVD)
<i>DataPreprocessing.ipynb</i>	Memuat dataset MovieLens 100K, membangun matriks interaksi <i>user-item</i> , melakukan normalisasi rating, dan membagi data menjadi <i>train/validation/test set</i>
<i>HyperParameterTuningOptuna.ipynb</i>	Melakukan pencarian hyperparameter optimal VAE menggunakan Optuna TPE (200 <i>trial</i>) dan hyperparameter RSVD menggunakan <i>grid search</i> (48 kombinasi)
<i>Training.ipynb</i>	Melatih VAE dan RSVD menggunakan hyperparameter terbaik hasil tuning dalam satu sesi penuh, menyimpan bobot model, dan memvisualisasikan kurva pelatihan
<i>CrossValidation.ipynb</i>	Menjalankan evaluasi 10- <i>fold cross-validation</i> , menghitung metrik per <i>fold</i> , merangkum statistik, dan melakukan <i>paired t-test</i> antar model
<i>Testing.ipynb</i>	Memuat bobot model hasil <i>Training.ipynb</i> , menjalankan prediksi pada <i>test set</i> final, mencari bobot

	ensemble optimal pada <i>validation set</i> , dan mengevaluasi performa ketiga model
--	--

4.1.1 Program Definisi Model

File `model.py` merupakan komponen inti yang mendefinisikan seluruh arsitektur model dalam penelitian ini. File ini terdiri dari enam kelas yang diorganisasi berdasarkan tanggung jawabnya masing-masing. Potongan kode yang mendefinisikan kelas-kelas tersebut ada pada Tabel 4.2 hingga Tabel 4.7.

Tabel 4.2. Kelas *Sampling*

```
class Sampling(Layer):
    def call(self, inputs):
        z_mean, z_log_var = inputs
        batch = tf.shape(z_mean)[0]
        dim = tf.shape(z_mean)[1]

        # Mengambil noise acak (epsilon) dari distribusi normal
        standar (mean=0, std=1)
        epsilon = tf.keras.backend.random_normal(shape=(batch,
dim))

        # Menghitung standar deviasi (sigma) dari log_var
        sigma = tf.exp(0.5 * z_log_var)

        # Mengembalikan titik sampel laten: z = mean + (sigma *
epsilon)
        return z_mean + (sigma * epsilon)
```

Tabel 4.2 adalah potongan program untuk kelas *sampling*. Kelas *Sampling* mengimplementasikan *reparameterization trick* yang memungkinkan operasi pengambilan sampel acak dari ruang laten tetap dapat diturunkan secara matematis (*differentiable*). Tanpa teknik ini, gradien tidak dapat mengalir melewati operasi *sampling* sehingga *backpropagation* tidak dapat berjalan. Kelas ini mewarisi *Layer* dari Keras dan hanya mendefinisikan satu *method* yaitu *call*, yang menerima

pasangan z_mean dan z_log_var dari *encoder* lalu menghasilkan titik sampel z menggunakan persamaan (7).

Tabel 4.3. Kelas *Encoder*

```
class Encoder(Model):
    def __init__(self, hidden_dims=[512, 256], latent_dim=50,
dropout_rate=0.5, **kwargs):
        super(Encoder, self).__init__(**kwargs)

        # Dropout digunakan untuk mencegah overfitting dengan
mematikan sebagian neuron secara acak
        self.dropout = Dropout(dropout_rate)

        # Membangun lapisan tersembunyi (Hidden Layers) secara
dinamis
        self.hidden_layers = [Dense(dim, activation='relu') for
dim in hidden_dims]

        # Output layer dari Encoder: Nilai Rata-rata (Mean) dan
Log Varians dari distribusi ruang laten
        self.z_mean = Dense(latent_dim, name='z_mean')
        self.z_log_var = Dense(latent_dim, name='z_log_var')

    def call(self, inputs):
        x = self.dropout(inputs)
        for layer in self.hidden_layers:
            x = layer(x)
        return self.z_mean(x), self.z_log_var(x)
```

Tabel 4.3 adalah potongan program untuk kelas *encoder*. Kelas *Encoder* memetakan vektor rating pengguna berdimensi tinggi menjadi representasi laten berdimensi rendah. Arsitekturnya dibangun secara dinamis berdasarkan parameter `hidden_dims`, sehingga jumlah dan ukuran lapisan tersembunyi dapat dikonfigurasi tanpa mengubah kode. Lapisan *dropout* diterapkan pada *input* untuk mencegah *overfitting*. *Encoder* menghasilkan dua keluaran yaitu z_mean dan z_log_var yang

merupakan parameter distribusi Gaussian dari ruang laten, bukan titik tunggal, sesuai dengan prinsip probabilitas VAE.

Tabel 4.4. Kelas *Decoder*

```
class Decoder(Model):
    def __init__(self, hidden_dims=[256, 512], output_dim=1682,
**kwargs):
        super(Decoder, self).__init__(**kwargs)
        self.hidden_layers = [Dense(dim, activation='relu') for
dim in hidden_dims]

        # Layer output menggunakan Sigmoid karena data input
        sudah dinormalisasi ke rentang 0 - 1
        self.reconstruction = Dense(output_dim,
activation='sigmoid', name='decoder_output')

    def call(self, inputs):
        x = inputs
        for layer in self.hidden_layers:
            x = layer(x)
        return self.reconstruction(x)
```

Tabel 4.4 adalah potongan program untuk kelas *decoder*. Kelas *Decoder* memiliki struktur simetris dengan *encoder* dimana lapisan tersembunyinya dibangun secara dinamis dari parameter `hidden_dims` yang diurutkan terbalik. *Layer* keluaran menggunakan fungsi aktivasi *sigmoid* yang membatasi nilai rekonstruksi ke rentang $[0, 1]$, sesuai dengan skala data yang telah dinormalisasi. Parameter `output_dim` diset sesuai jumlah film unik dalam dataset (1.682 film pada MovieLens 100K).

Tabel 4.5. Kelas VAE

```
class VAE(Model):
    # Tambahkan parameter beta di init (default 1.0 seperti VAE
biasa)
```

```

def __init__(self, encoder, decoder, beta=1.0, **kwargs):
    super(VAE, self).__init__(**kwargs)
    self.encoder = encoder
    self.decoder = decoder

    # Simpan beta sebagai tf.Variable agar nilainya bisa
    diperbarui secara
    # dinamis oleh KLANnealingCallback di setiap epoch tanpa
    perlu rebuild model
    self.beta = tf.Variable(float(beta), trainable=False,
dtype=tf.float32, name='beta')

    self.sampling = Sampling()

    # Pelacak nilai metrik selama proses training
    self.total_loss_tracker =
tf.keras.metrics.Mean(name="total_loss")
    self.reconstruction_loss_tracker =
tf.keras.metrics.Mean(name="reconstruction_loss")
    self.kl_loss_tracker =
tf.keras.metrics.Mean(name="kl_loss")

    @property
    def metrics(self):
        return [self.total_loss_tracker,
self.reconstruction_loss_tracker, self.kl_loss_tracker]

    def call(self, inputs):
        # Forward pass biasa untuk tahap evaluasi/testing
        z_mean, z_log_var = self.encoder(inputs)
        z = self.sampling([z_mean, z_log_var])
        return self.decoder(z)

    def train_step(self, data):
        if isinstance(data, tuple):
            data = data[0] # ambil x saja, y dibuang

```

```

with tf.GradientTape() as tape:
    z_mean, z_log_var = self.encoder(data)
    z = self.sampling([z_mean, z_log_var])
    reconstruction = self.decoder(z)

    # A. Reconstruction Loss (Masked MSE)
    # Membuat mask bernilai 1 jika rating ada, dan 0
jika kosong
    mask = tf.cast(data > 0, tf.float32)
    squared_diff = tf.square(data - reconstruction)
    masked_se = squared_diff * mask

    # Menghitung rata-rata error murni
    mse_loss = tf.reduce_sum(masked_se) /
(tf.reduce_sum(mask) + 1e-8)
    num_items = tf.cast(tf.shape(data)[1], tf.float32)
    reconstruction_loss = mse_loss * num_items

    # B. KL Divergence Loss
    # Berfungsi meregulasi agar distribusi ruang laten
mendekati normal Gaussian
    kl_loss = -0.5 * (1 + z_log_var - tf.square(z_mean)
- tf.exp(z_log_var))
    kl_loss = tf.reduce_mean(tf.reduce_sum(kl_loss,
axis=1))

    # C. Total Loss dengan Hyperparameter Beta
    # Gunakan self.beta yang didapat dari inisialisasi
model
    total_loss = reconstruction_loss + (self.beta *
kl_loss)

    # Backpropagation: Menghitung gradien dan memperbarui
bobot (weights)
    grads = tape.gradient(total_loss,
self.trainable_weights)

```

```

        self.optimizer.apply_gradients(zip(grads,
self.trainable_weights))

        # Memperbarui history loss untuk ditampilkan di progress
bar
        self.total_loss_tracker.update_state(total_loss)
        self.reconstruction_loss_tracker.update_state(reconstruc
tion_loss)
        self.kl_loss_tracker.update_state(kl_loss)

        return {
            "loss": self.total_loss_tracker.result(),
            "reconstruction_loss":
self.reconstruction_loss_tracker.result(),
            "kl_loss": self.kl_loss_tracker.result(),
        }

```

Tabel 4.5 adalah potongan program untuk kelas VAE. Kelas VAE menggabungkan Encoder, Sampling, dan Decoder menjadi satu model yang dapat dilatih. Kelas ini memiliki empat *method* utama:

- **`__init__`** menginisialisasi komponen model dan menetapkan beta sebagai `tf.Variable` karena dua alasan. Pertama, `tf.Variable` bersifat *mutable* sehingga nilainya dapat diubah secara *in-place* melalui `.assign()` tanpa membuat tensor baru, memungkinkan `KLAnnealingCallback` memperbarui beta di setiap *epoch* tanpa perlu membangun ulang model. Kedua, perubahan nilai tersebut terjadi di dalam *computational graph* TensorFlow sehingga kompatibel dengan mekanisme *graph execution* dan tidak memicu *retracing* pada setiap pembaruan. Tiga pelacak metrik (`total_loss_tracker`, `reconstruction_loss_tracker`, `kl_loss_tracker`) juga diinisialisasi di sini untuk memantau perkembangan *loss* selama pelatihan.
- **`metrics`** (*property*) mengembalikan daftar pelacak metrik tersebut agar Keras dapat *reset* nilainya secara otomatis di awal setiap *epoch*.
- **`call`** mendefinisikan *forward pass* standar yang digunakan saat evaluasi dan inferensi.

- **train_step** mengandung *masked reconstruction loss* dimana model hanya menghitung *error* pada posisi rating yang memiliki nilai eksplisit (> 0) dengan membuat *mask* biner, sehingga sel-sel kosong yang merepresentasikan film yang belum ditonton tidak ikut mempengaruhi gradien. *Total loss* dihitung sebagai jumlah *reconstruction loss* dan *KL divergence loss* yang dikali beta, sesuai dengan persamaan (6).

Tabel 4.6. Kelas KLANnealingCallback

```
class KLANnealingCallback(tf.keras.callbacks.Callback):
    def __init__(self, beta_target, annealing_epochs):
        super(KLANnealingCallback, self).__init__()
        self.beta_target = beta_target
        self.annealing_epochs = annealing_epochs

    def on_epoch_begin(self, epoch, logs=None):
        # Hitung proporsi kemajuan annealing (0.0 hingga 1.0)
        if self.annealing_epochs > 0:
            progress = min(epoch / self.annealing_epochs, 1.0)
        else:
            progress = 1.0

        # Hitung nilai beta saat ini secara linear
        beta_now = progress * self.beta_target

        # Perbarui tf.Variable beta di model tanpa perlu rebuild
        self.model.beta.assign(beta_now)

    def on_epoch_end(self, epoch, logs=None):
        current_beta = float(self.model.beta.numpy())
        print(f" [KL Annealing] Epoch {epoch + 1}: beta = {current_beta:.6f} / {self.beta_target}")
```

Tabel 4.6 adalah potongan program untuk kelas KLANnealingCallback. Kelas KLANnealingCallback mewarisi `tf.keras.callbacks.Callback` dan mengimplementasikan strategi *KL annealing* melalui tiga *method*:

- **__init__** menerima dua parameter yaitu `beta_target` yang merupakan nilai beta akhir hasil hyperparameter tuning, dan `annealing_epochs` yang menentukan durasi fase annealing.
- **on_epoch_begin** dipanggil otomatis oleh Keras di awal setiap *epoch*, di sinilah logika *annealing* berjalan. Nilai progress dihitung sebagai rasio *epoch* saat ini terhadap `annealing_epochs`, dibatasi maksimum 1.0 agar beta tidak melampaui `beta_target` setelah fase *annealing* selesai. Nilai beta baru kemudian ditetapkan ke model melalui `self.model.beta.assign(beta_now)` operasi ini memanfaatkan sifat *mutable* dari `tf.Variable` sehingga modifikasi terjadi *in-place* di dalam *computational graph* tanpa membuat tensor baru maupun memicu *retracing*.
- **on_epoch_end** hanya bertugas mencetak nilai beta aktif sebagai informasi monitoring selama proses pelatihan berlangsung.

Tabel 4.7. Kelas RSVD

```
class RSVD:
    def __init__(self, n_factors=50, learning_rate=0.001,
lambda_reg=0.001, epochs=100, patience=5):
        self.k = n_factors      # Jumlah dimensi laten
        self.eta = learning_rate # Kecepatan belajar
        self.lam = lambda_reg    # Penalti regularisasi untuk
mencegah overfitting
        self.epochs = epochs
        self.patience = patience # Jumlah epoch tanpa
perbaikan val MSE sebelum dihentikan
        self.loss_history = []    # Menyimpan nilai Total
Loss (MSE + L2 Penalty) setiap epoch

    def fit(self, Z, val_ratio=0.1, random_seed=42):
        m, n = Z.shape # m = Jumlah pengguna, n = Jumlah film
        np.random.seed(random_seed)

        # Ambil koordinat semua rating yang ada nilainya
        rated_indices = np.argwhere(Z > 0)
        n_val = int(len(rated_indices) * val_ratio)
```



```

        # Pilih indeks validation secara acak tanpa penggantian
        val_chosen = np.random.choice(len(rated_indices),
n_val, replace=False)
        val_idx = rated_indices[val_chosen]
        val_rows = val_idx[:, 0]
        val_cols = val_idx[:, 1]

        # Simpan nilai rating asli untuk evaluasi validation
        val_true = Z[val_rows, val_cols].copy()

        # Buat training matrix: salin Z lalu hapus rating
validation
        Z_train = Z.copy()
        Z_train[val_rows, val_cols] = 0.0

        print(f"[INFO] Total rating      : {len(rated_indices)}")
        print(f"[INFO] Training ratings : {len(rated_indices) -
n_val}")
        print(f"[INFO] Validation ratings: {n_val:,}")

        # Hitung rata-rata global HANYA dari rating training
(val sudah dihapus)
        nonzero_ratings = Z_train[Z_train > 0]
        self.mu = np.mean(nonzero_ratings) if
len(nonzero_ratings) > 0 else 0

        # Bias user (pemberi rating) dan item (film)
        self.b_u = np.zeros(m)
        self.b_i = np.zeros(n)

        # Inisialisasi U dan V dengan nilai kecil
        self.U = np.random.normal(scale=0.1, size=(m, self.k))
        self.V = np.random.normal(scale=0.1, size=(n, self.k))

        # Inisialisasi Sigma dengan Matriks Identitas

```

```

        # Mencegah error gradient terperangkap mendekati 0 di
epoch awal
        self.Sigma = np.eye(self.k)

        # Variabel untuk early stopping - memantau validation
MSE
        best_val_mse = float('inf')
        no_improve = 0

        # Snapshot bobot terbaik (disimpan saat val MSE mencapai
nilai terendah)
        best_U, best_V, best_Sigma = None, None, None
        best_b_u, best_b_i = None, None

        for epoch in range(self.epochs):
            for u in range(m):
                for i in range(n):
                    # KUNCI UTAMA: Hanya perbarui bobot jika
user benar-benar memberi rating
                    # Gunakan Z_train (bukan Z) agar rating
validation tidak ikut dilatih
                    if Z_train[u, i] > 0:
                        # Prediksi rating: Bias Global + Bias
User + Bias Item + (U * Sigma * V^T)
                        dot_product = np.dot(np.dot(self.U[u,
:], self.Sigma), self.V[i, :].T)
                        pred = self.mu + self.b_u[u] +
self.b_i[i] + dot_product

                        # Hitung jarak error (Asli dikurangi
Prediksi)
                        e_ui = Z_train[u, i] - pred

                        # Update Bias menggunakan Gradient
Descent & L2 Regularization
                        self.b_u[u] += self.eta * (e_ui -
self.lam * self.b_u[u])

```

```

        self.b_i[i] += self.eta * (e_ui -
self.lam * self.b_i[i])

        # Update Matriks Laten
        for k in range(self.k):
            U_uk = self.U[u, k]
            V_ik = self.V[i, k]
            Sigma_kk = self.Sigma[k, k]

            # Memperbarui bobot U, V, dan Sigma
            berdasarkan gradien error

            self.U[u, k] += self.eta * (e_ui *
Sigma_kk * V_ik - self.lam * U_uk)
            self.V[i, k] += self.eta * (e_ui *
Sigma_kk * U_uk - self.lam * V_ik)
            self.Sigma[k, k] += self.eta * (e_ui
* U_uk * V_ik)

            # Rekonstruksi matriks penuh
            bias_matrix = self.mu + self.b_u[:, np.newaxis] +
self.b_i[np.newaxis, :]
            latent_matrix = np.dot(np.dot(self.U, self.Sigma),
self.V.T)
            reconstructed_Z = bias_matrix + latent_matrix

            # Hitung training MSE (hanya pada rating training,
            sel kosong dan val diabaikan)
            train_mask = (Z_train > 0)
            current_mse = np.sum(np.square(Z_train[train_mask] -
reconstructed_Z[train_mask])) / np.sum(train_mask)

            # Hitung validation MSE (pada rating yang disisihkan
            di awal)
            val_pred = reconstructed_Z[val_rows, val_cols]
            val_mse = np.mean(np.square(val_true - val_pred))

            # lambda * (||U||^2_F + ||V||^2_F)

```

```

        # Sigma tidak diregularisasi (lihat Catatan Desain
        di docstring)
        l2_penalty = self.lam * (np.sum(np.square(self.U)) +
        np.sum(np.square(self.V)))

        # Total Loss = Training MSE + L2 Penalty (rumus 8
        proposal)
        current_loss = current_mse + l2_penalty
        self.loss_history.append(current_loss)

        # Print progress setiap epoch
        print(f"Epoch {epoch+1:03d}/{self.epochs} | Train
        MSE: {current_mse:.6f} | Val MSE: {val_mse:.6f} | L2:
        {l2_penalty:.6f} | Total Loss: {current_loss:.6f}")

        # Validation MSE dipakai karena training MSE selalu
        turun

        # dan tidak mencerminkan performa di data yang belum
        dilihat

        if val_mse < best_val_mse:
            best_val_mse = val_mse
            no_improve = 0

            # Simpan snapshot bobot terbaik saat ini
            best_U = self.U.copy()
            best_V = self.V.copy()
            best_Sigma = self.Sigma.copy()
            best_b_u = self.b_u.copy()
            best_b_i = self.b_i.copy()
        else:
            no_improve += 1
            if no_improve >= self.patience:
                print(f"\n[Early Stopping] Tidak ada
                perbaikan Val MSE selama {self.patience} epoch.")
                print(f"[Early Stopping] Berhenti di epoch
                {epoch+1}. Best Val MSE: {best_val_mse:.6f}")

            # Kembalikan bobot ke snapshot terbaik

```

```

self.U = best_U
self.V = best_V
self.Sigma = best_Sigma
self.b_u = best_b_u
self.b_i = best_b_i
break

```

Tabel 4.7 adalah potongan program untuk kelas RSVD. Kelas RSVD mengimplementasikan algoritma faktorisasi matriks dengan dua *method*:

- **__init__** menginisialisasi lima hyperparameter model:
 - **n_factors** (jumlah dimensi laten k)
 - **learning_rate** (kecepatan belajar η)
 - **lambda_reg** (koefisien regularisasi L2 λ)
 - **epochs** (batas maksimum iterasi)
 - **patience** (toleransi *early stopping*).

Atribut **loss_history** diinisialisasi sebagai *list* kosong untuk merekam *total loss* setiap *epoch*.

- **fit** menjalankan seluruh pipeline pelatihan RSVD yang terdiri dari empat tahap:
 - **validation split**, sebanyak **val_ratio** dari total rating disisihkan secara acak sebagai data validasi, dan matriks training **Z_train** dibuat dengan menghapus posisi rating validasi tersebut. Pemisahan ini diperlukan agar *early stopping* dapat memantau performa pada data yang tidak dilihat selama pelatihan.
 - **inisialisasi parameter**, bias global μ dihitung dari rata-rata rating training saja, bias *user* b_u dan bias *item* b_i diinisialisasi nol, matriks faktor laten **U** dan **V** diinisialisasi dengan nilai acak kecil, sedangkan **Sigma** diinisialisasi sebagai matriks identitas untuk mencegah *vanishing gradient* di *epoch-epoch* awal akibat nilai **Sigma** yang terlalu kecil.
 - **training loop**, untuk setiap *epoch*, model melakukan iterasi pada seluruh pasangan (*user*, *item*) dan hanya memperbarui parameter

pada posisi yang memiliki rating eksplisit. Prediksi dihitung menggunakan persamaan (9), *error* dihitung dengan persamaan (10) dan (11), lalu bias dan matriks laten diperbarui menggunakan SGD dengan regularisasi L2 menggunakan persamaan (12) – (15). Perlu diperhatikan bahwa Sigma tidak dikenai regularisasi L2 karena eksperimen awal menunjukkan bahwa meregularisasi Sigma menyebabkan *weight collapse* di mana nilai Sigma mendekati nol secara kaskade dan menghambat gradien U dan V.

- **early stopping**, di akhir setiap *epoch*, *validation MSE* dihitung dan dibandingkan dengan nilai terbaik sebelumnya. Jika tidak ada perbaikan selama *patience epoch* berturut-turut, pelatihan dihentikan dan bobot dikembalikan ke *snapshot* terbaik yang telah disimpan.

4.1.2 Program Preprocessing Data

Notebook DataPreprocessing.ipynb bertanggung jawab atas seluruh tahap persiapan data sebelum proses pelatihan model dimulai. *Notebook* ini diorganisasi ke dalam lima seksi utama dan menghasilkan empat *file* keluaran berformat .npz yang digunakan oleh *notebook-notebook* berikutnya dalam alur kerja sistem. Potongan kode *preprocessing data* terdapat pada tabel 4.8 - 4.15

1. File Paths

Seksi ini mendefinisikan dua variabel *file path* yang digunakan secara konsisten di seluruh *notebook*. *raw_data_path* menunjuk ke *file* u.data dari dataset *MovieLens* 100K, dan *output_dir* menunjuk ke folder TrainTest sebagai tujuan penyimpanan hasil *preprocessing*. Pendefinisian jalur secara terpusat di awal *notebook* memudahkan adaptasi apabila struktur direktori berubah. Kode seksi ini ditampilkan pada Tabel 4.8.

Tabel 4.8. Cell File Paths

```
# Path ke file data mentah MovieLens 100K
raw_data_path = '../Data/ml-100k/u.data'

# Path folder output untuk menyimpan hasil preprocessing
```

```
output_dir = '../Data/TrainTest'
```

2. Load Data

File `u.data` dibaca menggunakan `pd.read_csv` dengan pemisah *tab* dan empat nama kolom eksplisit yaitu `user_id`, `item_id`, `rating`, dan `timestamp`. Kolom `timestamp` dimuat tetapi tidak digunakan dalam pemodelan karena sistem rekomendasi yang dibangun tidak memodelkan preferensi berbasis waktu. Kode seksi ini ditampilkan pada Tabel 4.9.

Tabel 4.9. Cell Load Data

```
# Kolom pada file u.data: user_id, item_id, rating,
timestamp
# File menggunakan tab sebagai separator
column_names = ['user_id', 'item_id', 'rating',
'timestamp']
df = pd.read_csv(raw_data_path, sep='\t',
names=column_names)
```

3. Matriks Interaksi *User-Item*

Data tabular diubah menjadi matriks *user-item* dua dimensi menggunakan `pivot`, dengan `user_id` sebagai indeks baris, `item_id` sebagai indeks kolom, dan nilai `rating` sebagai isi sel. Pasangan *user-item* yang tidak memiliki `rating` diisi dengan nilai nol melalui `fillna(0)`, sehingga sel kosong dapat dibedakan dari `rating` terendah setelah normalisasi. Kode seksi ini ditampilkan pada Tabel 4.10.

Tabel 4.10. Cell Matriks Interaksi *User-Item*

```
# Pivot tabel menjadi matriks user x item
# Sel yang tidak memiliki rating diisi dengan 0
interaction_matrix = df.pivot(
    index='user_id',
    columns='item_id',
    values='rating'
).fillna(0)
```

4. Normalisasi Rating

Seluruh nilai rating dinormalisasi ke rentang $[0, 1]$ dengan membaginya terhadap nilai rating maksimum ($\text{MAX_RATING} = 5.0$). Strategi normalisasi ini dipilih agar sel bernilai nol yang merepresentasikan ketiadaan rating tetap bernilai nol setelah normalisasi, sehingga dapat dibedakan dari rating terendah yang bernilai 0,2 (yaitu $1/5$). Metode `.values` digunakan untuk mengekstrak representasi *NumPy array* dari *DataFrame* sebelum normalisasi dilakukan. Kode seksi ini ditampilkan pada Tabel 4.11.

Tabel 4.11. Cell Normalisasi Rating

```
# Normalisasi rating ke rentang [0, 1] dengan membagi
nilai maksimum rating (5.0)
MAX_RATING = 5.0

normalized_matrix = interaction_matrix.values / MAX_RATING
```

5. Train / Validation / Test Split

Split dilakukan pada level rating individual sehingga setiap pengguna dapat muncul di ketiga partisi. Tabel 4.12 menampilkan konfigurasi rasio *split* dan pengambilan koordinat rating, di mana seluruh koordinat rating yang bernilai non-nol diidentifikasi menggunakan `np.nonzero` lalu jumlah rating untuk masing-masing partisi dihitung berdasarkan proporsi 70/10/20. Tabel 4.13 menampilkan proses pengacakan indeks menggunakan `np.random.permutation` dengan *seed* tetap ($\text{RANDOM_SEED} = 42$) untuk menjamin reproduksibilitas, kemudian indeks dipotong menjadi tiga bagian sesuai proporsi dan koordinat masing-masing partisi diekstrak. Tabel 4.14 menampilkan pembangunan matriks terpisah untuk setiap partisi. Matriks *train* dimulai dari salinan matriks penuh kemudian posisi *validation* dan *test* dihapus menjadi nol, sedangkan matriks *validation* dan *test* masing-masing dibangun dari matriks nol yang kemudian diisi hanya pada posisi rating yang relevan.

Tabel 4.12. Cell Konfigurasi Rasio *Split*

```
# Konfigurasi split
VAL_RATIO = 0.10 # 10% untuk validasi
TEST_RATIO = 0.20 # 20% untuk test
RANDOM_SEED = 42

# Ambil koordinat semua rating yang ada nilainya
all_rows, all_cols = np.nonzero(normalized_matrix)
total_ratings = len(all_rows)

n_test = int(total_ratings * TEST_RATIO)
n_val = int(total_ratings * VAL_RATIO)
n_train = total_ratings - n_test - n_val
```

Tabel 4.13. *Cell Shuffle* dan Potong Indeks

```
# Acak seluruh indeks rating dengan seed tetap untuk
reproduksibilitas
np.random.seed(RANDOM_SEED)
shuffled_idx = np.random.permutation(total_ratings)

# Potong indeks menjadi tiga bagian berdasarkan proporsi
test_idx = shuffled_idx[:n_test]
val_idx = shuffled_idx[n_test:n_test + n_val]
train_idx = shuffled_idx[n_test + n_val:]

# Ambil koordinat (baris, kolom) masing-masing partisi
test_rows, test_cols =
all_rows[test_idx], all_cols[test_idx]
val_rows, val_cols =
all_rows[val_idx], all_cols[val_idx]
train_rows, train_cols = all_rows[train_idx],
all_cols[train_idx]
```

Tabel 4.14. Cell Bangun Matriks *Train, Validation, dan Test*

```
# Bangun matriks train: mulai dari matriks penuh, hapus
rating val dan test
train_data = normalized_matrix.copy().astype(np.float32)
```

```

train_data[val_rows, val_cols] = 0.0
train_data[test_rows, test_cols] = 0.0

# Bangun matriks validation: mulai dari nol, isi hanya
rating val
val_data = np.zeros(normalized_matrix.shape,
dtype=np.float32)
val_data[val_rows, val_cols] = normalized_matrix[val_rows,
val_cols]

# Bangun matriks test: mulai dari nol, isi hanya rating
test
test_data = np.zeros(normalized_matrix.shape,
dtype=np.float32)
test_data[test_rows, test_cols] =
normalized_matrix[test_rows, test_cols]

```

6. Save Data

Keempat matriks disimpan dalam format .npy menggunakan np.save ke folder output_dir. train_data.npy (70%), val_data.npy (10%), dan test_data.npy (20%) digunakan oleh *Training.ipynb* dan *Testing.ipynb*, sedangkan normalized_matrix.npy disimpan secara khusus untuk *CrossValidation.ipynb* agar setiap *fold* dapat membuat *split*-nya sendiri dari matriks rating yang utuh. Kode seksi ini ditampilkan pada Tabel 4.15.

Tabel 4.15. Cell Save Data

```

# Simpan matriks train (70%): dipakai untuk melatih VAE dan
RSVD di Training.ipynb
np.save(os.path.join(output_dir, 'train_data'),
train_data)

# Simpan matriks validation (10%): dipakai untuk mencari
alpha ensemble di Testing.ipynb
np.save(os.path.join(output_dir, 'val_data'), val_data)

```

```
# Simpan matriks test (20%): hanya disentuh satu kali untuk
# pelaporan performa akhir
np.save(os.path.join(output_dir, 'test_data'), test_data)

# Simpan matriks rating penuh sebelum di-split
# Dipakai oleh CrossValidation.ipynb agar setiap fold bisa
# membuat split-nya sendiri
np.save(os.path.join(output_dir, 'normalized_matrix'),
        normalized_matrix)
```

4.1.3 Program *Hyperparameter Tuning*

HyperParameterTuningOptuna.ipynb merupakan *notebook* yang mengelola proses pencarian hyperparameter optimal untuk model VAE dan RSVD. Secara keseluruhan, *notebook* ini terdiri dari empat seksi utama yang dijalankan secara berurutan: impor *library*, konfigurasi *path*, pemuatan data dan model, serta proses *tuning* untuk masing-masing model.

1. Import Library

Tabel 4.16. Cell *Import Library*

```
import numpy as np
import tensorflow as tf
import itertools
import sys
import os
import gc
import json
import optuna
```

Cell pertama pada seksi ini, ditampilkan pada Tabel 4.16, mengimpor seluruh *library* yang dibutuhkan selama proses *hyperparameter tuning*. *Library* numpy dan tensorflow digunakan untuk operasi numerik dan pelatihan model VAE. *Library* itertools digunakan untuk menghasilkan seluruh kombinasi hyperparameter pada proses *grid search* RSVD. *Library* sys dan os digunakan untuk manipulasi *path* sistem, gc untuk manajemen

memori secara eksplisit, json untuk membaca dan menyimpan hasil *tuning*, serta optuna sebagai *framework* optimasi hyperparameter berbasis TPE untuk VAE.

2. File Path

Tabel 4.17. Cell *Path* Folder

```
scripts_folder = os.path.abspath(os.path.join '..',
'Scripts'))
data_folder = os.path.abspath(os.path.join '..', 'Data',
'TrainTest'))
hyper_parameter_folder =
os.path.abspath(os.path.join '..', 'Data',
'HyperParameters'))
sys.path.append(scripts_folder)
```

Cell definisi *path* folder ditampilkan pada Tabel 4.17. Cell mendefinisikan tiga variabel *path* yaitu `scripts_folder` yang menunjuk ke direktori Scripts tempat `model.py` berada, `data_folder` yang menunjuk ke direktori TrainTest tempat data hasil *preprocessing* tersimpan, dan `hyper_parameter_folder` yang menunjuk ke direktori HyperParameters sebagai lokasi penyimpanan hasil *tuning*. Pada akhir cell `scripts_folder` ditambahkan ke `sys.path` agar modul `model.py` dapat diimpor secara langsung pada cell berikutnya.

3. Import Data & Model

Tabel 4.18. Cell *Import* Data & Model

```
from model import Encoder, Decoder, VAE, RSVD

train_data_path = os.path.join(data_folder,
'train_data.npy')

# Import training data
try:
    train_data =
np.load(train_data_path).astype(np.float32)
```

```

train_data_tf = tf.constant(train_data,
dtype=tf.float32) # Mencegah tensorflow duplikasi data
tiap iterasi (memory leak)

# Mengambil jumlah item
num_items = train_data.shape[1]
print(f"Data latih berhasil dimuat.")
print(f"    Dimensi data: {train_data.shape} (Pengguna
x Film)")
except FileNotFoundError:
    print("Error: File 'train_data.npy' tidak ditemukan.
Pastikan Anda sudah menjalankan preprocessing.")

```

Seksi ini memuat kelas model dan data latih melalui dua cell yang ditampilkan bersama pada Tabel 4.18. Cell pertama mengimpor kelas Encoder, Decoder, VAE, dan RSVD dari model.py. Cell kedua memuat data latih dari *file* train_data.npy ke dalam *array* NumPy bertipe float32. Data yang sama kemudian dikonversi menjadi tf.constant dan disimpan dalam variabel train_data_tf untuk mencegah TensorFlow menduplikasi data pada setiap iterasi pelatihan yang berpotensi menyebabkan *memory leak*. Selain itu, jumlah item num_items diekstrak dari dimensi kolom data latih dan akan digunakan sebagai ukuran lapisan keluaran *decoder* VAE.

4. Hyperparameter Tuning

Seksi ini merupakan inti dari notebook dan terbagi menjadi dua sub-seksi yaitu *tuning* VAE menggunakan Optuna dan *tuning* RSVD menggunakan *grid search exhaustif*.

a. Tuning VAE (Optuna TPE)

Sub-seksi VAE terdiri dari lima cell yang dijalankan secara berurutan. Cell pertama, ditampilkan pada Tabel 4.19, mendefinisikan search space hyperparameter VAE dalam bentuk dictionary vae_search_space. Ruang pencarian mencakup enam dimensi:

- i. latent_dim dengan empat nilai kandidat (20, 50, 100, 200)

- ii. `learning_rate` dengan lima nilai kandidat (1×10^{-4} hingga 1×10^{-2})
- iii. `batch_size` dengan tiga nilai kandidat (64, 128, 256)
- iv. `dropout_rate` dengan empat nilai kandidat (0.1 hingga 0.4)
- v. `hidden_dims` dengan empat konfigurasi arsitektur lapisan tersembunyi
- vi. `beta` dengan tujuh nilai kandidat (0.01 hingga 1.0).

Jumlah trial Optuna ditetapkan sebesar 200 melalui konstanta `N_TRIALS_VAE`.

Tabel 4.19. Cell Definisi Ruang Pencarian VAE

```
# Ruang pencarian hyperparameter VAE untuk Optuna
vae_search_space = {
    'latent_dim': [20, 50, 100, 200],
    'learning_rate': [1e-4, 5e-4, 1e-3, 5e-3, 1e-2],
    'batch_size': [64, 128, 256],
    'dropout_rate': [0.1, 0.2, 0.3, 0.4],
    'hidden_dims': [
        [512, 256],
        [512, 256, 128],
        [1024, 512],
        [1024, 512, 256],
    ],
    'beta': [0.01, 0.05, 0.1, 0.3, 0.5, 0.8, 1.0],
}

# Jumlah trial yang akan dijalankan Optuna
N_TRIALS_VAE = 200
```

Cell kedua, ditampilkan pada Tabel 4.20, mendefinisikan fungsi objektif `vae_objective` yang dipanggil Optuna pada setiap *trial*. Fungsi ini menerima objek trial dari Optuna dan menggunakan metode `suggest_categorical` untuk memilih nilai dari masing-masing dimensi *search space*. Karena `suggest_categorical` hanya menerima

tipe primitif, nilai `hidden_dims` yang berupa *list* dikonversi terlebih dahulu ke *string* sebelum dipilih, lalu dikembalikan ke *list* menggunakan `eval`. Setelah hyperparameter dipilih, sesi TensorFlow dibersihkan, model VAE dibangun dan dikompilasi, kemudian dilatih selama 30 *epoch*. Nilai `reconstruction_loss` pada *epoch* terakhir dikembalikan sebagai nilai objektif yang akan diminimalkan oleh Optuna. Penggunaan `reconstruction_loss` sebagai kriteria seleksi dilakukan untuk menghindari bias terhadap nilai β yang kecil, karena *total loss* akan lebih kecil secara artifisial ketika β bernilai rendah meskipun kualitas rekonstruksi tidak lebih baik.

Tabel 4.20. Cell Fungsi Objektif VAE

```
def vae_objective(trial):
    latent_dim =
trial.suggest_categorical('latent_dim',
vae_search_space['latent_dim'])
    learning_rate =
trial.suggest_categorical('learning_rate',
vae_search_space['learning_rate'])
    batch_size =
trial.suggest_categorical('batch_size',
vae_search_space['batch_size'])
    dropout_rate =
trial.suggest_categorical('dropout_rate',
vae_search_space['dropout_rate'])
    beta = trial.suggest_categorical('beta',
vae_search_space['beta'])

    hidden_dims_str =
trial.suggest_categorical('hidden_dims', [str(h) for
h in vae_search_space['hidden_dims']])
    hidden_dims = eval(hidden_dims_str)

    tf.keras.backend.clear_session()
    tf.compat.v1.reset_default_graph()
```

```

        encoder = Encoder(hidden_dims=hidden_dims,
                           latent_dim=latent_dim, dropout_rate=dropout_rate)
        decoder = Decoder(hidden_dims=hidden_dims[:-1],
                           output_dim=num_items)
        vae = VAE(encoder, decoder, beta=beta)

        dummy_output = vae(train_data_tf[:1])

        optimizer =
tf.keras.optimizers.Adam(learning_rate=learning_rate
)

        vae.compile(optimizer=optimizer,
run_eagerly=True)

        history = vae.fit(
            train_data_tf, train_data_tf,
            epochs = 30,
            batch_size = batch_size,
            verbose = 0
        )

        reconstruction_loss =
history.history['reconstruction_loss'][-1]

        del encoder, decoder, vae, history, optimizer,
dummy_output
        gc.collect()
        tf.keras.backend.clear_session()

        return reconstruction_loss

```

Cell ketiga, ditampilkan pada Tabel 4.21, membuat atau melanjutkan *Optuna study* menggunakan penyimpanan SQLite (*optuna_vae_study.db*). Penggunaan *load_if_exists=True* memungkinkan *study* dilanjutkan secara otomatis apabila proses

notebook terputus sebelum seluruh trial selesai. Cell ini juga menghitung jumlah trial yang sudah selesai (*n_completed*) dan trial yang tersisa (*n_remaining*) sebelum proses *tuning* dilanjutkan.

Tabel 4.21. Cell Inisialisasi *Study* Optuna

```
study_vae = optuna.create_study(
    study_name = 'vae_tuning',
    direction = 'minimize',
    storage = f'sqlite:///vae_study_db',
    load_if_exists = True    # otomatis resume jika
study sudah ada
)

n_completed = len([t for t in study_vae.trials if
t.state == optuna.trial.TrialState.COMPLETE])
n_remaining = max(0, N_TRIALS_VAE - n_completed)
```

Cell keempat, ditampilkan pada Tabel 4.22, menjalankan proses optimasi dengan memanggil *study_vae.optimize*. Sebuah *callback* *print_trial_callback* didefinisikan untuk mencetak hasil setiap *trial* secara langsung, termasuk penanda [NEW BEST] apabila *trial* tersebut menghasilkan *reconstruction loss* terendah baru. Optimasi hanya dijalankan apabila masih terdapat trial yang tersisa (*n_remaining* > 0), sehingga cell ini aman untuk dieksekusi ulang tanpa menimpa *trial* yang sudah selesai.

Tabel 4.22. Cell Eksekusi *Tuning* VAE

```
def print_trial_callback(study, trial):
    print(f" Trial {trial.number+1:>3} | Recon
Loss: {trial.value:.6f} | Params: {trial.params}")
    if study.best_trial.number == trial.number:
        print(f"[NEW BEST] Reconstruction Loss turun
ke {trial.value:.6f}")

if n_remaining > 0:
```

```

        print(f"[INFO] Memulai {n_remaining} trial
Optuna (TPE)...\n")
        study_vae.optimize(
            vae_objective,
            n_trials = n_remaining,
            callbacks = [print_trial_callback],
            gc_after_trial = True      # bersihkan memori
setelah setiap trial
        )
    else:
        print("[INFO] Semua trial sudah selesai. Tidak
ada yang perlu dijalankan.")

```

Cell kelima, ditampilkan pada Tabel 4.23, menyimpan hasil *tuning* terbaik VAE ke *file* JSON `tuning_progress_vae.json`. Sebelum disimpan, nilai `hidden_dims` dikonversi kembali dari *string* ke *list* menggunakan `eval`. Format JSON yang dihasilkan seragam dengan format yang dibaca oleh `Training.ipynb`, `CrossValidation.ipynb`, dan `Testing.ipynb`, sehingga tidak diperlukan modifikasi pada *notebook-notebook* tersebut.

Tabel 4.23. Cell Simpan Hasil Terbaik *Tuning* VAE

```

best_params_raw = study_vae.best_params.copy()
best_params_raw['hidden_dims'] =
eval(best_params_raw['hidden_dims'])

with open(vae_progress_file, 'w') as f:
    json.dump({
        'best_loss': study_vae.best_value,
        'best_params': best_params_raw,
    }, f, indent=4)

```

b. *Tuning* RSVD (Grid Search)

Sub-seksi RSVD terdiri dari tiga cell. Cell pertama, ditampilkan pada Tabel 4.24, mendefinisikan *grid search* dalam bentuk *dictionary* `rsvd_param_grid` yang mencakup tiga dimensi:

- i. `n_factors` dengan tiga kandidat (10, 20, 50)
- ii. `learning_rate` dengan empat kandidat (0.001 hingga 0.02)
- iii. `lambda_reg` dengan empat kandidat (0.001 hingga 0.02).

Seluruh kombinasi dihasilkan menggunakan `itertools.product`, menghasilkan 48 kombinasi yang akan dievaluasi secara exhaustif.

Tabel 4.24. Cell Definisi Ruang Pencarian RSVD

```
rsvd_param_grid = {
    'n_factors': [10, 20, 50],
    'learning_rate': [0.001, 0.005, 0.01, 0.02],
    'lambda_reg': [0.001, 0.005, 0.01, 0.02]
}

rsvd_keys, rsvd_values =
zip(*rsvd_param_grid.items())
rsvd_combinations = [dict(zip(rsvd_keys, v)) for v
in itertools.product(*rsvd_values)]
```

Cell kedua terbagi menjadi dua sub-cell yaitu pemuatan *progress* dan *looping* utama. Sub-cell pemuatan *progress*, ditampilkan pada Tabel 4.25, memuat *file* `tuning_progress_rsvd.json` apabila sudah tersedia. Dari *file* tersebut, nilai `best_error`, `best_params`, dan daftar `completed_indices` dipulihkan agar proses dapat dilanjutkan dari titik terakhir tanpa mengulang kombinasi yang sudah dievaluasi.

Tabel 4.25. Cell Load Progress *Tuning* RSVD

```
rsvd_progress_file =
os.path.join(hyper_parameter_folder,
'tuning_progress_rsvd.json')

if os.path.exists(rsvd_progress_file):
```

```

with open(rsvd_progress_file, 'r') as f:
    progress_data = json.load(f)

    best_rsvd_error = progress_data['best_error']
    best_rsvd_params = progress_data['best_params']
    completed_indices =
progress_data['completed_indices']

    print(f"\n[INFO] Melanjutkan sesi RSVD
    sebelumnya...")
    print(f"[INFO] {len(completed_indices)}
    kombinasi sudah dievaluasi.")
    print(f"[INFO] MSE terbaik saat ini:
    {best_rsvd_error:.4f}")
else:
    best_rsvd_error = float('inf')
    best_rsvd_params = None
    completed_indices = []

```

Sub-cell *looping*, ditampilkan pada Tabel 4.26, mengiterasi seluruh 48 kombinasi. Untuk setiap kombinasi yang belum dikerjakan, model RSVD diinisialisasi dan dilatih selama 15 *epoch*, kemudian nilai MSE pada *epoch* terakhir (`loss_history[-1]`) digunakan sebagai kriteria evaluasi. Apabila MSE yang diperoleh lebih rendah dari nilai terbaik sebelumnya, model dinyatakan sebagai model terbaik baru dan seluruh komponennya disimpan sebagai *file* .*numpy*. Setiap selesai mengevaluasi satu kombinasi, *progress* langsung disimpan ke *file* JSON sehingga proses aman untuk diinterupsi kapan saja.

Tabel 4.26. Cell Eksekusi *Tuning* RSVD

```

for i, params in enumerate(rsvd_combinations):
    if i in completed_indices:
        continue

    rsvd = RSVD(

```

```

        n_factors=params['n_factors'],
        learning_rate=params['learning_rate'],
        lambda_reg=params['lambda_reg'],
        epochs=15 # Epoch kecil untuk tuning
    )

    rsvd.fit(train_data)

    total_loss_error = rsvd.loss_history[-1]

    if total_loss_error < best_rsvd_error:
        best_rsvd_error = total_loss_error
        best_rsvd_params = params
        best_rsvd_model = rsvd

        np.save(os.path.join(hyper_parameter_folder,
                              'best_mu.npy'), np.array(rsvd.mu))
        np.save(os.path.join(hyper_parameter_folder,
                              'best_b_u.npy'), rsvd.b_u)
        np.save(os.path.join(hyper_parameter_folder,
                              'best_b_i.npy'), rsvd.b_i)
        np.save(os.path.join(hyper_parameter_folder,
                              'best_U.npy'), rsvd.U)
        np.save(os.path.join(hyper_parameter_folder,
                              'best_Sigma.npy'), rsvd.Sigma)
        np.save(os.path.join(hyper_parameter_folder,
                              'best_V.npy'), rsvd.V)

    completed_indices.append(i)
    with open(rsvd_progress_file, 'w') as f:
        json.dump({
            'best_error': best_rsvd_error,
            'best_params': best_rsvd_params,
            'completed_indices': completed_indices
        }, f)

    del rsvd

```

```
gc.collect()
```

4.1.4 Program Training

Notebook Training.ipynb mengimplementasikan proses pelatihan akhir untuk model VAE dan RSVD menggunakan hyperparameter terbaik yang telah diperoleh dari tahap *hyperparameter tuning*. *Notebook* ini terdiri dari lima seksi utama, yaitu impor *library*, konfigurasi jalur direktori, load data dan kelas model, proses *training* VAE dan RSVD beserta penyimpanan artefaknya, serta visualisasi kurva konvergensi. Implementasi pada *notebook* ini ditampilkan pada Tabel 4.27 hingga Tabel 4.35.

1. Import Library

Tabel 4.27 menampilkan cell impor *library* yang digunakan pada *notebook* ini. Cell ini mengimpor seluruh *library* yang dibutuhkan selama proses pelatihan. *Library* os dan sys digunakan untuk manajemen jalur sistem, json untuk membaca file hyperparameter hasil tuning, numpy untuk operasi numerik, tensorflow sebagai *framework* pelatihan VAE, matplotlib.pyplot untuk visualisasi kurva konvergensi, serta datetime untuk pemberian *timestamp* pada file keluaran.

Tabel 4.27. Cell Import Library Pada Training.ipynb

```
import os
import sys
import json
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from datetime import datetime
```

2. Konfigurasi Direktori

Tabel 4.28 menampilkan cell konfigurasi jalur direktori yang digunakan sepanjang *notebook*. Cell ini mendefinisikan seluruh jalur direktori yang digunakan selama proses pelatihan. Variabel *scripts_path* mengarah ke direktori Scripts tempat model.py disimpan, dan ditambahkan ke sys.path

agar kelas-kelas model dapat diimpor. Variabel `data_dir` mengarah ke direktori data latih, `hyperparameter_dir` ke direktori penyimpanan hasil tuning, `save_dir` ke direktori penyimpanan bobot model terlatih, `loss_history_dir` ke direktori riwayat loss, dan `visualization_dir` ke direktori penyimpanan gambar visualisasi.

Tabel 4.28. Cell Konfigurasi Direktori Pada Training.ipynb

```
scripts_path = os.path.abspath(os.path.join('.',
'Scripts'))
if scripts_path not in sys.path:
    sys.path.append(scripts_path)

data_dir = os.path.abspath(os.path.join('.', 'Data',
'TrainTest'))
hyperparameter_dir = os.path.abspath(os.path.join('.',
'Data', 'HyperParameters'))
save_dir = os.path.abspath(os.path.join('.', 'Data',
'SavedModels'))
loss_history_dir = os.path.abspath(os.path.join('.',
'Data', 'LossHistory'))
visualization_dir = os.path.abspath(os.path.join('.',
'Data', 'Visualization'))
```

3. Load Data & Model

Tabel 4.29 menampilkan cell pemuatan data latih dan impor kelas model yang digabungkan menjadi satu blok persiapan. Cell ini memuat matriks rating data latih dari file `train_data.npy` ke dalam variabel `train_data` bertipe `float32`. Dimensi kolom matriks tersebut diekstrak sebagai variabel `num_items` yang merepresentasikan jumlah total item (film) dan akan digunakan sebagai dimensi output layer VAE. Selanjutnya, kelas-kelas Encoder, Decoder, VAE, RSVD, dan `KLAnnealingCallback` diimpor dari `model.py`.

Tabel 4.29. Cell Load Data Latih dan Import Kelas Pada Training.ipynb

```
# Load matriks rating (data latih)
train_data = np.load(os.path.join(data_dir,
'train_data.npy')).astype(np.float32)

# Mengambil jumlah item (film) untuk layer output VAE
num_items = train_data.shape[1]

from model import Encoder, Decoder, VAE, RSVD,
KLANnealingCallback
```

4. *Training* VAE

Seksi pelatihan VAE terdiri dari tiga tahap yang ditampilkan pada Tabel 4.30, Tabel 4.31, dan Tabel 4.32.

Tabel 4.30. Cell Import *Hyperparameter* dan Konstruksi VAE Pada *Training.ipynb*

```
vae_progress_file = os.path.join(hyperparameter_dir,
'tuning_progress_vae.json')
with open(vae_progress_file, 'r') as f:
    best_vae_params = json.load(f) ['best_params']

encoder =
Encoder(hidden_dims=best_vae_params['hidden_dims'],
latent_dim=best_vae_params['latent_dim'],
dropout_rate=best_vae_params['dropout_rate'])
decoder =
Decoder(hidden_dims=best_vae_params['hidden_dims'][:-1],
output_dim=num_items)

# Inisialisasi VAE dengan beta=0.0 agar KL annealing mulai
dari nol
final_vae = VAE(encoder, decoder, beta=0.0)

_ = final_vae(train_data[:1])
```



```

optimizer =
tf.keras.optimizers.Adam(learning_rate=best_vae_params['le
arning_rate'])
final_vae.compile(optimizer=optimizer)

# Jumlah epoch untuk menaikkan beta dari 0 ke beta_target
secara linear.
KL_ANNEALING_EPOCHS = 30

# Callback KL annealing: naikan beta bertahap dari 0 →
best_vae_params['beta']
kl_annealing = KLANnealingCallback(
    beta_target=best_vae_params['beta'],
    annealing_epochs=KL_ANNEALING_EPOCHS
)

# Early stopping tetap dipakai seperti sebelumnya
early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor='loss', patience=10,
    restore_best_weights=True, verbose=1
)

```

Program pada Tabel 4.30 ini memuat hyperparameter VAE terbaik dari file `tuning_progress_vae.json`, kemudian menggunakannya untuk membangun arsitektur model. Objek Encoder dikonstruksi dengan `hidden_dims`, `latent_dim`, dan `dropout_rate` dari hasil *tuning*, sedangkan objek Decoder dikonstruksi dengan urutan `hidden_dims` yang dibalik (`[::-1]`) untuk membentuk arsitektur simetris. Model VAE diinisialisasi dengan $\beta = 0.0$ agar nilai *beta* dimulai dari nol dan dinaikkan secara bertahap oleh `KLANnealingCallback`. *Forward pass* awal dilakukan pada satu sampel data (`train_data[:1]`) untuk membangun bobot model sebelum kompilasi. *Optimizer* Adam dikompilasi dengan `learning_rate` dari hasil *tuning*. Nilai `KL_ANNEALING_EPOCHS` ditetapkan sebesar 30 sebagai jumlah *epoch* annealing, dan *callback* `EarlyStopping` dikonfigurasi dengan `patience=10`

untuk menghentikan pelatihan apabila tidak ada perbaikan *loss* dalam 10 *epoch* berturut-turut.

Tabel 4.31. Cell *Training* VAE Pada *Training.ipynb*

```
history = final_vae.fit(
    train_data, train_data,
    epochs=500,
    batch_size=best_vae_params['batch_size'],
    callbacks=[kl_annealing, early_stopping],
    verbose=1
)
```

Program pada Tabel 4.31 menjalankan proses pelatihan VAE menggunakan metode *fit*. Data latih digunakan sebagai masukan sekaligus target (*train_data*, *train_data*) sesuai dengan paradigma *autoencoder*. Jumlah *epoch* maksimum ditetapkan sebesar 500 dengan *callback* *kl_annealing* ditempatkan sebelum *early_stopping* agar nilai *beta* diperbarui terlebih dahulu sebelum *loss* dihitung pada setiap awal *epoch*.

Tabel 4.32. Cell Menyimpan Bobot dan *Loss History* VAE Pada *Training.ipynb*

```
final_vae.save_weights(os.path.join(save_dir,
    'trained_best_vae_weights.weights.h5'))

vae_history_dict = {'loss': history.history['loss']}

with open(os.path.join(loss_history_dir,
    'final_vae_loss_history.json'), 'w') as f:
    json.dump(vae_history_dict, f)
```

Program pada Tabel 4.32 menyimpan hasil pelatihan VAE. Bobot model disimpan dalam format *.h5* ke direktori *SavedModels* dengan nama *trained_best_vae_weights.weights.h5*. Riwayat nilai *loss* per *epoch* disimpan dalam format JSON ke direktori *LossHistory* dengan nama

final_vae_loss_history.json, yang akan digunakan kembali pada tahap visualisasi.

5. *Training* RSVD

Seksi pelatihan RSVD terdiri dari tiga tahap yang ditampilkan pada Tabel 4.33, Tabel 4.34, dan Tabel 4.35.

Tabel 4.33. Cell Load *Hyperparameter* dan Konstruksi Model RSVD Pada Training.ipynb

```
rsvd_progress_file = os.path.join(hyperparameter_dir,
'tuning_progress_rsvd.json')
with open(rsvd_progress_file, 'r') as f:
    best_rsvd_params = json.load(f)['best_params']

final_rsvd = RSVD(
    n_factors=best_rsvd_params['n_factors'],
    learning_rate=best_rsvd_params['learning_rate'],
    lambda_reg=best_rsvd_params['lambda_reg'],
    epochs=100
)
```

Program pada Tabel 4.33 memuat hyperparameter RSVD terbaik dari file tuning_progress_rsvd.json, kemudian menginisialisasi objek RSVD dengan nilai n_factors, learning_rate, dan lambda_reg dari hasil *tuning*. Jumlah *epoch* ditetapkan sebesar 100 untuk memastikan proses dekomposisi matriks berjalan hingga konvergen pada pelatihan final.

Tabel 4.34. Cell *Training* Model RSVD Pada Training.ipynb

```
# Train dengan data asli
final_rsvd.fit(train_data)

# Mengambil loss dari evaluasi akhir array
final_total_loss = final_rsvd.loss_history[-1]
```

Program pada Tabel 4.34 menjalankan pelatihan RSVD dengan memanggil metode fit pada train_data. Setelah pelatihan selesai, nilai *loss* pada iterasi

terakhir diambil dari `loss_history[-1]` dan ditampilkan sebagai informasi konvergensi akhir model.

Tabel 4.35. Cell Menyimpan Bobot dan *Loss History* RSVD Pada Training.ipynb

```
np.save(os.path.join(save_dir, 'final_mu.npy'),
np.array(final_rsvd.mu))
np.save(os.path.join(save_dir, 'final_b_u.npy'),
final_rsvd.b_u)
np.save(os.path.join(save_dir, 'final_b_i.npy'),
final_rsvd.b_i)
np.save(os.path.join(save_dir, 'best_U.npy'),
final_rsvd.U)
np.save(os.path.join(save_dir, 'best_Sigma.npy'),
final_rsvd.Sigma)
np.save(os.path.join(save_dir, 'best_V.npy'),
final_rsvd.V)

np.save(os.path.join(loss_history_dir,
'final_rsvd_loss_history.npy'), final_rsvd.loss_history)
```

Program pada Tabel 4.35 ini menyimpan hasil pelatihan RSVD. Komponen dekomposisi matriks disimpan secara terpisah dalam format `.npy`. Riwayat *loss* pelatihan disimpan dalam file `final_rsvd_loss_history.npy` ke direktori `LossHistory` untuk keperluan visualisasi.

4.1.5 Program Cross Validation

Notebook `CrossValidation.ipynb` mengimplementasikan proses validasi silang 10-*fold* untuk mengevaluasi konsistensi dan kemampuan generalisasi model VAE, RSVD, dan *hybrid ensemble* secara menyeluruh. *Notebook* ini terdiri dari sebelas seksi utama, yaitu impor *library*, konfigurasi direktori, pemuatan data dan hyperparameter, konfigurasi parameter CV, pembentukan *fold*, definisi fungsi-fungsi pembantu, eksekusi loop CV, perhitungan ringkasan statistik beserta penyimpanan hasil, dan uji signifikansi statistik *paired t-test*. Implementasi pada *notebook* ini ditampilkan pada Tabel 4.36 hingga Tabel 4.48.

1. *Import Library*

Tabel 4.36. Cell *Import Library* Pada *CrossValidation.ipynb*

```
import os
import sys
import gc
import json
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
from datetime import datetime
```

Program pada Tabel 4.36 mengimpor seluruh *library* yang dibutuhkan selama proses validasi silang. Selain *library* standar yang juga digunakan di *notebook* sebelumnya, terdapat dua tambahan yang spesifik untuk kebutuhan CV yaitu modul *gc* (*garbage collector*) untuk pembersihan memori secara eksplisit antar *fold*, dan *matplotlib.ticker* untuk pemformatan sumbu pada grafik visualisasi hasil CV.

2. Konfigurasi Direktori

Tabel 4.37. Cell Konfigurasi Direktori Pada *CrossValidation.ipynb*

```
# Menambahkan folder Scripts ke path agar model.py bisa
diimport
scripts_path = os.path.abspath(os.path.join '..',
'Scripts'))
if scripts_path not in sys.path:
    sys.path.append(scripts_path)

# Path folder data
data_dir = os.path.abspath(os.path.join '..',
'Data', 'TrainTest'))
hyperparameter_dir = os.path.abspath(os.path.join '..',
'Data', 'HyperParameters'))
```

```

cv_output_dir      = os.path.abspath(os.path.join('..',
'Data', 'CrossValidation'))

# Buat folder CrossValidation jika belum ada
os.makedirs(cv_output_dir, exist_ok=True)

```

Program pada Tabel 4.37 mendefinisikan seluruh jalur direktori yang dibutuhkan. Variabel `scripts_path` ditambahkan ke `sys.path` agar `model.py` dapat diimpor. Variabel `data_dir` mengarah ke direktori data latih, `hyperparameter_dir` ke direktori hasil *tuning*, dan `cv_output_dir` ke direktori keluaran hasil CV. Direktori `cv_output_dir` dibuat secara otomatis menggunakan `os.makedirs` dengan parameter `exist_ok=True` apabila belum tersedia.

3. Load Data & Hyperparameter

Tabel 4.38 menampilkan cell impor kelas model, pemuatan data, dan pemuatan hyperparameter yang digabungkan sebagai satu blok persiapan.

Tabel 4.38. Cell *Import Model, Load Data dan Hyperparameter* Pada *CrossValidation.ipynb*

```

from model import Encoder, Decoder, VAE, RSVD,
KLANnealingCallback

# Load matriks rating yang sudah dinormalisasi (0-1),
belum di-split
normalized_matrix_path = os.path.join(data_dir,
'normalized_matrix.npy')

try:
    normalized_matrix =
np.load(normalized_matrix_path).astype(np.float32)
    num_users, num_items = normalized_matrix.shape
except FileNotFoundError:
    print("[ERROR] File 'normalized_matrix.npy' tidak
ditemukan.")

```

```
# Load hyperparameter terbaik VAE
vae_progress_path = os.path.join(hyperparameter_dir,
'tuning_progress_vae.json')
with open(vae_progress_path, 'r') as f:
    best_vae_params = json.load(f)['best_params']

# Load hyperparameter terbaik RSVD
rsvd_progress_path = os.path.join(hyperparameter_dir,
'tuning_progress_rsvd.json')
with open(rsvd_progress_path, 'r') as f:
    best_rsvd_params = json.load(f)['best_params']
```

Cell ini menjalankan tiga persiapan sekaligus. Pertama, kelas-kelas model diimpor dari `model.py`. Kedua, matriks *rating* yang telah dinormalisasi dimuat dari file `normalized_matrix.npy`. Dimensi matriks diekstrak sebagai `num_users` dan `num_items`. Ketiga, hyperparameter terbaik VAE dan RSVD dimuat dari masing-masing file JSON hasil *tuning* tanpa melakukan *tuning* ulang di setiap *fold*, karena hal tersebut akan memakan waktu komputasi yang tidak proporsional.

4. Konfigurasi Cross-Validation

**Tabel 4.39. Cell Konfigurasi *Cross-Validation* Pada
CrossValidation.ipynb**

```
# Jumlah fold untuk K-Fold Cross Validation
N_FOLDS = 10

# Epoch untuk setiap fold (lebih sedikit dari Training
final karena tujuannya konsistensi)
CV_VAE_EPOCHS = 50
CV_RSVD_EPOCHS = 30

# Skala rating asli untuk denormalisasi hasil prediksi
MAX_RATING = 5.0
```

```
# Seed acak untuk memastikan hasil dapat direplikasi
RANDOM_SEED = 42
```

Program pada Tabel 4.39 mendefinisikan seluruh konstanta konfigurasi CV dalam satu lokasi terpusat. Variabel `N_FOLDS` ditetapkan sebesar 10 sesuai skema *10-fold cross-validation*. Jumlah *epoch* per *fold* untuk VAE (`CV_VAE_EPOCHS = 50`) dan RSVD (`CV_RSVD_EPOCHS = 30`) ditetapkan lebih kecil dibandingkan pelatihan final untuk menjaga efisiensi komputasi. Variabel `MAX_RATING` bernilai 5.0 digunakan untuk mendenormalisasi prediksi kembali ke skala rating asli (1–5). Variabel `RANDOM_SEED = 42` digunakan untuk memastikan hasil pembagian *fold* dapat direplikasi.

5. Pembentukan *Fold*

Tabel 4.40 menampilkan cell ekstraksi koordinat rating dan pembagian indeks ke dalam *fold* yang digabungkan sebagai satu blok pembentukan *fold*.

Tabel 4.40. Cell Ekstraksi Koordinat Rating dan Pembagian *Fold* Pada `CrossValidation.ipynb`

```
# Ambil koordinat (baris, kolom) dari semua rating yang
ada nilainya (> 0)
all_rows, all_cols = np.nonzero(normalized_matrix)

total_ratings = len(all_rows)

# Set seed untuk reproduksibilitas
np.random.seed(RANDOM_SEED)

# Acak urutan semua indeks rating
shuffled_indices = np.random.permutation(total_ratings)

# Bagi indeks yang sudah diacak menjadi N_FOLDS bagian
yang (hampir) sama besar
```



```
fold_index_groups = np.array_split(shuffled_indices,
N_FOLDS)
```

Cell ini membangun struktur pembagian *fold* dari matriks *rating* penuh. Fungsi `np.nonzero` digunakan untuk mengekstrak koordinat baris dan kolom dari seluruh 100.000 entri *rating* yang tersedia. Seluruh indeks tersebut kemudian diacak menggunakan `np.random.permutation` dengan *seed* tetap untuk memastikan reproduksibilitas, lalu dibagi menjadi 10 kelompok yang hampir sama besar menggunakan `np.array_split`.

6. Fungsi Pembantu (*Helper Functions*)

Seksi ini mendefinisikan lima fungsi pembantu yang digunakan di dalam loop CV, ditampilkan pada Tabel 4.41 hingga Tabel 4.45.

Tabel 4.41. Cell Fungsi `create_fold_matrices` Pada `CrossValidation.ipynb`

```
def create_fold_matrices(normalized_matrix, all_rows,
all_cols, test_fold_indices, val_ratio=0.1,
random_seed=42):
    # LANGKAH 1: Pisahkan TEST dari keseluruhan data
    test_rows = all_rows[test_fold_indices]
    test_cols = all_cols[test_fold_indices]

    test_matrix = np.zeros(normalized_matrix.shape,
dtype=np.float32)
    test_matrix[test_rows, test_cols] =
normalized_matrix[test_rows, test_cols]

    train_full_matrix = normalized_matrix.copy()
    train_full_matrix[test_rows, test_cols] = 0.0

    # LANGKAH 2: Pisahkan VAL_INNER dari TRAIN dengan
hold-out
    train_rows, train_cols = np.nonzero(train_full_matrix)
    n_train = len(train_rows)
    n_val = int(n_train * val_ratio)
```

```

    rng = np.random.default_rng(random_seed)
    val_chosen = rng.choice(n_train, size=n_val,
replace=False)
    val_inner_rows = train_rows[val_chosen]
    val_inner_cols = train_cols[val_chosen]

    val_inner_values = train_full_matrix[val_inner_rows,
val_inner_cols].copy()
    val_inner_indices = (val_inner_rows, val_inner_cols)

    train_inner_matrix = train_full_matrix.copy()
    train_inner_matrix[val_inner_rows, val_inner_cols] =
0.0

    return train_inner_matrix, val_inner_indices,
val_inner_values, test_matrix, (test_rows, test_cols)

```

Fungsi `create_fold_matrices` pada Tabel 4.41 bertugas membentuk tiga partisi data untuk setiap *fold* yaitu `train_inner_matrix`, `val_inner`, dan `test_matrix`. Fungsi ini bekerja dalam dua langkah. Pertama, rating pada *fold* yang sedang diproses dipisahkan sebagai *test set* dengan mengisi `test_matrix` dari koordinat `test_fold_indices`, sementara rating tersebut dihapus dari salinan matriks penuh untuk membentuk `train_full_matrix`. Kedua, sebesar 10% dari `train_full_matrix` dipisahkan secara acak sebagai `val_inner` menggunakan `np.random.default_rng` dengan *seed* tetap yang nanti akan digunakan untuk mencari bobot *ensemble* alpha tanpa menyentuh *test set*. Sisa rating yang tersisa membentuk `train_inner_matrix` yang digunakan untuk melatih VAE dan RSVD. Struktur partisi per *fold* adalah *test* (10%), *val_inner* (9%), dan *train_inner* (81%).

Tabel 4.42. Cell Fungsi `train_vae_fold` Pada `CrossValidation.ipynb`

```

def train_vae_fold(train_matrix, vae_params, num_items,
epochs, kl_annealing_epochs=15):
# Bersihkan sesi TensorFlow sebelum membuat model baru

```

```

tf.keras.backend.clear_session()

# Ubah data ke TensorFlow constant agar tidak ada
duplikasi memori
train_tf = tf.constant(train_matrix, dtype=tf.float32)

# Bangun arsitektur VAE dengan hyperparameter terbaik
encoder = Encoder(
    hidden_dims=vae_params['hidden_dims'],
    latent_dim=vae_params['latent_dim'],
    dropout_rate=vae_params['dropout_rate']
)
decoder = Decoder(
    hidden_dims=vae_params['hidden_dims'][:-1],
    output_dim=num_items
)

# Inisialisasi VAE dengan beta=0.0 agar KL annealing
mulai dari nol
vae_model = VAE(encoder, decoder, beta=0.0)

# Inisialisasi bobot dengan melewati 1 baris data
_ = vae_model(train_tf[:1])

# Kompilasi dengan optimizer Adam
optimizer =
tf.keras.optimizers.Adam(learning_rate=vae_params['learning_rate'])
vae_model.compile(optimizer=optimizer,
run_eagerly=True)

# Callback KL annealing: naikkan beta dari 0 ke
vae_params['beta'] secara linear
kl_annealing = KLANnealingCallback(
    beta_target=vae_params['beta'],
    annealing_epochs=kl_annealing_epochs
)

```

```

        # Latih model dengan KL annealing (verbose=0 agar
        output tidak terlalu panjang)
        vae_model.fit(
            train_tf, train_tf,
            epochs=epochs,
            batch_size=vae_params['batch_size'],
            callbacks=[kl_annealing],
            verbose=0
        )

        # Ekstrak ruang laten (z_mean) lalu rekonstruksi
        dengan decoder
        z_mean, _ = vae_model.encoder.predict(train_tf,
        verbose=0)
        pred_matrix = vae_model.decoder.predict(z_mean,
        verbose=0)

        # Bersihkan semua objek model dari memori
        del encoder, decoder, vae_model, optimizer, train_tf,
        z_mean, _, kl_annealing
        gc.collect()
        tf.keras.backend.clear_session()

        return pred_matrix

```

Fungsi `train_vae_fold` pada Tabel 4.42 melatih satu instans VAE untuk satu *fold* dan mengembalikan matriks prediksi rekonstruksi penuh berukuran 943×1682 . Fungsi ini memanggil `tf.keras.backend.clear_session()` di awal dan akhir eksekusi untuk membersihkan grafik komputasi TensorFlow dan mencegah *memory leak* antar *fold*. Arsitektur VAE dibangun ulang dari nol menggunakan hyperparameter terbaik, dilatih dengan `KLAnnealingCallback` tanpa `EarlyStopping` untuk menjaga jumlah *epoch* tetap terkontrol. Prediksi dihasilkan melalui dua langkah yaitu `encoder.predict` menghasilkan *z_mean* dari ruang *laten*, kemudian

decoder.predict merekonstruksi prediksi *rating* dari *z_mean* tersebut. Seluruh objek model dihapus secara eksplisit menggunakan `del` dan `gc.collect()` setelah prediksi selesai diambil.

Tabel 4.43. Cell Fungsi train_rsvd_fold Pada CrossValidation.ipynb

```
def train_rsvd_fold(train_matrix, rsvd_params, epochs):
    # Inisialisasi model RSVD dengan hyperparameter
    terbaik

    rsvd_model = RSVD(
        n_factors=rsvd_params['n_factors'],
        learning_rate=rsvd_params['learning_rate'],
        lambda_reg=rsvd_params['lambda_reg'],
        epochs=epochs
    )

    # Latih model dengan data training fold ini
    rsvd_model.fit(train_matrix)

    # Rekonstruksi prediksi penuh: bias + interaksi laten
    bias_matrix = float(rsvd_model.mu) + rsvd_model.b_u[:,
np.newaxis] + rsvd_model.b_i[np.newaxis, :]
    latent_matrix = np.dot(np.dot(rsvd_model.U,
rsvd_model.Sigma), rsvd_model.V.T)
    pred_matrix = bias_matrix + latent_matrix

    # Bersihkan objek model dari memori
    del rsvd_model, bias_matrix, latent_matrix
    gc.collect()

    return pred_matrix
```

Fungsi `train_rsvd_fold` pada Tabel 4.43 melatih satu instans RSVD untuk satu *fold* dan mengembalikan matriks prediksi penuh berukuran 943×1682 . Setelah pelatihan selesai melalui metode `fit`, prediksi rating direkonstruksi dengan menjumlahkan komponen *bias* yang terdiri dari *global bias*, *user bias*, dan *item bias* dengan komponen interaksi *laten* yang diperoleh dari

perkalian matriks U, Sigma, dan V.T. Seluruh objek model dan matriks antara dihapus menggunakan `del` dan `gc.collect()` setelah prediksi selesai.

Tabel 4.44. Cell Fungsi `find_best_ensemble_alpha` Pada `CrossValidation.ipynb`

```
def find_best_ensemble_alpha(pred_vae_full,
pred_rsvd_full, val_inner_indices, val_inner_values):
    best_alpha = 0.0
    best_val_rmse = float('inf')

    # Ambil prediksi di koordinat val_inner lalu
    denormalisasi ke skala 1-5
    val_pred_vae =
np.clip(pred_vae_full[val_inner_indices] * MAX_RATING,
1.0, MAX_RATING)
    val_pred_rsvd =
np.clip(pred_rsvd_full[val_inner_indices] * MAX_RATING,
1.0, MAX_RATING)
    val_actual = val_inner_values * MAX_RATING

    # Coba setiap nilai alpha dari 0% hingga 100% dengan
    langkah 1%
    for alpha in np.linspace(0, 1, 101):
        beta = 1.0 - alpha
        pred_ensemble = np.clip((alpha * val_pred_vae) +
(beta * val_pred_rsvd), 1.0, MAX_RATING)
        rmse = np.sqrt(np.mean(np.square(val_actual -
pred_ensemble)))

        if rmse < best_val_rmse:
            best_val_rmse = rmse
            best_alpha = alpha

    return best_alpha, best_val_rmse
```

Fungsi `find_best_ensemble_alpha` pada Tabel 4.44 mencari bobot alpha optimal untuk *weighted average ensemble* menggunakan data `val_inner`,

bukan *test set*, sehingga evaluasi akhir di *test set* tetap tidak bias. Fungsi ini mengiterasi 101 nilai alpha dari 0,00 hingga 1,00 dengan langkah 0,01 menggunakan `np.linspace`. Pada setiap iterasi, prediksi *ensemble* dihitung sebagai $(\alpha \times \text{VAE}) + ((1 - \alpha) \times \text{RSVD})$ dan RMSE-nya dievaluasi terhadap `val_inner`. Nilai alpha yang menghasilkan RMSE terkecil di `val_inner` dipilih sebagai bobot terbaik untuk *fold* tersebut.

Tabel 4.45. Cell Fungsi `calculate_metrics` Pada `CrossValidation.ipynb`

```
def calculate_metrics(y_true, y_pred):
    mse = np.mean(np.square(y_true - y_pred))
    rmse = np.sqrt(mse)
    mae = np.mean(np.abs(y_true - y_pred))
    return mse, rmse, mae
```

Fungsi `calculate_metrics` pada Tabel 4.45 menghitung tiga metrik evaluasi regresi sekaligus dari pasangan nilai aktual `y_true` dan nilai prediksi `y_pred`. MSE dihitung sebagai rata-rata kuadrat selisih, RMSE sebagai akar dari MSE, dan MAE sebagai rata-rata nilai absolut selisih. Ketiga metrik dikembalikan sekaligus dalam satu pemanggilan fungsi untuk efisiensi.

7. Loop K-Fold Cross-Validation

Tabel 4.46. Cell *Loop Utama Cross-Validation* Pada `CrossValidation.ipynb`

```
cv_results = { 'fold': [], 'vae_mse': [], 'vae_rmse': [],
               'vae_mae': [], 'rsvd_mse': [], 'rsvd_rmse': [],
               'rsvd_mae': [], 'hybrid_mse': [], 'hybrid_rmse': [],
               'hybrid_mae': [], 'best_alpha': [], 'best_val_rmse': [],
               'test_size': []}

for fold_idx, test_fold_indices in
    enumerate(fold_index_groups):
    fold_num = fold_idx + 1
```

```

# LANGKAH 1: Buat matriks train_inner, val_inner, dan
test
    train_inner_matrix, val_inner_indices,
val_inner_values, test_matrix, test_indices =
create_fold_matrices(
    normalized_matrix, all_rows, all_cols,
test_fold_indices,
    val_ratio=0.1, random_seed=RANDOM_SEED
)

# Ambil nilai aktual dari test set lalu denormalisasi
actual_values_norm = test_matrix[test_indices]
actual_values = actual_values_norm * MAX_RATING

# LANGKAH 2: Training dan Prediksi VAE
pred_vae_full = train_vae_fold(
    train_inner_matrix, best_vae_params, num_items,
CV_VAE_EPOCHS
)

# LANGKAH 3: Training dan Prediksi RSVD
pred_rsvd_full = train_rsvd_fold(
    train_inner_matrix, best_rsvd_params,
CV_RSVD_EPOCHS
)

# LANGKAH 4: Cari alpha ensemble terbaik menggunakan
VAL_INNER
    best_alpha, best_val_rmse = find_best_ensemble_alpha(
        pred_vae_full, pred_rsvd_full,
        val_inner_indices, val_inner_values
    )
    best_beta = 1.0 - best_alpha

# LANGKAH 5: Evaluasi di TEST SET menggunakan alpha
dari val_inner

```



```

        pred_vae_test = np.clip(pred_vae_full[test_indices] *
MAX_RATING, 1.0, MAX_RATING)
        pred_rsvd_test = np.clip(pred_rsvd_full[test_indices]
* MAX_RATING, 1.0, MAX_RATING)
        pred_hybrid_test = np.clip(
            (best_alpha * pred_vae_test) + (best_beta *
pred_rsvd_test),
            1.0, MAX_RATING
        )

        # Hitung metrik evaluasi
        mse_v, rmse_v, mae_v =
calculate_metrics(actual_values, pred_vae_test)
        mse_r, rmse_r, mae_r =
calculate_metrics(actual_values, pred_rsvd_test)
        mse_h, rmse_h, mae_h =
calculate_metrics(actual_values, pred_hybrid_test)

        # LANGKAH 6: Simpan hasil fold ke dictionary
        cv_results['fold'].append(fold_num)
        cv_results['test_size'].append(n_test)
        cv_results['best_alpha'].append(float(best_alpha))
        cv_results['best_val_rmse'].append(float(best_val_rmse
))

        cv_results['vae_mse'].append(float(mse_v))
        cv_results['vae_rmse'].append(float(rmse_v))
        cv_results['vae_mae'].append(float(mae_v))

        cv_results['rsvd_mse'].append(float(mse_r))
        cv_results['rsvd_rmse'].append(float(rmse_r))
        cv_results['rsvd_mae'].append(float(mae_r))

        cv_results['hybrid_mse'].append(float(mse_h))
        cv_results['hybrid_rmse'].append(float(rmse_h))
        cv_results['hybrid_mae'].append(float(mae_h))

```

```
# Bersihkan memori sebelum fold berikutnya
del train_inner_matrix, test_matrix
del val_inner_indices, val_inner_values
del actual_values, actual_values_norm
del pred_vae_full, pred_rsvd_full
del pred_vae_test, pred_rsvd_test, pred_hybrid_test
gc.collect()
```

Program pada Tabel 4.46 mengeksekusi loop utama *10-fold cross-validation* dalam enam langkah berurutan untuk setiap *fold*. Langkah pertama memanggil `create_fold_matrices` untuk membentuk partisi `train_inner`, `val_inner`, dan `test` pada *fold* yang sedang berjalan. Langkah kedua dan ketiga melatih VAE dan RSVD secara terpisah menggunakan `train_inner_matrix` melalui fungsi `train_vae_fold` dan `train_rsvd_fold`. Langkah keempat mencari bobot *alpha ensemble* terbaik pada `val_inner` menggunakan `find_best_ensemble_alpha` tanpa menyentuh *test set*. Langkah kelima menerapkan *alpha* tersebut untuk menghasilkan prediksi *hybrid* pada *test set*, lalu menghitung MSE, RMSE, dan MAE ketiga model menggunakan `calculate_metrics`. Langkah keenam menyimpan seluruh hasil ke *dictionary* `cv_results`, diikuti pembersihan memori secara eksplisit menggunakan `del` dan `gc.collect()` sebelum *fold* berikutnya dimulai.

8. Perhitungan Ringkasan dan Penyimpanan Hasil

Tabel 4.47. Cell Perhitungan Ringkasan dan Penyimpanan Hasil Pada `CrossValidation.ipynb`

```
# Metrik yang ingin diringkaskan
metric_keys = [
    'vae_mse', 'vae_rmse', 'vae_mae',
    'rsvd_mse', 'rsvd_rmse', 'rsvd_mae',
    'hybrid_mse', 'hybrid_rmse', 'hybrid_mae'
]

# Hitung mean dan std dev untuk setiap metrik
```

```

cv_summary = {}
for key in metric_keys:
    values = np.array(cv_results[key])
    cv_summary[key] = {
        'mean' : float(np.mean(values)),
        'std' : float(np.std(values)),
        'min' : float(np.min(values)),
        'max' : float(np.max(values))
    }

timestamp = datetime.now().strftime('%Y%m%d_%H%M')
cv_run_dir = os.path.join(cv_output_dir,
f'cv_{timestamp}')
os.makedirs(cv_run_dir, exist_ok=True)

final_cv_output = {
    'timestamp': timestamp,
    'configuration': {
        'n_folds': N_FOLDS,
        'cv_vae_epochs': CV_VAE_EPOCHS,
        'cv_rsvd_epochs': CV_RSVD_EPOCHS,
        'random_seed': RANDOM_SEED,
        'vae_params': best_vae_params,
        'rsvd_params': best_rsvd_params
    },
    'per_fold_results': cv_results,
    'summary': cv_summary
}

cv_json_path = os.path.join(cv_run_dir,
f'cv_results_{timestamp}.json')
with open(cv_json_path, 'w') as f:
    json.dump(final_cv_output, f, indent=4)

```

Program pada Tabel 4.47 menjalankan dua operasi pasca-pemrosesan. Pertama, ringkasan statistik dihitung untuk setiap metrik dari seluruh 10 *fold*, meliputi nilai *mean*, *standard deviation*, *minimum*, dan *maksimum*,

yang disimpan dalam *dictionary* `cv_summary`. Kedua, seluruh hasil CV digabungkan ke dalam satu *dictionary* `final_cv_output` dan disimpan ke file JSON dalam subfolder bertanda waktu (format `cv_YYYYMMDD_HHMM`) di dalam direktori `CrossValidation`. Penamaan berbasis *timestamp* memastikan setiap sesi CV tersimpan rapi tanpa menimpa hasil sebelumnya.

9. Uji Signifikansi Statistik *Paired t-Test*

Tabel 4.48. Cell Perhitungan Ringkasan dan Penyimpanan Hasil Pada `CrossValidation.ipynb`

```
from scipy import stats

cv_output_dir = os.path.abspath(os.path.join('.', 'Data',
'CrossValidation'))
CV_RESULT_FILE =
'cv_20260402_2042/cv_results_20260402_2042.json'
cv_json_path = os.path.join(cv_output_dir, CV_RESULT_FILE)

try:
    with open(cv_json_path, 'r') as f:
        cv_data = json.load(f)
except FileNotFoundError:
    print(f"[ERROR] File tidak ditemukan: {cv_json_path}")

# Ekstrak RMSE dan MAE per fold untuk setiap model
per_fold = cv_data['per_fold_results']
N_FOLDS = cv_data['configuration']['n_folds']
rmse_vae = np.array(per_fold['vae_rmse'])
rmse_rsvd = np.array(per_fold['rsvd_rmse'])
rmse_hybrid = np.array(per_fold['hybrid_rmse'])
mae_vae = np.array(per_fold['vae_mae'])
mae_rsvd = np.array(per_fold['rsvd_mae'])
mae_hybrid = np.array(per_fold['hybrid_mae'])

# Paired t-test untuk RMSE
```

```

t_rmse_h_r, p_rmse_h_r = stats.ttest_rel(rmse_hybrid,
rmse_rsvd)
t_rmse_h_v, p_rmse_h_v = stats.ttest_rel(rmse_hybrid,
rmse_vae)
t_rmse_r_v, p_rmse_r_v =
stats.ttest_rel(rmse_rsvd,    rmse_vae)

# Paired t-test untuk MAE
t_mae_h_r, p_mae_h_r = stats.ttest_rel(mae_hybrid,
mae_rsvd)
t_mae_h_v, p_mae_h_v = stats.ttest_rel(mae_hybrid,
mae_vae)
t_mae_r_v, p_mae_r_v =
stats.ttest_rel(mae_rsvd,    mae_vae)

alpha = 0.05
ttest_results = {
    'alpha': alpha,
    'n_folds': N_FOLDS,
    'rmse': {
        'hybrid_vs_rsvd': {'t_statistic':
float(t_rmse_h_r), 'p_value': float(p_rmse_h_r),
'significant': bool(p_rmse_h_r < alpha)},
        'hybrid_vs_vae': {'t_statistic':
float(t_rmse_h_v), 'p_value': float(p_rmse_h_v),
'significant': bool(p_rmse_h_v < alpha)},
        'rsvd_vs_vae': {'t_statistic': float(t_rmse_r_v),
'p_value': float(p_rmse_r_v), 'significant':
bool(p_rmse_r_v < alpha)},
    },
    'mae': {
        'hybrid_vs_rsvd': {'t_statistic':
float(t_mae_h_r), 'p_value': float(p_mae_h_r),
'significant': bool(p_mae_h_r < alpha)},
        'hybrid_vs_vae': {'t_statistic': float(t_mae_h_v),
'p_value': float(p_mae_h_v), 'significant': bool(p_mae_h_v
< alpha)},
    },
}

```

```

        'rsvd_vs_vae': {'t_statistic': float(t_mae_r_v),
        'p_value': float(p_mae_r_v), 'significant': bool(p_mae_r_v
        < alpha)},
        },
    }

cv_data['paired_ttest'] = ttest_results
with open(cv_json_path, 'w') as f:
    json.dump(cv_data, f, indent=4)

```

Program pada Tabel 4.48 melaksanakan uji *paired t-test* untuk membuktikan bahwa perbedaan performa antar model tidak terjadi secara kebetulan. Uji dilakukan terhadap dua metrik, yaitu RMSE dan MAE, dengan tiga pasangan perbandingan: *Hybrid* vs RSVD, *Hybrid* vs VAE, dan RSVD vs VAE. Fungsi `stats.ttest_rel` dari *library* *scipy* digunakan karena setiap pasangan pengamatan berasal dari *fold* yang sama, sehingga data bersifat berpasangan (*paired*). Hipotesis nol (H_0) menyatakan tidak ada perbedaan signifikan antara dua model, dan H_0 ditolak apabila *p-value* < 0,05. Hasil uji disimpan kembali ke file JSON hasil CV yang sama di bawah *key* `paired_ttest`.

4.1.6 Program Testing

Notebook `Testing.ipynb` mengimplementasikan proses pengujian akhir model VAE, RSVD, dan *hybrid ensemble* pada *hold-out test set*. *Notebook* ini terdiri dari enam seksi utama, yaitu impor *library*, konfigurasi direktori, pemuatan data dan model, prediksi ketiga model, pencarian dan penerapan bobot *ensemble*, serta evaluasi dan penyimpanan hasil. Implementasi pada *notebook* ini ditampilkan pada Tabel 4.49 hingga Tabel 4.60.

1. Import Library

Tabel 4.49. Cell Import Library Pada Testing.ipynb

```

import os
import sys
import json

```

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from datetime import datetime
```

Program pada Tabel 4.49 mengimpor seluruh *library* yang dibutuhkan selama proses pengujian. *Library* *os* dan *sys* digunakan untuk manajemen jalur sistem, *json* untuk pemuatan hyperparameter dan penyimpanan hasil evaluasi, *numpy* untuk operasi numerik, *tensorflow* untuk inferensi VAE, *matplotlib.pyplot* untuk visualisasi kurva pencarian alpha, serta *datetime* untuk pemberian *timestamp* pada file keluaran.

2. Konfigurasi Direktori

Tabel 4.50. Cell Konfigurasi Direktori Pada Testing.ipynb

```
scripts_path = os.path.abspath(os.path.join '..',
'Scripts'))
if scripts_path not in sys.path:
    sys.path.append(scripts_path)

data_dir = os.path.abspath(os.path.join '..', 'Data',
'TrainTest'))
save_dir = os.path.abspath(os.path.join '..', 'Data',
'SavedModels'))
hyperparameters_dir = os.path.abspath(os.path.join '..',
'Data', 'HyperParameters'))
visualization_dir = os.path.abspath(os.path.join '..',
'Data', 'Visualization'))
results_dir = os.path.abspath(os.path.join '..', 'Data',
'EvaluationResults'))
```

Program pada Tabel 4.50 mendefinisikan lima jalur direktori yang digunakan selama pengujian. Variabel *scripts_path* ditambahkan ke *sys.path* agar *model.py* dapat diimpor. Variabel *data_dir* mengarah ke direktori data latih dan uji, *save_dir* ke direktori bobot model terlatih,

hyperparameters_dir ke direktori hasil *tuning*, visualization_dir ke direktori penyimpanan gambar, dan results_dir ke direktori penyimpanan hasil evaluasi akhir.

3. Load Data & Model

Tabel 4.51. Cell Load Data dan Model Pada Testing.ipynb

```
# Load data training (70% dari total rating)
# Dipakai sebagai input encoder VAE untuk mengekstrak
representasi laten
train_data = np.load(os.path.join(data_dir,
"train_data.npy")).astype(np.float32)
num_users, num_items = train_data.shape

# Load data validasi (10% dari total rating)
# Dipakai untuk mencari bobot ensemble optimal (alpha)
tanpa melihat test set
MAX_RATING = 5.0

val_data = np.load(os.path.join(data_dir,
"val_data.npy")).astype(np.float32)
val_indices = np.where(val_data > 0)

# Denormalisasi nilai aktual validasi ke skala rating asli
(1-5)
actual_val_values = val_data[val_indices] * MAX_RATING

# Load data test (20% dari total rating)
# Hanya disentuh satu kali untuk pelaporan performa akhir
model
test_data = np.load(os.path.join(data_dir,
"test_data.npy")).astype(np.float32)
test_indices = np.where(test_data > 0)

# Denormalisasi nilai aktual test ke skala rating asli (1-
5)
```



```
actual_test_values = test_data[test_indices] * MAX_RATING

from model import Encoder, Decoder, VAE
```

Program pada Tabel 4.51 memuat tiga partisi data sekaligus mengimpor kelas model. Data latih dimuat dari `train_data.npy` dan digunakan sebagai input *encoder* VAE pada tahap inferensi untuk mengekstrak representasi *laten* seluruh pengguna. Data validasi dimuat dari `val_data.npy` dan digunakan semata-mata untuk mencari bobot *alpha ensemble* yang optimal tanpa menyentuh *test set*, sehingga evaluasi akhir tetap tidak bias. Data uji dimuat dari `test_data.npy` dan hanya digunakan satu kali pada tahap evaluasi akhir. Nilai aktual dari data validasi dan data uji dinormalisasi ke skala rating 1–5 dengan mengalikan $\text{MAX_RATING} = 5.0$. Koordinat entri *rating* yang tersedia pada kedua partisi tersebut diekstrak menggunakan `np.where`.

4. Konstruksi dan *Load* Bobot Model

Seksi ini terdiri dari dua tahap yang ditampilkan pada Tabel 4.52 dan Tabel 4.53.

Tabel 4.52. Cell Konstruksi dan *Load* Bobot VAE Pada `Testing.ipynb`

```
# Load hyperparameter terbaik
vae_progress_file = os.path.join(hyperparameters_dir,
'tuning_progress_vae.json')
with open(vae_progress_file, 'r') as f:
    best_vae_params = json.load(f)['best_params']

# Membangun Arsitektur VAE
encoder =
Encoder(hidden_dims=best_vae_params['hidden_dims'],
latent_dim=best_vae_params['latent_dim'],
dropout_rate=best_vae_params['dropout_rate'])
decoder =
Decoder(hidden_dims=best_vae_params['hidden_dims'][:-1],
output_dim=num_items)
```

```

eval_vae = VAE(encoder, decoder,
beta=best_vae_params['beta'])

# Menyuntikkan Bobot ke VAE
_ = eval_vae(train_data[:1])
eval_vae.load_weights(os.path.join(save_dir,
'trained_best_vae_weights.weights.h5'))

```

Program pada Tabel 4.52 membangun ulang arsitektur VAE dari hyperparameter terbaik yang dimuat dari `tuning_progress_vae.json`, kemudian memuat bobot terlatih dari file `.h5` yang dihasilkan oleh `Training.ipynb`. *Forward pass* awal pada satu sampel data (`train_data[:1]`) dilakukan terlebih dahulu untuk menginisialisasi bobot model sebelum `load_weights` dipanggil, karena TensorFlow memerlukan bobot model untuk dibangun terlebih dahulu sebelum dapat dimuat dari file.

Tabel 4.53. Cell *Load* Komponen Bobot RSVD Pada *Testing.ipynb*

```

# Memuat komponen Bias
mu = np.load(os.path.join(save_dir, 'final_mu.npy'))
b_u = np.load(os.path.join(save_dir, 'final_b_u.npy'))
b_i = np.load(os.path.join(save_dir, 'final_b_i.npy'))

# Memuat komponen Laten
U = np.load(os.path.join(save_dir, 'best_U.npy'))
Sigma = np.load(os.path.join(save_dir, 'best_Sigma.npy'))
V = np.load(os.path.join(save_dir, 'best_V.npy'))

```

Program pada Tabel 4.53 memuat enam komponen bobot RSVD yang disimpan secara terpisah oleh `Training.ipynb` dalam format `.npy`. Komponen *bias* terdiri dari *global bias* `mu`, *user bias* `b_u`, dan *item bias* `b_i`. Komponen *laten* terdiri dari matriks faktor pengguna `U`, matriks nilai singular `Sigma`, dan matriks faktor item `V`. Keenam komponen ini akan digabungkan pada tahap prediksi untuk merekonstruksi matriks *rating* penuh.

5. Prediksi Model

Seksi prediksi terdiri dari dua tahap yang ditampilkan pada Tabel 4.54 dan Tabel 4.55.

Tabel 4.54. Cell Prediksi VAE Pada Testing.ipynb

```
# Ubah train data menjadi tensor
train_data_tf = tf.constant(train_data, dtype=tf.float32)

# Ekstrak matriks laten
Z_mean, Z_log_var =
eval_vae.encoder.predict(train_data_tf, verbose=0)

# Prediksi dari Decoder (skala 0 - 1)
pred_vae_norm = eval_vae.decoder.predict(Z_mean,
verbose=0)

# Filter ke koordinat test
pred_vae_test = pred_vae_norm[test_indices]

# Kembalikan ke skala asli
pred_vae_test_denorm = pred_vae_test * MAX_RATING

# Batasi prediksi VAE agar tidak keluar dari rentang
rating yang valid (1.0 - 5.0)
pred_vae_test_clipped = np.clip(pred_vae_test_denorm, 1.0,
MAX_RATING)
```

Program pada Tabel 4.54 menghasilkan prediksi *rating* VAE dalam lima langkah berurutan. Pertama, data latih dikonversi ke `tf.constant` untuk diproses oleh TensorFlow. Kedua, `encoder.predict` mengekstrak representasi *laten* `Z_mean` dari seluruh pengguna. Ketiga, `decoder.predict` merekonstruksi prediksi *rating* penuh dalam skala 0–1 dari `Z_mean`. Keempat, prediksi difilter ke koordinat *test set* menggunakan `test_indices`. Kelima, prediksi didenormalisasi ke skala 1–5 dan dibatasi menggunakan `np.clip` agar tidak melebihi rentang *rating* yang valid.

Tabel 4.55. Cell Prediksi RSVD Pada Testing.ipynb

```

# Kalkulasi matriks laten
latent_matrix = np.dot(np.dot(U, Sigma), V.T)

# Menghitung matriks bias (rata-rata global + bias
pengguna + bias film)
bias_matrix = float(mu) + b_u[:, np.newaxis] +
b_i[np.newaxis, :]

# Gabungkan bias dan interaksi laten menjadi tebakan penuh
(Skala 0 - 1)
full_rsvd_pred = bias_matrix + latent_matrix

# Filter untuk indeks test data saja
pred_rsvd_test = full_rsvd_pred[test_indices]

# Kembalikan prediksi ke skala asli
pred_rsvd_test_denorm = pred_rsvd_test * MAX_RATING

# Batasi prediksi RSVD agar tidak keluar dari rentang
rating yang valid (1.0 - 5.0)
pred_rsvd_test_clipped = np.clip(pred_rsvd_test_denorm,
1.0, MAX_RATING)

```

Program pada Tabel 4.55 merekonstruksi prediksi *rating* RSVD dari komponen-komponen bobot yang telah dimuat. Matriks interaksi *laten* diperoleh dari perkalian *U*, *Sigma*, dan *V.T*. Matriks *bias* dibangun dengan menjumlahkan *global bias* *mu*, *user bias* *b_u* (diperluas secara kolom menggunakan `[:, np.newaxis]`), dan *item bias* *b_i* (diperluas secara baris menggunakan `[np.newaxis, :]`) sehingga menghasilkan matriks berukuran 943×1682 . Prediksi penuh diperoleh dari penjumlahan kedua matriks tersebut, kemudian difilter ke koordinat *test set*, didenormalisasi, dan diclip ke rentang rating yang valid.

6. Pencarian dan Penerapan Bobot *Ensemble*

Seksi ini terdiri dari dua tahap yang ditampilkan pada Tabel 4.56 dan Tabel 4.57.

Tabel 4.56. Cell Pencarian Bobot Alpha *Ensemble* Pada Testing.ipynb

```
# Ekstrak prediksi VAE dan RSVD di koordinat validasi
# Denormalisasi dan clip ke skala 1-5
pred_vae_val = np.clip(pred_vae_norm[val_indices] *
MAX_RATING, 1.0, MAX_RATING)
pred_rsvd_val = np.clip(full_rsvd_pred[val_indices] *
MAX_RATING, 1.0, MAX_RATING)

best_val_rmse = float("inf")
best_alpha = 0.0
best_beta = 0.0

# Menyimpan history untuk visualisasi kurva tuning
alpha_history = []
rmse_history = []

# Grid search alpha dari 0.00 hingga 1.00 dengan langkah
0.01
# Alpha = proporsi kontribusi VAE, (1 - alpha) = proporsi
kontribusi RSVD
for alpha in np.linspace(0, 1, 101):
    beta = 1.0 - alpha
    pred_ensemble = np.clip((alpha * pred_vae_val) + (beta
* pred_rsvd_val), 1.0, MAX_RATING)
    rmse =
np.sqrt(np.mean(np.square(actual_val_values -
pred_ensemble)))

    alpha_history.append(alpha)
    rmse_history.append(rmse)

    if rmse < best_val_rmse:
        best_val_rmse = rmse
```

```
best_alpha = alpha
best_beta = beta
```

Program pada Tabel 4.56 Cell ini mencari bobot alpha optimal untuk *weighted average ensemble* menggunakan data validasi. Prediksi VAE dan RSVD diekstrak pada koordinat validasi (*val_indices*), didenormalisasi, dan diclip ke skala 1–5. Selanjutnya, *grid search* dilakukan terhadap 101 nilai alpha dari 0,00 hingga 1,00 dengan langkah 0,01. Pada setiap iterasi, prediksi *ensemble* dihitung sebagai $(\alpha \times \text{VAE}) + ((1 - \alpha) \times \text{RSVD})$ dan RMSE-nya dievaluasi terhadap nilai aktual validasi. Nilai alpha yang menghasilkan RMSE terkecil dipilih sebagai bobot terbaik. Riwayat nilai RMSE per alpha disimpan dalam *rmse_history* untuk keperluan visualisasi.

Tabel 4.57. Cell Penerapan Alpha ke Test Set Pada Testing.ipynb

```
# Terapkan alpha terbaik (yang ditemukan di validasi) ke
prediksi test set
pred_vae_test_clipped =
np.clip(pred_vae_norm[test_indices] * MAX_RATING, 1.0,
MAX_RATING)
pred_rsvd_test_clipped =
np.clip(full_rsvd_pred[test_indices] * MAX_RATING, 1.0,
MAX_RATING)

pred_hybrid = (best_alpha * pred_vae_test_clipped) +
(best_beta * pred_rsvd_test_clipped)

# Batasi prediksi agar tidak keluar dari rentang rating
yang valid (1.0 - 5.0)
pred_hybrid_clipped = np.clip(pred_hybrid, 1.0,
MAX_RATING)
```

Program pada Tabel 4.57 menerapkan alpha terbaik yang diperoleh dari data validasi ke prediksi *test set*. Prediksi VAE dan RSVD pada koordinat *test set* didenormalisasi dan diclip terlebih dahulu, kemudian digabungkan dengan formula *weighted average* menggunakan *best_alpha* dan *best_beta*. Prediksi

hybrid hasil penggabungan diclip sekali lagi untuk memastikan seluruh nilai berada dalam rentang 1,0–5,0.

7. Evaluasi dan Penyimpanan Hasil

Seksi evaluasi terdiri dari tiga tahap yang ditampilkan pada Tabel 4.58, Tabel 4.59, dan Tabel 4.60.

Tabel 4.58. Fungsi `calculate_metrics` Pada `Testing.ipynb`

```
# Fungsi helper untuk menghitung MSE, RMSE, dan MAE
# sekaligus
def calculate_metrics(y_true, y_pred):
    mse = np.mean(np.square(y_true - y_pred))
    rmse = np.sqrt(mse)
    mae = np.mean(np.abs(y_true - y_pred))
    return mse, rmse, mae
```

Fungsi `calculate_metrics` pada Tabel 4.58 menghitung tiga metrik evaluasi regresi sekaligus dari pasangan nilai aktual dan prediksi. Implementasi fungsi ini identik dengan yang digunakan pada `CrossValidation.ipynb`, didefinisikan ulang secara lokal agar *notebook* ini dapat dijalankan secara independen tanpa bergantung pada *notebook* lain.

Tabel 4.59. Cell Perhitungan Metrik Pada `Testing.ipynb`

```
# Hitung metrik evaluasi untuk ketiga model di test set
mse_v, rmse_v, mae_v =
calculate_metrics(actual_test_values,
pred_vae_test_clipped) # VAE
mse_r, rmse_r, mae_r =
calculate_metrics(actual_test_values,
pred_rsvd_test_clipped) # RSVD
mse_h, rmse_h, mae_h =
calculate_metrics(actual_test_values, pred_hybrid_clipped)
# Hybrid
```

Program pada Tabel 4.59 menghitung MSE, RMSE, dan MAE untuk ketiga model pada *test set* menggunakan fungsi `calculate_metrics`. Hasil numerik dari cell ini merupakan performa akhir model.

Tabel 4.60. Cell Penyimpanan Hasil Evaluasi Pada `Testing.ipynb`

```
# Simpan seluruh ringkasan metrik agar bisa dipanggil
kembali tanpa perlu re-run
evaluation_results = {
    'timestamp': timestamp,
    'ensemble_weights': {
        'best_alpha_vae': float(best_alpha),
        'best_beta_rsvd': float(best_beta),
        'best_val_rmse': float(best_val_rmse)
    },
    'metrics': {
        'vae_standalone': {'MSE': float(mse_v), 'RMSE':
float(rmse_v), 'MAE': float(mae_v)},
        'rsvd_standalone': {'MSE': float(mse_r), 'RMSE':
float(rmse_r), 'MAE': float(mae_r)},
        'hybrid_ensemble': {'MSE': float(mse_h), 'RMSE':
float(rmse_h), 'MAE': float(mae_h)}
    }
}

evaluation_file_path = os.path.join(results_dir,
f'final_testing_results_{timestamp}.json')

with open(evaluation_file_path, 'w') as f:
    json.dump(evaluation_results, f, indent=4)
```

Program pada Tabel 4.60 menyimpan seluruh hasil evaluasi akhir ke dalam file JSON di direktori `EvaluationResults`. Struktur JSON mencakup *timestamp* sesi pengujian, bobot *ensemble* optimal (`best_alpha`, `best_beta`, `best_val_rmse`), serta metrik MSE, RMSE, dan MAE untuk ketiga model. Penyimpanan hasil dalam format JSON memungkinkan hasil evaluasi

dipanggil kembali untuk keperluan pelaporan tanpa perlu menjalankan ulang seluruh proses pengujian.

4.2 Hasil *Preprocessing* Data

Preprocessing data dilakukan untuk mempersiapkan dataset *MovieLens* 100K menjadi representasi numerik yang siap digunakan oleh model VAE dan RSVD. Tahap ini mencakup tiga proses utama, yaitu pembangunan matriks interaksi *user-item*, normalisasi *rating*, dan pembagian data menjadi partisi *train*, *validation*, dan *test*. Hasil dari setiap proses dirangkum pada Tabel 4.61.

Tabel 4.61. Atribut Dataset Setelah *Preprocessing*

Atribut	Nilai
Jumlah Pengguna (User)	943
Jumlah Film (Item)	1.682
Total <i>Rating</i>	100.000
Dimensi Matriks Interaksi	943 x 1.682
Total Sel Matriks	1.586.126
<i>Density</i> Matriks	6,30%
<i>Sparsity</i> Matriks	93,70%
Skala <i>Rating</i> Asli	1 – 5
Rentang <i>Rating</i> Setelah Normalisasi	0,2 – 1,0
Jumlah <i>Rating Train</i>	70.000
Jumlah <i>Rating Validation</i>	10.000
Jumlah <i>Rating Testing</i>	20.000

Dataset *MovieLens* 100K dimuat dari *file* u.data yang memuat 100.000 entri *rating* dari 943 pengguna terhadap 1.682 film. Data tabular tersebut diubah menjadi matriks interaksi *user-item* berukuran 943×1.682 menggunakan operasi *pivot*, menghasilkan matriks dengan 1.586.126 sel di mana hanya 100.000 sel (6,30%) yang terisi *rating* eksplisit dan sisanya (93,70%) bernilai nol yang merepresentasikan tidak adanya interaksi. Tingkat *sparsity* sebesar 93,70% ini merupakan karakteristik umum pada dataset sistem rekomendasi dan menjadi salah

satu motivasi penggunaan model berbasis representasi laten seperti VAE dan RSVD.

Seluruh nilai *rating* dinormalisasi ke rentang $[0, 1]$ dengan membaginya terhadap nilai *rating* maksimum ($\text{MAX_RATING} = 5,0$). Strategi ini dipilih agar sel bernilai nol yang merepresentasikan ketiadaan *rating* tetap bernilai nol setelah normalisasi, sehingga dapat dibedakan dari *rating* terendah yang bernilai 0,2 (yaitu $1/5$). Dengan demikian, *masked reconstruction loss* pada VAE dapat mengidentifikasi posisi *rating* eksplisit secara akurat menggunakan kondisi nilai > 0 .

Pembagian data dilakukan pada level *rating* individual menggunakan proporsi 70/10/20 dengan *random seed* tetap ($\text{RANDOM_SEED} = 42$) untuk menjamin reproduksibilitas. Dari total 100.000 *rating*, sebanyak 70.000 *rating* dialokasikan sebagai data *train*, 10.000 sebagai data *validation*, dan 20.000 sebagai data *test*. Data *validation* digunakan secara eksklusif pada proses pencarian bobot *ensemble alpha*, sedangkan data *test* hanya diakses satu kali pada tahap pengujian *final* untuk menghindari kebocoran data (*data leakage*). Selain keempat matriks tersebut, *normalized_matrix* (matriks *rating* penuh sebelum *split*) juga disimpan secara khusus sebagai masukan bagi *CrossValidation.ipynb* agar setiap *fold* dapat membentuk partisinya sendiri secara independen.

4.3 Hasil Hyperparameter Tuning

Proses pencarian *hyperparameter* optimal dilakukan secara terpisah untuk model VAE dan RSVD menggunakan pendekatan yang berbeda sesuai dengan karakteristik masing-masing model. VAE memiliki ruang pencarian yang besar dan tidak terstruktur sehingga digunakan Optuna dengan algoritma *Tree-structured Parzen Estimator* (TPE), sedangkan RSVD memiliki ruang pencarian yang lebih kecil sehingga dapat dijelajahi secara *exhaustif* menggunakan *grid search*.

4.3.1 Hasil Hyperparameter Tuning VAE

Pencarian *hyperparameter* VAE dilakukan menggunakan Optuna TPE dengan 200 *trial* terhadap ruang pencarian yang telah didefinisikan pada Tabel 3.3. Setiap *trial* melatih VAE selama 30 *epoch* dan mengembalikan nilai *reconstruction loss* akhir sebagai kriteria seleksi. Penggunaan *reconstruction loss* sebagai kriteria

seleksi dilakukan untuk menghindari bias terhadap nilai β yang kecil, karena $total\ loss$ akan lebih kecil secara artifisial ketika β bernilai rendah meskipun kualitas rekonstruksi tidak lebih baik. Setelah 200 *trial* selesai dijalankan, Optuna mengidentifikasi kombinasi *hyperparameter* terbaik yang ditampilkan pada Tabel 4.62. Nilai *reconstruction loss* terbaik sebesar 7,2457 merupakan nilai *masked MSE* dalam skala yang telah dikalikan jumlah item (1.682), sesuai dengan formula *reconstruction loss* pada persamaan (5). Nilai ini bukan RMSE pada skala *rating* asli dan tidak dapat dibandingkan langsung dengan metrik evaluasi akhir.

Tabel 4.62. Hyperparameter Terbaik VAE

<i>Hyperparameter</i>	Nilai Terbaik
latent_dim	100
learning_rate	0,001
batch_size	64
dropout_rate	0,1
hidden_dims	[1024, 512]
beta	0,01
<i>Reconstruction loss</i> terbaik	7,2457

Kombinasi terbaik menggunakan $latent_dim = 100$ yang memberikan ruang representasi cukup besar untuk menangkap keragaman preferensi pengguna tanpa terlalu memperbesar kompleksitas model. Nilai $\beta = 0,01$ mengindikasikan bahwa regularisasi KL *divergence* yang ringan memberikan hasil rekonstruksi terbaik pada dataset ini, sejalan dengan karakteristik data *sparse* di mana tekanan KL yang terlalu kuat cenderung menyebabkan *posterior collapse*. Arsitektur $hidden_dims = [1024, 512]$ dengan $dropout_rate = 0,1$ menunjukkan bahwa model berkapasitas lebih besar dengan regularisasi minimal lebih efektif dibandingkan arsitektur yang lebih kecil atau *dropout* yang lebih agresif.

4.3.2 Hasil Hyperparameter Tuning RSVD

Pencarian *hyperparameter* RSVD dilakukan menggunakan *exhaustive grid search* terhadap 48 kombinasi yang dihasilkan dari ruang pencarian yang telah didefinisikan pada Tabel 3.4. Setiap kombinasi melatih RSVD selama 15 *epoch* dan dievaluasi menggunakan *validation MSE* pada data validasi yang disisihkan secara internal oleh model. Setelah seluruh 48 kombinasi selesai dievaluasi, kombinasi dengan *validation error* terkecil dipilih sebagai *hyperparameter* terbaik yang ditampilkan pada Tabel 4.63. Nilai *validation error* terbaik sebesar 0,2801 merupakan *validation MSE* dalam skala *rating* yang telah dinormalisasi ke rentang $[0, 1]$, sehingga tidak dapat dibandingkan langsung dengan metrik evaluasi akhir yang beroperasi pada skala *rating* asli (1–5).

Tabel 4.63. Hyperparameter Terbaik RSVD

<i>Hyperparameter</i>	Nilai Terbaik
<i>n_factors</i>	10
<i>learning_rate</i>	0,005
<i>lambda_reg</i>	0,001
<i>Validation error</i> terbaik	0,2801

Nilai *n_factors* = 10 yang optimal mengindikasikan bahwa representasi laten berdimensi rendah sudah cukup untuk menangkap pola interaksi *user-item* pada dataset *MovieLens* 100K tanpa *overfitting*, didukung oleh nilai *lambda_reg* = 0,001 yang memberikan regularisasi ringan. Nilai *learning_rate* = 0,005 merupakan titik keseimbangan antara kecepatan konvergensi dan stabilitas pelatihan SGD pada dataset ini.

4.4 Performa Sistem Rekomendasi

Evaluasi performa sistem rekomendasi dilakukan menggunakan dua skema yang saling melengkapi yaitu *10-fold cross-validation* untuk mengukur konsistensi dan kemampuan generalisasi model lintas partisi data, serta pengujian *final* pada *hold-out test set* untuk mengukur performa pada data yang sepenuhnya belum pernah dilihat selama proses pelatihan maupun *tuning*. Signifikansi statistik dari

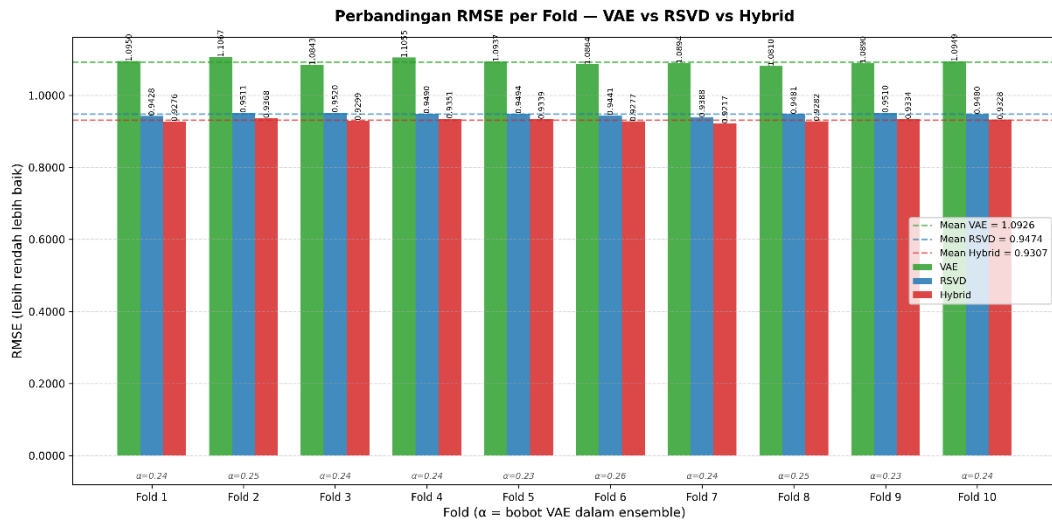
perbedaan performa antar model dikonfirmasi melalui *paired t-test*. Hasil evaluasi pada bagian ini menjawab rumusan masalah (a) mengenai performa sistem rekomendasi yang dikembangkan.

4.4.1 Hasil Cross-Validation

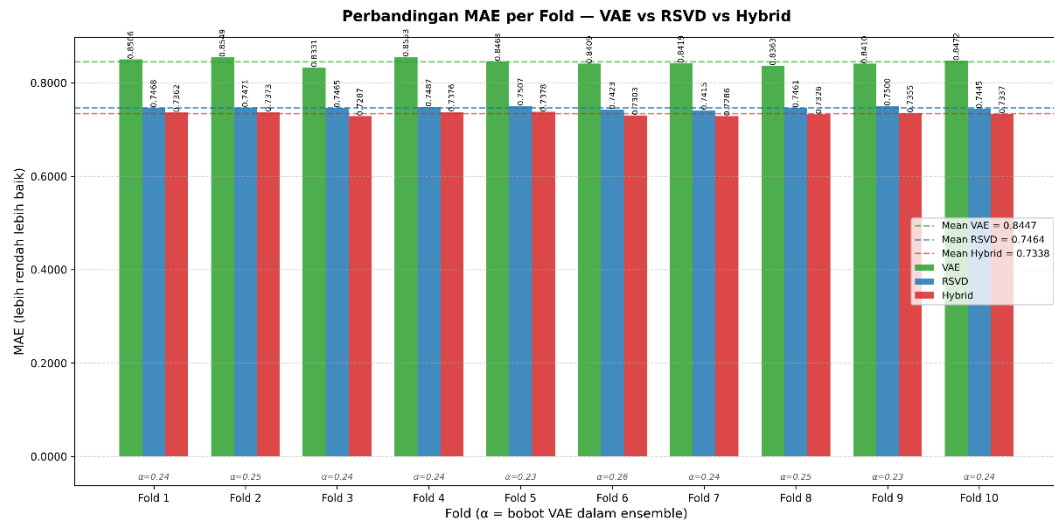
Evaluasi *cross-validation* dilakukan menggunakan skema 10-fold pada *normalized_matrix* (matriks *rating* penuh) dengan *random seed* tetap (RANDOM_SEED = 42). Pada setiap *fold*, VAE dilatih selama 50 *epoch* dan RSVD selama 30 *epoch* menggunakan *hyperparameter* terbaik hasil tuning. Bobot *ensemble* alpha dicari secara independen pada *validation inner* setiap *fold* menggunakan *grid search* dengan langkah 0,01.

a. Metrik Per Fold

Perbandingan RMSE ketiga model pada setiap *fold* ditampilkan pada Gambar 4.2, sedangkan perbandingan MAE ditampilkan pada Gambar 4.3.



Gambar 4.2. Perbandingan RMSE Per Fold

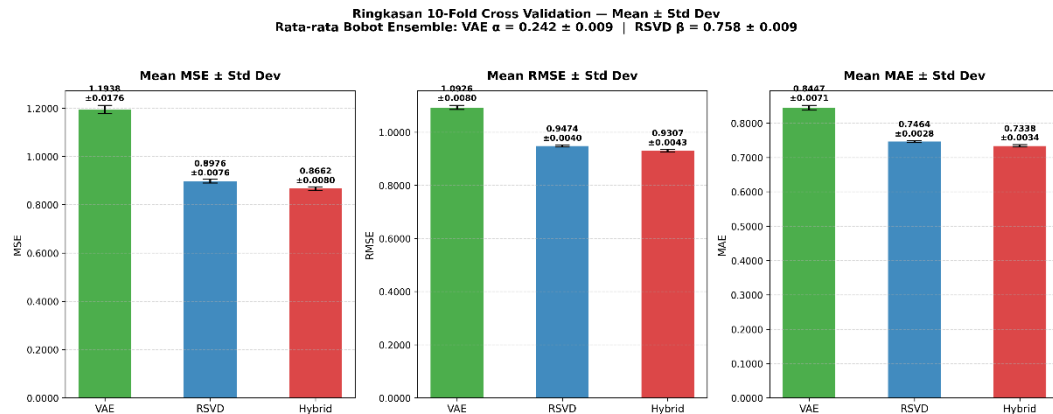


Gambar 4.3. Perbandingan MAE Per *Fold*

Gambar 4.2 dan Gambar 4.3 menunjukkan bahwa pada seluruh *fold* tanpa kecuali, *hybrid ensemble* menghasilkan RMSE dan MAE yang lebih rendah dibandingkan kedua model individual. Garis putus-putus horizontal pada masing-masing gambar merepresentasikan nilai rata-rata lintas *fold* untuk setiap model, yang memperlihatkan bahwa fluktuasi antar *fold* sangat kecil dan ketiga model beroperasi pada rentang yang stabil. Nilai *alpha* VAE yang ditampilkan pada sumbu-x setiap *fold* berkisar antara 0,23 hingga 0,26, mengonfirmasi bahwa proporsi kontribusi optimal VAE terhadap *ensemble* adalah konsisten sekitar 25% di seluruh partisi data.

b. Ringkasan Statistik

Ringkasan rata-rata dan standar deviasi metrik dari 10 *fold* divisualisasikan pada Gambar 4.4 dan dirangkum secara numerik pada Tabel 4.64.



Gambar 4.4. Perbandingan MAE Per *Fold*

Tabel 4.64. Ringkasan Statistik RMSE dan MAE 10-Fold Cross-Validation

Model	RMSE	RMSE Min	RMSE Max	MAE	MAE Min	MAE Max
VAE	1,0926 $\pm$ 0,0080	1,0810	1,1067	0,8447 $\pm$ 0,0071	0,8331	0,8553
RSVD	0,9474 $\pm$ 0,0040	0,9388	0,9520	0,7464 $\pm$ 0,0028	0,7415	0,7507
Hybrid	0,9307 $\pm$ 0,0043	0,9217	0,9368	0,7338 $\pm$ 0,0034	0,7286	0,7378

Hybrid ensemble mencapai RMSE rata-rata sebesar $0,9307 \pm 0,0043$ dan MAE rata-rata sebesar $0,7338 \pm 0,0034$, mengungguli RSVD (RMSE $0,9474 \pm 0,0040$) dan VAE (RMSE $1,0926 \pm 0,0080$). Standar deviasi yang rendah pada ketiga model mengindikasikan stabilitas performa yang baik di seluruh *fold*, dengan *hybrid* memiliki standar deviasi RMSE yang sebanding dengan RSVD ($0,0043$ vs $0,0040$), menunjukkan bahwa penggabungan kedua model tidak mengorbankan stabilitas.

c. Stabilitas *Alpha*

Nilai *alpha* VAE yang diperoleh dari pencarian independen pada setiap *fold* berkisar antara 0,23 hingga 0,26 dengan rata-rata $0,242 \pm 0,009$ (terdapat pada Gambar 4.2, 4.3, dan 4.4). Konsistensi ini menunjukkan

bahwa proporsi kontribusi optimal VAE terhadap *ensemble* adalah sekitar 25% (VAE) dan 75% (RSVD), dan nilai tersebut stabil terhadap perubahan partisi data. Stabilitas *alpha* lintas *fold* memperkuat validitas pemilihan bobot *ensemble* yang digunakan pada tahap pengujian *final*.

4.4.2 Hasil Uji Signifikansi Statistis (*Paired t-Test*)

Uji *paired t-test* dilakukan untuk menentukan apakah perbedaan performa antar model yang teramati pada 10 *fold* signifikan secara statistik atau dapat terjadi secara kebetulan. Pengujian dilakukan pada tingkat signifikansi $\alpha = 0,05$ dengan hipotesis nol (H_0) bahwa tidak terdapat perbedaan performa yang signifikan antara kedua model yang dibandingkan. Tiga pasang perbandingan diuji untuk metrik RMSE dan MAE, yaitu *Hybrid* vs RSVD, *Hybrid* vs VAE, dan RSVD vs VAE. Hasil pengujian ditampilkan pada Tabel 4.65.

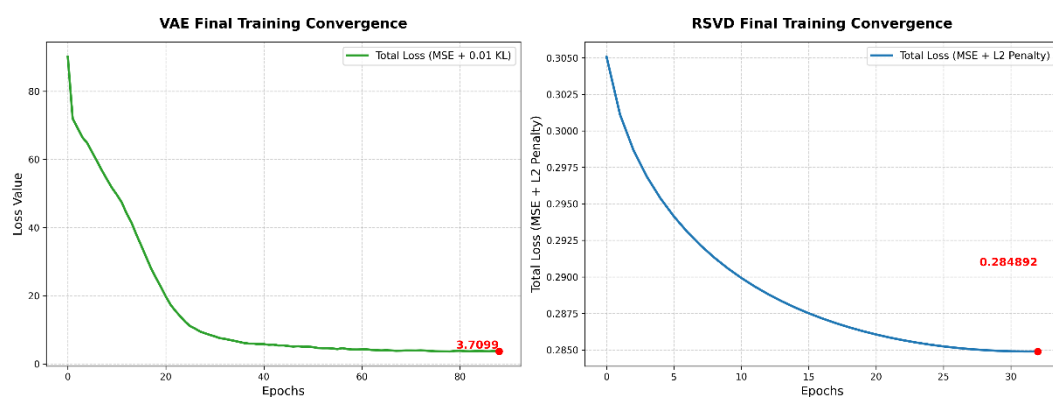
Tabel 4.65. Hasil *Paired t-Test* Antar Model

Perbandingan	Metrik	t-statistik	p-value	Kesimpulan
<i>Hybrid</i> vs RSVD	RMSE	-20,2548	$8,12 \times 10^{-9}$	Signifikan
<i>Hybrid</i> vs VAE	RMSE	-77,4756	$5,03 \times 10^{-14}$	Signifikan
RSVD vs VAE	RMSE	-52,0504	$1,79 \times 10^{-12}$	Signifikan
<i>Hybrid</i> vs RSVD	MAE	-17,0866	$3,62 \times 10^{-8}$	Signifikan
<i>Hybrid</i> vs VAE	MAE	-67,1092	$1,83 \times 10^{-13}$	Signifikan
RSVD vs VAE	MAE	-42,7360	$1,05 \times 10^{-11}$	Signifikan

Seluruh enam perbandingan menghasilkan *p-value* yang jauh di bawah ambang signifikansi 0,05, sehingga H_0 ditolak pada seluruh pasang perbandingan. Nilai *t-statistik* yang negatif pada seluruh perbandingan mengonfirmasi bahwa model pertama pada setiap pasang memiliki nilai metrik yang lebih rendah (lebih baik) secara konsisten lintas *fold*. Besarnya nilai *t-statistik* absolut mengindikasikan bahwa perbedaan performa bukan sekadar signifikan secara statistik, tetapi juga memiliki *effect size* yang sangat besar. Hasil ini memberikan dasar statistik bahwa keunggulan *hybrid ensemble* atas kedua model individual adalah nyata dan dapat direplikasi.

4.4.3 Hasil Pelatihan Model

Pelatihan model dilakukan dalam satu sesi penuh menggunakan seluruh data *train* (70.000 *rating*) dengan *hyperparameter* terbaik. Pelatihan VAE menggunakan maksimum 500 *epoch* dengan *early stopping* (*patience* = 10) yang memantau *training loss*, serta *KL annealing* yang menaikkan nilai *beta* secara linear dari 0 menuju *beta* target (0,01) selama 30 *epoch* pertama. Pelatihan RSVD menggunakan maksimum 100 *epoch* dengan *early stopping* internal yang memantau *validation MSE*. Kurva konvergensi kedua model ditampilkan pada Gambar 4.5.



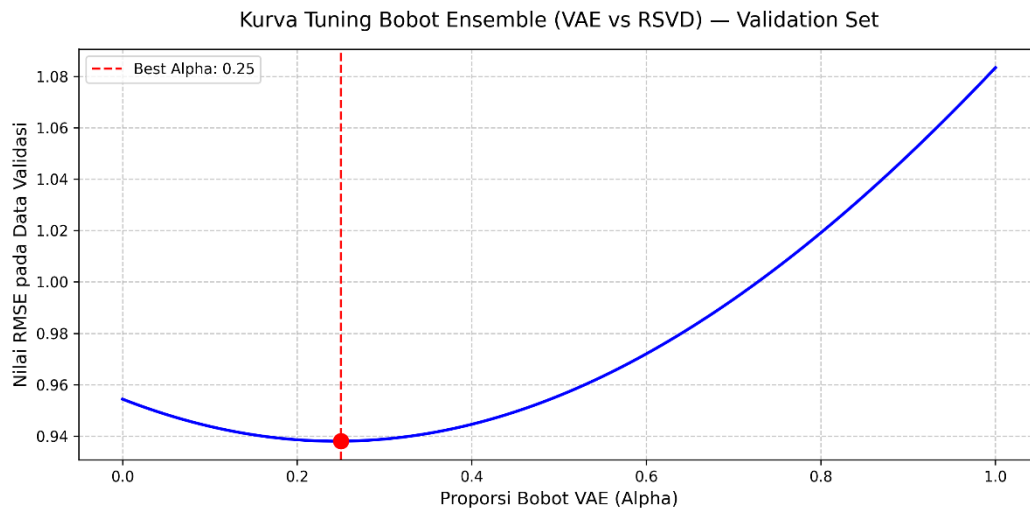
Gambar 4.5. Kurva Konvergensi VAE dan RSVD

Gambar 4.5 menunjukkan bahwa VAE mencapai konvergensi pada *epoch* ke-90 dengan *total loss* akhir sebesar 3,7099, sedangkan RSVD mencapai konvergensi pada *epoch* ke-33 dengan *total loss* akhir sebesar 0,2849. Kedua kurva menunjukkan penurunan *loss* yang monoton dan mulus tanpa osilasi yang berarti proses pelatihan berjalan stabil. Konvergensi RSVD yang jauh lebih cepat konsisten dengan kompleksitas komputasinya yang lebih rendah dibandingkan VAE. Bobot VAE disimpan dalam format *.weights.h5*, sedangkan komponen bobot RSVD (*U*, *Sigma*, *V*, *b_u*, *b_i*, *mu*) disimpan dalam format *.npy* untuk digunakan pada tahap pengujian *final*.

4.4.4 Hasil Pengujian Model

Pengujian *final* dilakukan menggunakan bobot model yang dihasilkan dari pelatihan, dievaluasi terhadap *test set* (20.000 *rating*) yang tidak pernah dilihat selama proses pelatihan maupun tuning. Sebelum evaluasi pada *test set*, bobot *ensemble alpha* dicari terlebih dahulu pada *validation set* menggunakan *grid search*

dengan langkah 0,01. Kurva RMSE validasi terhadap seluruh nilai alpha yang diuji ditampilkan pada Gambar 4.6.



Gambar 4.6. Kurva Tuning Bobot *Ensemble Alpha* pada *Validation Set*

Gambar 4.6 menunjukkan bahwa kurva RMSE validasi berbentuk U asimetris dengan titik minimum pada $\alpha = 0,25$ (RMSE validasi = 0,9380). Kurva menanjak lebih curam ke arah kanan dibandingkan ke arah kiri, mengindikasikan bahwa model *ensemble* lebih sensitif terhadap peningkatan proporsi VAE daripada peningkatan proporsi RSVD. Nilai $\alpha = 0,25$ ini kemudian diterapkan pada *test set* untuk menghasilkan prediksi *hybrid* final. Hasil metrik ketiga model pada *test set* ditampilkan pada Tabel 4.66.

Tabel 4.66. Hasil Pengujian Pada *Test Set*

Model	MSE	RMSE	MAE
VAE	1,1829	1,0876	0,8446
RSVD	0,9037	0,9506	0,7508
<i>Hybrid Ensemble</i>	0,8771	0,9365	0,7401

Hybrid ensemble mencapai RMSE sebesar 0,9365 dan MAE sebesar 0,7401 pada *test set*, mengungguli RSVD (RMSE 0,9506) dan VAE (RMSE 1,0876). Nilai $\alpha = 0,25$ yang diperoleh dari *validation set* pada pengujian *final* ini konsisten dengan rata-rata α lintas *fold* pada *cross-validation* ($0,242 \pm 0,009$),

mengonfirmasi bahwa proporsi kontribusi optimal kedua model bersifat stabil dan tidak bergantung pada partisi data tertentu.

4.5 Model Optimal

Dalam sistem rekomendasi *hybrid*, menentukan seberapa besar kontribusi masing-masing model terhadap prediksi akhir merupakan keputusan kritis yang langsung memengaruhi akurasi. Bagian ini menyajikan bobot *ensemble* yang optimal antara VAE dan RSVD serta analisis mengapa konfigurasi *hybrid* tersebut secara konsisten mengungguli kedua model individual, sebagai jawaban atas rumusan masalah (b) mengenai model yang optimal untuk sistem rekomendasi dengan VAE dan RSVD.

4.5.1 Bobot Ensemble Optimal

Pencarian bobot ensemble optimal dilakukan melalui grid search pada validation set dengan langkah 0,01, menjelajahi seluruh kombinasi alpha dari 0,00 hingga 1,00 di mana alpha merepresentasikan proporsi kontribusi VAE dan (1 - alpha) merepresentasikan proporsi kontribusi RSVD. Hasil pencarian divisualisasikan pada Gambar 4.6 yang memperlihatkan kurva RMSE validasi berbentuk U asimetris. Titik minimum kurva berada pada $\alpha = 0,25$ dengan RMSE validasi sebesar 0,9380, dan kurva menanjak lebih curam ke arah kanan dibandingkan ke arah kiri mengindikasikan bahwa model ensemble jauh lebih sensitif terhadap peningkatan proporsi VAE daripada peningkatan proporsi RSVD. Bobot optimal $\alpha = 0,25$ ini selanjutnya diterapkan pada test set untuk menghasilkan prediksi hybrid final, dengan hasil metrik ketiga model ditampilkan pada Tabel 4.66.

Pada test set, hybrid ensemble mencapai RMSE sebesar 0,9365 dan MAE sebesar 0,7401, mengungguli RSVD (RMSE 0,9506; MAE 0,7508) dan VAE (RMSE 1,0876; MAE 0,8446). Yang perlu digarisbawahi adalah konsistensi antara $\alpha = 0,25$ yang diperoleh dari validation set pada pengujian final ini dengan rata-rata alpha lintas fold pada cross-validation ($0,242 \pm 0,009$). Konsistensi ini mengonfirmasi bahwa proporsi kontribusi optimal kedua model tidak bersifat kebetulan atau bergantung pada partisi data tertentu, melainkan mencerminkan karakteristik inheren dari interaksi kedua model pada dataset MovieLens 100K.

4.5.2 Perbandingan *Hybrid* dan *Standalone*

Keunggulan hybrid ensemble atas kedua model individual dapat dijelaskan melalui sifat komplementer VAE dan RSVD. Sebagaimana terlihat pada Gambar 4.2 dan Gambar 4.3, hybrid ensemble secara konsisten menghasilkan RMSE dan MAE yang lebih rendah dibandingkan kedua model individual pada seluruh fold tanpa kecuali, mengindikasikan bahwa keunggulan ini bukan merupakan artefak dari partisi data tertentu melainkan pola yang sistematis. VAE memodelkan distribusi probabilistik preferensi pengguna di ruang laten sehingga mampu menangkap pola implisit dan non-linear antar pengguna, sedangkan RSVD secara eksplisit memodelkan bias global, bias pengguna, dan bias item melalui komponen *bias_matrix*, sehingga lebih efektif dalam menangkap tendensi sistematis seperti pengguna yang cenderung memberi rating tinggi atau film yang secara umum dinilai rendah. Penggabungan manfaat generalisasi pola non-linear dari VAE dan memorisasi interaksi fitur dari RSVD dalam satu model hybrid menghasilkan prediksi yang lebih akurat dibandingkan masing-masing model secara individual (Bougteb et al., 2022).

Dominansi kontribusi RSVD sebesar 75% pada bobot ensemble mengindikasikan bahwa pada dataset MovieLens 100K, struktur bias dan faktor laten linear yang dimodelkan RSVD lebih informatif dibandingkan representasi probabilistik VAE. Hal ini konsisten dengan karakteristik dataset yang memiliki sparsity tinggi sebesar 93,70%. VAE dengan arsitektur [1024, 512] dan *latent_dim* = 100 memiliki jutaan parameter yang harus diestimasi dari sinyal yang relatif terbatas, sehingga kontribusinya lebih kecil. Meski demikian, kontribusi VAE sebesar 25% tetap memberikan peningkatan yang signifikan secara statistik saat digabungkan, sebagaimana dikonfirmasi oleh hasil paired t-test pada Tabel 4.65. Dengan demikian, model hybrid ensemble dengan bobot $\alpha = 0,25$ untuk VAE dan $\beta = 0,75$ untuk RSVD merupakan konfigurasi optimal yang mampu memanfaatkan kekuatan kedua model secara sinergis.

4.6 Pembahasan

Hasil eksperimen secara keseluruhan menunjukkan bahwa pendekatan *hybrid ensemble* yang menggabungkan VAE dan RSVD melalui *weighted average*

berhasil menghasilkan prediksi *rating* yang lebih akurat dibandingkan kedua model individual pada seluruh skema evaluasi yang digunakan. Pada *cross-validation* 10-fold, *hybrid ensemble* mencapai RMSE rata-rata $0,9307 \pm 0,0043$ dan MAE rata-rata $0,7338 \pm 0,0034$, sedangkan pada pengujian *final* terhadap *hold-out test set* diperoleh RMSE 0,9365 dan MAE 0,7401. Keunggulan ini dikonfirmasi secara statistik melalui *paired t-test* yang menolak H_0 pada seluruh enam pasang perbandingan dengan *p-value* jauh di bawah 0,05, mengindikasikan bahwa perbedaan performa merupakan pola yang konsisten dan dapat direplikasi.

Nilai $\beta = 0,01$ yang optimal pada VAE mengindikasikan bahwa tekanan regularisasi KL *divergence* yang minimal memberikan hasil terbaik pada dataset ini. Temuan ini sejalan dengan fenomena *posterior collapse* yang didokumentasikan pada model VAE berbasis *collaborative filtering* (Wang et al., 2025), di mana representasi laten yang dipelajari mengabaikan data masukan dan bergantung sepenuhnya pada distribusi *prior*, sehingga model gagal menangkap pola data yang kompleks. Penerapan KL *annealing* sebagai mitigasi membantu model membangun representasi laten yang bermakna sebelum regularisasi KL mulai aktif, sehingga *posterior collapse* dapat dihindari.

Jika dibandingkan dengan hasil penelitian terdahulu pada dataset *MovieLens* 100K, RMSE *hybrid ensemble* sebesar 0,9365 berada pada posisi yang kompetitif. (Sami et al., 2024) melaporkan bahwa beberapa metode *baseline collaborative filtering* pada dataset yang sama menghasilkan RMSE berkisar antara 0,912 hingga 0,943, mencakup CF berbasis temporal (0,912), SVD *standalone* (0,943), dan *matrix factorization* (0,939). RSVD *standalone* dalam penelitian ini menghasilkan RMSE 0,9506 yang berada pada rentang yang sebanding dengan metode-metode tersebut, sementara *hybrid ensemble* berhasil melampaui seluruh *baseline* tersebut. Perlu dicatat bahwa perbandingan ini bersifat indikatif dan tidak sepenuhnya *apple-to-apple* karena perbedaan skema *split* dan detail implementasi antar penelitian.

Terdapat beberapa keterbatasan dalam penelitian ini yang perlu diakui. Pertama, proses *hyperparameter tuning* dilakukan pada *fixed train-validation split* sebelum *cross-validation*, sehingga terdapat kemungkinan *mild optimistic bias* di mana *hyperparameter* yang dipilih secara tidak langsung dioptimalkan untuk

karakteristik partisi tertentu. Meski demikian, pendekatan ini lebih defensibel dibandingkan alternatif *tuning per fold* yang berisiko *overfitting* terhadap distribusi *fold* spesifik (Wainer & Cawley, 2021), dan stabilitas performa lintas *fold* yang teramati mengindikasikan bahwa bias ini minimal. Kedua, sistem yang dikembangkan bersifat murni *collaborative filtering* dan tidak memanfaatkan informasi konten film seperti genre atau metadata, sehingga rentan terhadap masalah *cold start* untuk pengguna atau film baru yang tidak memiliki riwayat interaksi. Ketiga, evaluasi dilakukan pada dataset *MovieLens* 100K yang merupakan dataset berskala relatif kecil, sehingga generalisasi hasil ke dataset berskala lebih besar perlu diverifikasi lebih lanjut.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan penelitian yang dilakukan pada implementasi hybrid ensemble yang menggabungkan Variational Autoencoder (VAE) dengan KL annealing dan Regularized Singular Value Decomposition (RSVD) pada sistem rekomendasi film menggunakan dataset MovieLens 100K, dapat disimpulkan sebagai berikut.

- a. Sistem rekomendasi yang dikembangkan menghasilkan performa yang baik dan unggul dibandingkan kedua model individual pada seluruh skema evaluasi yang digunakan. Pada cross-validation 10-fold, hybrid ensemble mencapai RMSE rata-rata sebesar $0,9307 \pm 0,0043$ dan MAE rata-rata sebesar $0,7338 \pm 0,0034$, mengungguli RSVD (RMSE $0,9474 \pm 0,0040$) dan VAE (RMSE $1,0926 \pm 0,0080$). Pada pengujian final terhadap hold-out test set, hybrid ensemble memperoleh RMSE sebesar 0,9365 dan MAE sebesar 0,7401, mengungguli RSVD (RMSE 0,9506) dan VAE (RMSE 1,0876). Seluruh perbedaan performa dikonfirmasi signifikan secara statistik melalui paired t-test pada tingkat signifikansi $\alpha = 0,05$, di mana keenam pasang perbandingan menghasilkan p-value yang jauh di bawah ambang batas, sehingga keunggulan hybrid ensemble dapat dinyatakan nyata dan bukan terjadi secara kebetulan.
- b. Model yang optimal untuk sistem rekomendasi dengan VAE dan RSVD adalah hybrid ensemble dengan bobot $\alpha = 0,25$ untuk VAE dan $\beta = 0,75$ untuk RSVD. Bobot ini diperoleh melalui grid search pada validation set dan terbukti konsisten lintas fold pada cross-validation (rata-rata α $0,242 \pm 0,009$), mengindikasikan bahwa proporsi kontribusi optimal kedua model bersifat stabil dan tidak bergantung pada partisi data tertentu. Dominansi kontribusi RSVD sebesar 75% mencerminkan bahwa struktur bias dan faktor laten linear lebih informatif pada dataset MovieLens 100K yang memiliki sparsity tinggi, sementara kontribusi VAE sebesar 25% melalui representasi probabilistik di ruang laten tetap memberikan peningkatan performa yang signifikan secara statistik saat digabungkan.

5.2 Saran

Untuk pengembangan sistem lebih lanjut agar menjadi lebih baik, beberapa saran yang dapat diberikan untuk penelitian berikutnya yaitu mengintegrasikan informasi konten film seperti genre dan metadata ke dalam sistem untuk mengatasi keterbatasan *cold start* yang melekat pada pendekatan *collaborative filtering* murni. Selain itu, pengujian pada dataset yang lebih besar seperti *MovieLens* 1M atau 10M perlu dilakukan untuk memverifikasi apakah keunggulan arsitektur *hybrid* VAE-RSVD dapat dipertahankan pada skala yang lebih besar. Penelitian selanjutnya juga dapat mengeksplorasi variasi arsitektur VAE seperti penambahan mekanisme *attention* untuk menangkap hubungan struktural yang lebih kaya antara pengguna dan item, serta mengeksplorasi pendekatan *end-to-end* yang mengoptimalkan *hyperparameter* kedua model dan bobot *ensemble* secara simultan.

DAFTAR PUSTAKA

- Abedin, T., Xu, H., & Uddin, S. (2026). The impact of K selection in K-fold cross-validation on bias and variance in supervised learning models. *Scientific Reports* 2026 16:1, 16(1), 6084-. <https://doi.org/10.1038/s41598-026-37247-x>
- Alshbanat, H. I., Benhidour, H., & Kerrache, S. (2025). A survey of latent factor models in recommender systems. *Information Fusion*, 117(2), 102905. <https://doi.org/10.1016/j.inffus.2024.102905>
- Ayu, D., Safitri, N., Helilintar, R., & Wahyuniar, L. S. (2021). Sistem Rekomendasi Skincare Menggunakan Metode Content-Based Filtering dan Algoritma Apriori. *Prosiding SEMNAS INOTEK (Seminar Nasional Inovasi Teknologi)*, 5(2), 242–248. <https://doi.org/10.29407/INOTEK.V5I2.1136>
- Bobadilla, J., Ortega, F., Gutiérrez, A., & González-Prieto, Á. (2021). Deep Variational Models for Collaborative Filtering-based Recommender Systems. *Neural Computing and Applications*, 35(10), 7817–7831. <http://arxiv.org/abs/2107.12677>
- Bougteb, Y., Ouhbi, B., Frikh, B., & Zemmouri, E. (2022). A Deep Autoencoder-Based Hybrid Recommender System. *Https://Services.Igi-Global.Com/Resolvedoi/Resolve.aspx?Doi=10.4018/IJMCMC.297963*, 13(1), 1–19. <https://doi.org/10.4018/IJMCMC.297963>
- Hancock, D., Slee, J., & Wunsch-Vincent, S. (2024, March 28). *Resurgence of Global Cinema: 2022 and 2023 witness forceful comeback but still shy of pre-pandemic norms*. https://www.wipo.int/global_innovation_index/en/gii-insights-blog/2024/global-cinema.html
- Liang, S., Pan, Z., Liu, W., Yin, J., & De Rijke, M. (2024). A Survey on Variational Autoencoders in Recommender Systems. *ACM Computing Surveys*, 56(10). <https://doi.org/10.1145/3663364>
- Mu'afa, M. N., & Baizal, Z. K. A. (2022). Implementation of Dimensionality Reduction with SVD to Improve Rating Prediction in Recommender System. *Journal of Computer System and Informatics (JoSYC)*, 3(4), 544–551.

<https://doi.org/10.47065/JOSYC.V3I4.2110>

- Nugroho, F., & Rahayu, M. I. (2020). SISTEM REKOMENDASI PRODUK UKM DI KOTA BANDUNG MENGGUNAKAN ALGORITMA COLLABORATIVE FILTERING. *Jurnal Riset Sistem Informasi Dan Teknologi Informasi (JURSISTEKNI)*, 2(3), 23–31. <https://doi.org/10.52005/JURSISTEKNI.V2I3.63>
- Putra, I. M. A. D., & Santiyasa, I. W. (2025). Implementation of Regularized Singular Value Decomposition in Collaborative Filtering Book Recommendation System. *Jurnal Teknik Informatika Dan Sistem Informasi*, 11(2), 213-225–213 – 225. <https://doi.org/10.28932/jutisi.v11i2.10186>
- Rahmani, D. A., Risnawati, & Hamdani, M. F. (2025). Uji T-Student Dua Sampel Saling Berpasangan/Dependend (Paired Sample t-Test). *Jurnal Penelitian Ilmu Pendidikan Indonesia*, 4(2), 568–576. <https://jpion.org/index.php/jpi568>Situswebjurnal:<https://jpion.org/index.php/jpi>
- Rajput, I. S., Tewari, A. S., & Tiwari, A. K. (2024). An autoencoder-based deep learning model for solving the sparsity issues of Multi-Criteria Recommender System. *Procedia Computer Science*, 235, 414–425. <https://doi.org/10.1016/J.PROCS.2024.04.041>
- Sami, A., Adrousy, W. El, Sarhan, S., & Elmougy, S. (2024). A deep learning based hybrid recommendation model for internet users. *Scientific Reports 2024 14:1*, 14(1), 29390-. <https://doi.org/10.1038/s41598-024-79011-z>
- Sharma, S., & Kumar Shakya, H. (2024). Hybrid recommendation system for movies using artificial neural network. *Expert Systems with Applications*, 258, 125194. <https://doi.org/10.1016/J.ESWA.2024.125194>
- Sutjiningtyas, S., Arofa Dharmawan, A., & Penulis Korespondensi, E. (2022). Rancang Bangun Sistem Rekomendasi Produk Sepatu pada Toko Online Menggunakan Metode User-Base Collaborative Filtering. *Bulletin of Information Technology (BIT)*, 3(2), 143–148. <https://doi.org/10.47065/BIT.V3I2.288>
- Wainer, J., & Cawley, G. (2021). Nested cross-validation when selecting classifiers

- is overzealous for most practical applications. *Expert Systems with Applications*, 182(1), 115222. <https://doi.org/10.1016/j.eswa.2021.115222>
- Wang, F., Chen, C., Liu, W., Lei, M., Chen, J., Liu, Y., Zheng, X., & Yin, J. (2025). DR-VAE: Debiased and Representation-enhanced Variational Autoencoder for Collaborative Recommendation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(12), 12703–12711. <https://doi.org/10.1609/aaai.v39i12.33385>
- Wibisono, C., Surya, L., #2, H., & Widyaya, J. E. (2021). Sistem Rekomendasi Suku Cadang Berdasarkan Item Based Filtering. *Jurnal Teknik Informatika Dan Sistem Informasi*, 7(1), 2443–2229. <https://doi.org/10.28932/JUTISI.V7I1.3036>

LAMPIRAN

Lampiran 1. Program DataPreprocessing.ipynb

```
# 1. Import Libraries
import pandas as pd
import numpy as np
import os

# 2. File Paths
raw_data_path = '../Data/ml-100k/u.data'
output_dir = '../Data/TrainTest'

print(f'[INFO] Raw data path : {raw_data_path}')
print(f'[INFO] Output dir : {output_dir}')

# 3. Load Data
column_names = ['user_id', 'item_id', 'rating', 'timestamp']
df = pd.read_csv(raw_data_path, sep='\t', names=column_names)

print(f'[INFO] Total baris : {len(df):,}')
print(f'[INFO] Kolom : {list(df.columns)}')
df.head()

# 4. Preprocess Data
## 4.1. Matriks Interaksi User-Item
n_users = df['user_id'].nunique()
n_items = df['item_id'].nunique()

print(f'[INFO] Pengguna unik : {n_users}')
print(f'[INFO] Film unik : {n_items}')
```

```

print(f'[INFO] Total rating : {len(df):,}')

# Pivot ke matriks user x item; sel tanpa rating diisi 0
interaction_matrix = df.pivot(
    index='user_id',
    columns='item_id',
    values='rating'
).fillna(0)

print(f'[INFO] Dimensi matriks: {interaction_matrix.shape}')
interaction_matrix.head()

## 4.2. Normalisasi Rating
MAX_RATING = 5.0

# Bagi dengan rating maksimum sehingga skala menjadi [0, 1]
normalized_matrix = interaction_matrix.values / MAX_RATING

print(f'[INFO] Min (non-zero) : {normalized_matrix[normalized_matrix > 0].min():.2f}')
print(f'[INFO] Max : {normalized_matrix.max():.2f}')
print(f'[INFO] Dimensi : {normalized_matrix.shape}')

## 4.3. Train / Validation / Test Split (Hold-Out)
# Proporsi split: 70% train | 10% validasi | 20% test
VAL_RATIO = 0.10
TEST_RATIO = 0.20
RANDOM_SEED = 42

all_rows, all_cols = np.nonzero(normalized_matrix)

```

```

total_ratings = len(all_rows)

n_test = int(total_ratings * TEST_RATIO)
n_val = int(total_ratings * VAL_RATIO)
n_train = total_ratings - n_test - n_val

print(f'[INFO] Total : {total_ratings:,}')
print(f'[INFO] Train : {n_train:,} ({n_train/total_ratings*100:.0f}%)')
print(f'[INFO] Val : {n_val:,} ({n_val/total_ratings*100:.0f}%)')
print(f'[INFO] Test : {n_test:,} ({n_test/total_ratings*100:.0f}%)')

np.random.seed(RANDOM_SEED)
shuffled_idx = np.random.permutation(total_ratings)

test_idx = shuffled_idx[:n_test]
val_idx = shuffled_idx[n_test:n_test + n_val]
train_idx = shuffled_idx[n_test + n_val:]

test_rows, test_cols = all_rows[test_idx], all_cols[test_idx]
val_rows, val_cols = all_rows[val_idx], all_cols[val_idx]
train_rows, train_cols = all_rows[train_idx], all_cols[train_idx]

# Bangun matriks train: hapus rating val dan test dari matriks penuh
train_data = normalized_matrix.copy().astype(np.float32)
train_data[val_rows, val_cols] = 0.0
train_data[test_rows, test_cols] = 0.0

# Matriks val dan test: mulai dari nol, isi hanya rating masing-masing partisi
val_data = np.zeros(normalized_matrix.shape, dtype=np.float32)
val_data[val_rows, val_cols] = normalized_matrix[val_rows, val_cols]

```

```

test_data = np.zeros(normalized_matrix.shape, dtype=np.float32)
test_data[test_rows, test_cols] = normalized_matrix[test_rows, test_cols]

n_train_check = int(np.sum(train_data > 0))
n_val_check = int(np.sum(val_data > 0))
n_test_check = int(np.sum(test_data > 0))

print(f'[VERIFIKASI] Train : {n_train_check:,}')
print(f'[VERIFIKASI] Val : {n_val_check:,}')
print(f'[VERIFIKASI] Test : {n_test_check:,}')
print(f'[VERIFIKASI] Total : {n_train_check + n_val_check + n_test_check:,}
(harus {total_ratings:,})')

# 5. Save Data
np.save(os.path.join(output_dir, 'train_data'), train_data)
np.save(os.path.join(output_dir, 'val_data'), val_data)
np.save(os.path.join(output_dir, 'test_data'), test_data)
# Simpan juga matriks penuh untuk keperluan cross validation
np.save(os.path.join(output_dir, 'normalized_matrix'), normalized_matrix)

print(f'[SUCCESS] train_data.npy disimpan ({n_train_check:,} rating)')
print(f'[SUCCESS] val_data.npy disimpan ({n_val_check:,} rating)')
print(f'[SUCCESS] test_data.npy disimpan ({n_test_check:,} rating)')
print(f'[SUCCESS] normalized_matrix.npy disimpan ({total_ratings:,} rating)')

```

Lampiran 2. Program HyperParameterTuningOptuna.ipynb

```

# 1. Import Libraries
import numpy as np
import tensorflow as tf
import itertools

```

```
import sys
import os
import gc
import json
import optuna

# 2. Data & Model Paths
## 2.1. Folders
scripts_folder = os.path.abspath(os.path.join('..', 'Scripts'))
data_folder = os.path.abspath(os.path.join('..', 'Data', 'TrainTest'))
hyper_parameter_folder = os.path.abspath(os.path.join('..', 'Data', 'HyperParameters'))

## 2.2. System Paths
sys.path.append(scripts_folder)

# 3. Import Data & Models
## 3.1. Import Models
from model import Encoder, Decoder, VAE, RSVD

## 3.2. Import Data
train_data_path = os.path.join(data_folder, 'train_data.npy')

try:
    train_data = np.load(train_data_path).astype(np.float32)
    train_data_tf = tf.constant(train_data, dtype=tf.float32)
    num_items = train_data.shape[1]
    print(f'Data latih berhasil dimuat. Dimensi: {train_data.shape}')
except FileNotFoundError:
    print("Error: 'train_data.npy' tidak ditemukan. Jalankan preprocessing terlebih dahulu.")
```


4. Hyperparameter Tuning

4.1. VAE

4.1.1. Search Space

```
vae_search_space = {
    'latent_dim' : [20, 50, 100, 200],
    'learning_rate': [1e-4, 5e-4, 1e-3, 5e-3, 1e-2],
    'batch_size' : [64, 128, 256],
    'dropout_rate' : [0.1, 0.2, 0.3, 0.4],
    'hidden_dims' : [
        [512, 256],
        [512, 256, 128],
        [1024, 512],
        [1024, 512, 256],
    ],
    'beta': [0.01, 0.05, 0.1, 0.3, 0.5, 0.8, 1.0],
}

N_TRIALS_VAE = 200

print("[INFO] Search space VAE:")
for key, values in vae_search_space.items():
    print(f"    {key:<15}: {values}")
print(f"\n[INFO] Jumlah trial: {N_TRIALS_VAE}")

### 4.1.2. Optuna Objective Function
optuna.logging.set_verbosity(optuna.logging.WARNING)

def vae_objective(trial):
    """
```

Fungsi objektif Optuna untuk tuning VAE.

Mengembalikan reconstruction_loss sebagai target minimisasi agar hasil tidak bias terhadap nilai beta yang kecil.

```
"""
```

```
latent_dim = trial.suggest_categorical('latent_dim',
vae_search_space['latent_dim'])
learning_rate = trial.suggest_categorical('learning_rate',
vae_search_space['learning_rate'])
batch_size = trial.suggest_categorical('batch_size',
vae_search_space['batch_size'])
dropout_rate = trial.suggest_categorical('dropout_rate',
vae_search_space['dropout_rate'])
beta = trial.suggest_categorical('beta', vae_search_space['beta'])

# suggest_categorical hanya menerima tipe primitif; konversi list ke string
hidden_dims_str = trial.suggest_categorical('hidden_dims', [str(h) for h in
vae_search_space['hidden_dims']])
hidden_dims = eval(hidden_dims_str)

tf.keras.backend.clear_session()
tf.compat.v1.reset_default_graph()

encoder = Encoder(hidden_dims=hidden_dims, latent_dim=latent_dim,
dropout_rate=dropout_rate)
decoder = Decoder(hidden_dims=hidden_dims[::-1],
output_dim=num_items)
vae = VAE(encoder, decoder, beta=beta)

dummy_output = vae(train_data_tf[:1])
optimizer = tf.keras.optimizers.Adam(learning_rate=learning_rate)
vae.compile(optimizer=optimizer, run_eagerly=True)
```

```

history = vae.fit(
    train_data_tf, train_data_tf,
    epochs=30, batch_size=batch_size, verbose=0
)

reconstruction_loss = history.history['reconstruction_loss'][-1]

del encoder, decoder, vae, history, optimizer, dummy_output
gc.collect()
tf.keras.backend.clear_session()

return reconstruction_loss

### 4.1.3. Optuna Study (Resume-able)
best_vae_path = os.path.join(hyper_parameter_folder,
                              'best_vae_weights.weights.h5')
vae_progress_file = os.path.join(hyper_parameter_folder,
                                  'tuning_progress_vae.json')
vae_study_db = os.path.join(hyper_parameter_folder, 'optuna_vae_study.db')

# Storage SQLite memungkinkan study dilanjutkan jika kernel terputus
study_vae = optuna.create_study(
    study_name='vae_tuning',
    direction='minimize',
    storage=f'sqlite:/// {vae_study_db}',
    load_if_exists=True
)

n_completed = len([t for t in study_vae.trials if t.state ==
optuna.trial.TrialState.COMPLETE])

```

```

n_remaining = max(0, N_TRIALS_VAE - n_completed)

print(f'[INFO] Trial selesai : {n_completed}')
print(f'[INFO] Trial tersisa : {n_remaining}')

if n_completed > 0:
    print(f'[INFO] Best loss sejauh ini : {study_vae.best_value:.6f}')
    print(f'[INFO] Best params      : {study_vae.best_params}')

### 4.1.4. Run Tuning
def print_trial_callback(study, trial):
    print(f' Trial {trial.number+1:>3} | Recon Loss: {trial.value:.6f} | Params: {trial.params}')
    if study.best_trial.number == trial.number:
        print(f'      [NEW BEST] -> {trial.value:.6f}')

if n_remaining > 0:
    print(f'[INFO] Memulai {n_remaining} trial Optuna (TPE)...\n')
    study_vae.optimize(
        vae_objective,
        n_trials=n_remaining,
        callbacks=[print_trial_callback],
        gc_after_trial=True
    )
else:
    print("[INFO] Semua trial sudah selesai.")

print(f'\n[SELESAI] Best Loss  : {study_vae.best_value:.6f}')
print(f'[SELESAI] Best Params : {study_vae.best_params}')

### 4.1.5. Simpan Hasil Terbaik

```

```

best_params_raw = study_vae.best_params.copy()
best_params_raw['hidden_dims'] = eval(best_params_raw['hidden_dims'])

with open(vae_progress_file, 'w') as f:
    json.dump({'best_loss': study_vae.best_value, 'best_params':
best_params_raw}, f, indent=4)

print(f'[SUCCESS] Parameter terbaik disimpan ke: {vae_progress_file}')
print(f'[INFO] Isi: {best_params_raw}')

## 4.2. RSVD
### 4.2.1. Hyperparameter Search Space
rsvd_param_grid = {
    'n_factors' : [10, 20, 50],
    'learning_rate': [0.001, 0.005, 0.01, 0.02],
    'lambda_reg' : [0.001, 0.005, 0.01, 0.02],
}

rsvd_keys, rsvd_values = zip(*rsvd_param_grid.items())
rsvd_combinations = [dict(zip(rsvd_keys, v)) for v in
itertools.product(*rsvd_values)]

best_rsvd_error = float('inf')
best_rsvd_params = None
best_rsvd_model = None

### 4.2.2. Grid Search
#### 4.2.2.1. Load Progress
rsvd_progress_file = os.path.join(hyper_parameter_folder,
'tuning_progress_rsvd.json')

```

```

if os.path.exists(rsvd_progress_file):
    with open(rsvd_progress_file, 'r') as f:
        progress_data = json.load(f)

    best_rsvd_error = progress_data['best_error']
    best_rsvd_params = progress_data['best_params']
    completed_indices = progress_data['completed_indices']

    print(f'[INFO] Melanjutkan sesi sebelumnya...")
    print(f'[INFO] {len(completed_indices)} kombinasi sudah dievaluasi.")
    print(f'[INFO] MSE terbaik saat ini: {best_rsvd_error:.4f}")
else:
    best_rsvd_error = float('inf')
    best_rsvd_params = None
    completed_indices = []

#### 4.2.2.2. Looping
for i, params in enumerate(rsvd_combinations):
    if i in completed_indices:
        continue

    print(f"\n[{i+1}/{len(rsvd_combinations)}] Menguji RSVD | Params:
    {params}")

    rsvd = RSVD(
        n_factors=params['n_factors'],
        learning_rate=params['learning_rate'],
        lambda_reg=params['lambda_reg'],
        epochs=15
    )
    rsvd.fit(train_data)

```

```

total_loss_error = rsvd.loss_history[-1]
print(f" -> MSE : {total_loss_error:.4f}")

if total_loss_error < best_rsvd_error:
    print(f" [NEW BEST] MSE turun ke {total_loss_error:.4f}. Menyimpan
model...")
    best_rsvd_error = total_loss_error
    best_rsvd_params = params
    best_rsvd_model = rsvd

    # Simpan komponen RSVD terbaik
    np.save(os.path.join(hyper_parameter_folder, 'best_mu.npy'),
np.array(rsvd.mu))
    np.save(os.path.join(hyper_parameter_folder, 'best_b_u.npy'), rsvd.b_u)
    np.save(os.path.join(hyper_parameter_folder, 'best_b_i.npy'), rsvd.b_i)
    np.save(os.path.join(hyper_parameter_folder, 'best_U.npy'), rsvd.U)
    np.save(os.path.join(hyper_parameter_folder, 'best_Sigma.npy'),
rsvd.Sigma)
    np.save(os.path.join(hyper_parameter_folder, 'best_V.npy'), rsvd.V)

    completed_indices.append(i)
    with open(rsvd_progress_file, 'w') as f:
        json.dump({
            'best_error' : best_rsvd_error,
            'best_params' : best_rsvd_params,
            'completed_indices': completed_indices
        }, f)

del rsvd
gc.collect()

```

```

print("\n=====")
)
print(f"[HASIL TERBAIK RSVD]")
print(f"Parameter Terbaik : {best_rsvd_params}")
print(f"MSE Terendah      : {best_rsvd_error:.4f}")
print("=====")

```

Lampiran 3. Program Training.ipynb

```

# 1. Import Libraries
import os
import sys
import json
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from datetime import datetime

# 2. File Paths
scripts_path = os.path.abspath(os.path.join('..', 'Scripts'))
if scripts_path not in sys.path:
    sys.path.append(scripts_path)

data_dir      = os.path.abspath(os.path.join('..', 'Data', 'TrainTest'))
hyperparameter_dir      = os.path.abspath(os.path.join('..', 'Data',
'HyperParameters'))
save_dir      = os.path.abspath(os.path.join('..', 'Data', 'SavedModels'))
loss_history_dir = os.path.abspath(os.path.join('..', 'Data', 'LossHistory'))
visualization_dir = os.path.abspath(os.path.join('..', 'Data', 'Visualization'))

```



```

# 3. Load Models & Data
## 3.1. Data
train_data = np.load(os.path.join(data_dir, 'train_data.npy')).astype(np.float32)
num_items = train_data.shape[1]

## 3.2. Models
from model import Encoder, Decoder, VAE, RSVD, KLANnealingCallback

# 4. Training
## 4.1. VAE
### 4.1.1. Import Hyperparameters
vae_progress_file = os.path.join(hyperparameter_dir,
'tuning_progress_vae.json')
with open(vae_progress_file, 'r') as f:
    best_vae_params = json.load(f)['best_params']

print(f'[INFO] Melatih VAE dengan parameter: {best_vae_params}')

### 4.1.2. Construct Model
encoder = Encoder(
    hidden_dims=best_vae_params['hidden_dims'],
    latent_dim=best_vae_params['latent_dim'],
    dropout_rate=best_vae_params['dropout_rate']
)
decoder = Decoder(hidden_dims=best_vae_params['hidden_dims'][:-1],
output_dim=num_items)

# Inisialisasi dengan beta=0.0 agar KL annealing dimulai dari nol
final_vae = VAE(encoder, decoder, beta=0.0)

_ = final_vae(train_data[:1])

```

```

optimizer =
tf.keras.optimizers.Adam(learning_rate=best_vae_params['learning_rate'])
final_vae.compile(optimizer=optimizer)

# Naikkan beta secara linear dari 0 ke beta_target selama 30 epoch pertama
KL_ANNEALING_EPOCHS = 30

kl_annealing = KLANnealingCallback(
    beta_target=best_vae_params['beta'],
    annealing_epochs=KL_ANNEALING_EPOCHS
)

early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor='loss', patience=10, restore_best_weights=True, verbose=1
)

### 4.1.3. Training Model
history = final_vae.fit(
    train_data, train_data,
    epochs=500,
    batch_size=best_vae_params['batch_size'],
    callbacks=[kl_annealing, early_stopping],
    verbose=1
)

### 4.1.4. Save Weights
final_vae.save_weights(os.path.join(save_dir,
'trained_best_vae_weights.weights.h5'))
print("[SUCCESS] Bobot VAE berhasil disimpan.")

```

```

#### 4.1.5. Save Loss History
vae_history_dict = {'loss': history.history['loss']}
with open(os.path.join(loss_history_dir, 'final_vae_loss_history.json'), 'w') as f:
    json.dump(vae_history_dict, f)

## 4.2. RSVD
#### 4.2.1. Import Hyperparameters
rsvd_progress_file = os.path.join(hyperparameter_dir,
'tuning_progress_rsvd.json')
with open(rsvd_progress_file, 'r') as f:
    best_rsvd_params = json.load(f)['best_params']

print(f"[INFO] Melatih RSVD dengan parameter: {best_rsvd_params}")

#### 4.2.2. Construct Model
final_rsvd = RSVD(
    n_factors=best_rsvd_params['n_factors'],
    learning_rate=best_rsvd_params['learning_rate'],
    lambda_reg=best_rsvd_params['lambda_reg'],
    epochs=100
)

#### 4.2.3. Training Model
final_rsvd.fit(train_data)

final_total_loss = final_rsvd.loss_history[-1]
print(f"Training selesai. Final loss: {final_total_loss:.4f}")

#### 4.2.4. Save Weights
np.save(os.path.join(save_dir, 'final_mu.npy'), np.array(final_rsvd.mu))
np.save(os.path.join(save_dir, 'final_b_u.npy'), final_rsvd.b_u)

```

```

np.save(os.path.join(save_dir, 'final_b_i.npy'), final_rsvd.b_i)
np.save(os.path.join(save_dir, 'best_U.npy'), final_rsvd.U)
np.save(os.path.join(save_dir, 'best_Sigma.npy'), final_rsvd.Sigma)
np.save(os.path.join(save_dir, 'best_V.npy'), final_rsvd.V)
print("[SUCCESS] Komponen RSVD berhasil disimpan.")

### 4.2.5. Save Loss History
np.save(os.path.join(loss_history_dir, 'final_rsvd_loss_history.npy'),
final_rsvd.loss_history)

# 5. Training Visualization
## 5.1. Import Loss History
### 5.1.1. VAE
try:
    with open(os.path.join(loss_history_dir, 'final_vae_loss_history.json'), 'r') as f:
        vae_loss_history = json.load(f)['loss']
except FileNotFoundError as e:
    print(f"[ERROR] {e}")

### 5.1.2. RSVD
rsvd_loss_history = np.load(os.path.join(loss_history_dir,
'final_rsvd_loss_history.npy'))

## 5.2. Visualization
fig, axes = plt.subplots(1, 2, figsize=(16, 6), dpi=100)

# --- Plot 1: Kurva Loss VAE ---
ax1 = axes[0]
beta_val = best_vae_params.get('beta', ' $\beta$ ')
ax1.plot(vae_loss_history, label=f'Total Loss (MSE + {beta_val} KL)',
color='#2ca02c', linewidth=2.5)

```

```

vae_final_epoch = len(vae_loss_history) - 1
vae_final_loss = vae_loss_history[-1]
ax1.scatter(vae_final_epoch, vae_final_loss, color='red', s=50, zorder=5)
ax1.text(vae_final_epoch, vae_final_loss + (vae_final_loss * 0.02),
        f'{vae_final_loss:.4f}', ha='right', va='bottom', fontsize=12,
        fontweight='bold', color='red')

ax1.set_title('VAE Final Training Convergence', fontsize=15, fontweight='bold',
pad=15)
ax1.set_xlabel('Epochs', fontsize=13)
ax1.set_ylabel('Loss Value', fontsize=13)
ax1.grid(True, linestyle='--', alpha=0.6)
ax1.legend(loc='upper right', fontsize=11)

# --- Plot 2: Kurva Loss RSVD ---
ax2 = axes[1]
ax2.plot(rsvd_loss_history, label='Total Loss (MSE + L2 Penalty)',
color='#1f77b4', linewidth=2.5)

rsvd_final_epoch = len(rsvd_loss_history) - 1
rsvd_final_loss = rsvd_loss_history[-1]
ax2.scatter(rsvd_final_epoch, rsvd_final_loss, color='red', s=50, zorder=5)
ax2.text(rsvd_final_epoch, rsvd_final_loss + (rsvd_final_loss * 0.02),
        f'{rsvd_final_loss:.6f}', ha='right', va='bottom', fontsize=12,
        fontweight='bold', color='red')

ax2.set_title('RSVD Final Training Convergence', fontsize=15,
fontweight='bold', pad=15)
ax2.set_xlabel('Epochs', fontsize=13)
ax2.set_ylabel('Total Loss (MSE + L2 Penalty)', fontsize=13)

```

```

ax2.grid(True, linestyle='--', alpha=0.6)
ax2.legend(loc='upper right', fontsize=11)

plt.tight_layout()

timestamp = datetime.now().strftime('%Y%m%d_%H%M')
plot_path = os.path.join(visualization_dir,
f'final_training_curves_{timestamp}.png')
plt.savefig(plot_path, dpi=300, bbox_inches='tight')
print(f'[SUCCESS] Grafik disimpan di: {plot_path}')
plt.show()

```

Lampiran 4. Program Testing.ipynb

```

# 1. Import Libraries
import os
import sys
import json
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from datetime import datetime

# 2. File Paths
scripts_path = os.path.abspath(os.path.join('..', 'Scripts'))
if scripts_path not in sys.path:
    sys.path.append(scripts_path)

data_dir = os.path.abspath(os.path.join('..', 'Data', 'TrainTest'))
save_dir = os.path.abspath(os.path.join('..', 'Data', 'SavedModels'))

```

```

hyperparameters_dir = os.path.abspath(os.path.join('..', 'Data',
'HyperParameters'))
visualization_dir = os.path.abspath(os.path.join('..', 'Data', 'Visualization'))
results_dir = os.path.abspath(os.path.join('..', 'Data', 'EvaluationResults'))

# 3. Load Models & Data
## 3.1. Data
MAX_RATING = 5.0

### 3.1.1. Train Data
train_data = np.load(os.path.join(data_dir, "train_data.npy")).astype(np.float32)
num_users, num_items = train_data.shape
print(f"[INFO] Train: {num_users} users x {num_items} items |
{int(np.sum(train_data > 0))}, rating")

### 3.1.2. Validation Data
# Data validasi dipakai untuk mencari alpha ensemble (tidak menyentuh test set)
val_data = np.load(os.path.join(data_dir, "val_data.npy")).astype(np.float32)
val_indices = np.where(val_data > 0)
actual_val_values = val_data[val_indices] * MAX_RATING
print(f"[INFO] Val : {len(actual_val_values)}, rating")

### 3.1.3. Test Data
test_data = np.load(os.path.join(data_dir, "test_data.npy")).astype(np.float32)
test_indices = np.where(test_data > 0)
actual_test_values = test_data[test_indices] * MAX_RATING
print(f"[INFO] Test : {len(actual_test_values)}, rating")

## 3.2. Models
from model import Encoder, Decoder, VAE

```

```

# 4. Construct Model
## 4.1. VAE
vae_progress_file = os.path.join(hyperparameters_dir,
'tuning_progress_vae.json')
with open(vae_progress_file, 'r') as f:
    best_vae_params = json.load(f)['best_params']

encoder = Encoder(
    hidden_dims=best_vae_params['hidden_dims'],
    latent_dim=best_vae_params['latent_dim'],
    dropout_rate=best_vae_params['dropout_rate']
)
decoder = Decoder(hidden_dims=best_vae_params['hidden_dims'][::-1],
output_dim=num_items)
eval_vae = VAE(encoder, decoder, beta=best_vae_params['beta'])

_ = eval_vae(train_data[:1])
eval_vae.load_weights(os.path.join(save_dir,
'trained_best_vae_weights.weights.h5'))

## 4.2. RSVD
mu = np.load(os.path.join(save_dir, 'final_mu.npy')) # rata-rata global
b_u = np.load(os.path.join(save_dir, 'final_b_u.npy')) # bias pengguna
b_i = np.load(os.path.join(save_dir, 'final_b_i.npy')) # bias item
U = np.load(os.path.join(save_dir, 'best_U.npy'))
Sigma = np.load(os.path.join(save_dir, 'best_Sigma.npy'))
V = np.load(os.path.join(save_dir, 'best_V.npy'))

# 5. Prediction
## 5.1. VAE
### 5.1.1. Extract Latent Space

```



```

train_data_tf = tf.constant(train_data, dtype=tf.float32)
Z_mean, Z_log_var = eval_vae.encoder.predict(train_data_tf, verbose=0)

### 5.1.2. Get Prediction
pred_vae_norm = eval_vae.decoder.predict(Z_mean, verbose=0) # skala 0-1

### 5.1.3. Filter, Denormalize, Clip
pred_vae_test_clipped = np.clip(pred_vae_norm[test_indices] *
MAX_RATING, 1.0, MAX_RATING)

## 5.2. RSVD
### 5.2.1. Rekonstruksi Prediksi Penuh
latent_matrix = np.dot(np.dot(U, Sigma), V.T)
bias_matrix = float(mu) + b_u[:, np.newaxis] + b_i[np.newaxis, :]
full_rsvd_pred = bias_matrix + latent_matrix

### 5.2.2. Filter, Denormalize, Clip
pred_rsvd_test_clipped = np.clip(full_rsvd_pred[test_indices] *
MAX_RATING, 1.0, MAX_RATING)

## 5.3. Ensemble
### 5.3.1. Cari Bobot Ensemble Optimal (Alpha)
# Alpha dicari di data validasi agar evaluasi test set tetap tidak bias

pred_vae_val = np.clip(pred_vae_norm[val_indices] * MAX_RATING, 1.0,
MAX_RATING)
pred_rsvd_val = np.clip(full_rsvd_pred[val_indices] * MAX_RATING, 1.0,
MAX_RATING)

best_val_rmse = float("inf")
best_alpha = 0.0

```

```

best_beta = 0.0

alpha_history = []
rmse_history = []

# Grid search alpha dari 0.00 sampai 1.00 dengan langkah 0.01
for alpha in np.linspace(0, 1, 101):
    beta = 1.0 - alpha
    pred_ensemble = np.clip((alpha * pred_vae_val) + (beta * pred_rsvd_val), 1.0,
MAX_RATING)
    rmse = np.sqrt(np.mean(np.square(actual_val_values - pred_ensemble)))

    alpha_history.append(alpha)
    rmse_history.append(rmse)

    if rmse < best_val_rmse:
        best_val_rmse = rmse
        best_alpha = alpha
        best_beta = beta

print(f"[INFO] Alpha (VAE) : {best_alpha:.2f}")
print(f"[INFO] Beta (RSVD) : {best_beta:.2f}")
print(f"[INFO] Val RMSE : {best_val_rmse:.4f}")

plt.figure(figsize=(10, 5))
plt.plot(alpha_history, rmse_history, color="blue", linewidth=2)
plt.axvline(x=best_alpha, color="red", linestyle="--", label=f"Best Alpha:
{best_alpha:.2f}")
plt.scatter([best_alpha], [best_val_rmse], color="red", s=100, zorder=5)
plt.title("Kurva Tuning Bobot Ensemble (VAE vs RSVD) — Validation Set",
fontsize=14, pad=15)

```

```

plt.xlabel("Proporsi Bobot VAE (Alpha)", fontsize=12)
plt.ylabel("RMSE pada Data Validasi", fontsize=12)
plt.grid(True, linestyle="--", alpha=0.7)
plt.legend()
plt.tight_layout()

timestamp = datetime.now().strftime("%Y%m%d_%H%M")
plot_path = os.path.join(visualization_dir,
f'ensemble_weight_tuning_{timestamp}.png')
plt.savefig(plot_path, dpi=300)
plt.show()

### 5.3.2. Terapkan Alpha ke Test Set
pred_vae_test_clipped = np.clip(pred_vae_norm[test_indices] *
MAX_RATING, 1.0, MAX_RATING)
pred_rsvd_test_clipped = np.clip(full_rsvd_pred[test_indices] *
MAX_RATING, 1.0, MAX_RATING)

pred_hybrid = (best_alpha * pred_vae_test_clipped) + (best_beta *
pred_rsvd_test_clipped)
pred_hybrid_clipped = np.clip(pred_hybrid, 1.0, MAX_RATING)

# 6. Evaluation
## 6.1. Helper Function
def calculate_metrics(y_true, y_pred):
    """Menghitung MSE, RMSE, dan MAE sekaligus."""
    mse = np.mean(np.square(y_true - y_pred))
    rmse = np.sqrt(mse)
    mae = np.mean(np.abs(y_true - y_pred))
    return mse, rmse, mae

```

6.2. Calculate Metrics

```
mse_v,    rmse_v,    mae_v    =    calculate_metrics(actual_test_values,  
pred_vae_test_clipped)
```

```
mse_r,    rmse_r,    mae_r    =    calculate_metrics(actual_test_values,  
pred_rsvd_test_clipped)
```

```
mse_h,    rmse_h,    mae_h    =    calculate_metrics(actual_test_values,  
pred_hybrid_clipped)
```

6.3. Comparison

```
print(f'=====
```

```
print(f' HASIL EVALUASI FINAL - TEST SET")
```

```
print(f'=====
```

```
print(f' {'Metrik':<8} | {'VAE':^12} | {'RSVD':^12} | {'Hybrid':^12}")
```

```
print(f' {'-'*8}-+-{'-'*12}-+-{'-'*12}-+-{'-'*12}")
```

```
print(f' {'MSE':<8} | {mse_v:^12.4f} | {mse_r:^12.4f} | {mse_h:^12.4f}")
```

```
print(f' {'RMSE':<8} | {rmse_v:^12.4f} | {rmse_r:^12.4f} | {rmse_h:^12.4f}")
```

```
print(f' {'MAE':<8} | {mae_v:^12.4f} | {mae_r:^12.4f} | {mae_h:^12.4f}")
```

```
print(f'=====
```

```
print(f'      Alpha    optimal    (dari    validasi):    VAE={best_alpha:.2f},  
RSVD={best_beta:.2f}")
```

6.4. Save Results

```
evaluation_results = {  
    'timestamp': timestamp,  
    'ensemble_weights': {  
        'best_alpha_vae' : float(best_alpha),  
        'best_beta_rsvd' : float(best_beta),  
        'best_val_rmse'  : float(best_val_rmse)
```

```

    },
    'metrics': {
        'vae_standalone' : {'MSE': float(mse_v), 'RMSE': float(rmse_v), 'MAE':
float(mae_v)},
        'rsvd_standalone' : {'MSE': float(mse_r), 'RMSE': float(rmse_r), 'MAE':
float(mae_r)},
        'hybrid_ensemble' : {'MSE': float(mse_h), 'RMSE': float(rmse_h), 'MAE':
float(mae_h)}
    }
}

evaluation_file_path = os.path.join(results_dir,
f'final_testing_results_{timestamp}.json')
with open(evaluation_file_path, 'w') as f:
    json.dump(evaluation_results, f, indent=4)

print(f'[SUCCESS] Hasil evaluasi disimpan ke: {evaluation_file_path}')

```

Lampiran 5. Program CrossValidation.ipynb

```

# 1. Import Libraries
import os
import sys
import gc
import json
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
from datetime import datetime

```

2. File Paths

```
scripts_path = os.path.abspath(os.path.join('..', 'Scripts'))
if scripts_path not in sys.path:
    sys.path.append(scripts_path)

data_dir      = os.path.abspath(os.path.join('..', 'Data', 'TrainTest'))
hyperparameter_dir = os.path.abspath(os.path.join('..', 'Data',
'HyperParameters'))
cv_output_dir  = os.path.abspath(os.path.join('..', 'Data', 'CrossValidation'))

os.makedirs(cv_output_dir, exist_ok=True)

print(f"[INFO] Scripts path : {scripts_path}")
print(f"[INFO] Data dir    : {data_dir}")
print(f"[INFO] CV output dir : {cv_output_dir}")
```

3. Import Model

```
from model import Encoder, Decoder, VAE, RSVD, KLANnealingCallback
```

4. Load Data & Hyperparameters

4.1. Load Normalized Matrix

```
# Cross validation membutuhkan data sebelum di-split agar setiap fold
```

```
# mendapat pembagian yang adil dan terkontrol.
```

```
try:
```

```
    normalized_matrix = np.load(
        os.path.join(data_dir, 'normalized_matrix.npy')
    ).astype(np.float32)
    num_users, num_items = normalized_matrix.shape
    print(f"[SUCCESS] Normalized matrix: {num_users} users x {num_items}
items")
```

```

except FileNotFoundError:
    print("[ERROR] 'normalized_matrix.npy' tidak ditemukan. Jalankan
    DataPreprocessing terlebih dahulu.")

## 4.2. Load Best Hyperparameters
with open(os.path.join(hyperparameter_dir, 'tuning_progress_vae.json'), 'r') as f:
    best_vae_params = json.load(f)['best_params']

with open(os.path.join(hyperparameter_dir, 'tuning_progress_rsud.json'), 'r') as f:
    best_rsud_params = json.load(f)['best_params']

print(f'[INFO] Best VAE params : {best_vae_params}')
print(f'[INFO] Best RSVD params : {best_rsud_params}')

# 5. Configure Cross Validation
N_FOLDS = 10
CV_VAE_EPOCHS = 50
CV_RSVD_EPOCHS = 30
MAX_RATING = 5.0
RANDOM_SEED = 42

print(f'[INFO] N_FOLDS={N_FOLDS} | VAE Epochs={CV_VAE_EPOCHS}
| RSVD Epochs={CV_RSVD_EPOCHS}')

# 6. Create Fold Splits
## 6.1. All Ratings Coordinates
all_rows, all_cols = np.nonzero(normalized_matrix)
total_ratings = len(all_rows)
print(f'[INFO] Total rating: {total_ratings:,}')

## 6.2. Randomize and Divide into K-Fold

```

```

np.random.seed(RANDOM_SEED)
shuffled_indices = np.random.permutation(total_ratings)
fold_index_groups = np.array_split(shuffled_indices, N_FOLDS)

print(f"[INFO] Pembagian {N_FOLDS} fold:")
for fold_num, fold_indices in enumerate(fold_index_groups):
    print(f"    Fold {fold_num + 1}: {len(fold_indices):,} rating sebagai data
test")

# 7. Helper Functions
## 7.1. Train/Test Matrix per Fold
def create_fold_matrices(normalized_matrix, all_rows, all_cols,
                        test_fold_indices, val_ratio=0.1, random_seed=42):
    """
    Membuat matriks train_inner, val_inner, dan test untuk satu fold.

    Struktur partisi:
        TEST    (~10%) : fold saat ini
        VAL_INNER (~9%) : hold-out dari training, untuk mencari alpha
ensemble
        TRAIN_INNER (~81%): untuk training VAE dan RSVD

    Returns:
        train_inner_matrix, val_inner_indices, val_inner_values,
        test_matrix, test_indices
    """
    # Pisahkan test
    test_rows = all_rows[test_fold_indices]
    test_cols = all_cols[test_fold_indices]

    test_matrix = np.zeros(normalized_matrix.shape, dtype=np.float32)

```



```

test_matrix[test_rows, test_cols] = normalized_matrix[test_rows, test_cols]

train_full_matrix = normalized_matrix.copy()
train_full_matrix[test_rows, test_cols] = 0.0

# Pisahkan val_inner dari sisa training
train_rows, train_cols = np.nonzero(train_full_matrix)
n_train = len(train_rows)
n_val = int(n_train * val_ratio)

rng = np.random.default_rng(random_seed)
val_chosen = rng.choice(n_train, size=n_val, replace=False)
val_inner_rows = train_rows[val_chosen]
val_inner_cols = train_cols[val_chosen]

val_inner_values = train_full_matrix[val_inner_rows, val_inner_cols].copy()
val_inner_indices = (val_inner_rows, val_inner_cols)

train_inner_matrix = train_full_matrix.copy()
train_inner_matrix[val_inner_rows, val_inner_cols] = 0.0

return (train_inner_matrix, val_inner_indices, val_inner_values,
        test_matrix, (test_rows, test_cols))

```

7.2. Training VAE per Fold

```

def train_vae_fold(train_matrix, vae_params, num_items, epochs,
kl_annealing_epochs=15):

```

```

    """

```

Melatih VAE untuk satu fold dengan KL annealing.

Seluruh objek model dihapus setelah selesai untuk mencegah memory leak.

Returns:

pred_matrix : np.ndarray rekonstruksi VAE (skala 0-1)

"""

tf.keras.backend.clear_session()

train_tf = tf.constant(train_matrix, dtype=tf.float32)

encoder = Encoder(

hidden_dims=vae_params['hidden_dims'],

latent_dim=vae_params['latent_dim'],

dropout_rate=vae_params['dropout_rate']

)

decoder = Decoder(hidden_dims=vae_params['hidden_dims'][:-1],
output_dim=num_items)

vae_model = VAE(encoder, decoder, beta=0.0)

_ = vae_model(train_tf[:1])

optimizer =

tf.keras.optimizers.Adam(learning_rate=vae_params['learning_rate'])

vae_model.compile(optimizer=optimizer, run_eagerly=True)

kl_annealing = KLANnealingCallback(

beta_target=vae_params['beta'],

annealing_epochs=kl_annealing_epochs

)

vae_model.fit(

train_tf, train_tf,

epochs=epochs,

batch_size=vae_params['batch_size'],

callbacks=[kl_annealing],

verbose=0

```

)

z_mean, _ = vae_model.encoder.predict(train_tf, verbose=0)
pred_matrix = vae_model.decoder.predict(z_mean, verbose=0)

del encoder, decoder, vae_model, optimizer, train_tf, z_mean, _, kl_annealing
gc.collect()
tf.keras.backend.clear_session()

return pred_matrix

## 7.3. Training RSVD per Fold
def train_rsvd_fold(train_matrix, rsvd_params, epochs):
    """
    Melatih RSVD untuk satu fold dan mengembalikan matriks prediksi penuh.

    Returns:
        pred_matrix : np.ndarray prediksi rating (skala 0-1)
    """
    rsvd_model = RSVD(
        n_factors=rsvd_params['n_factors'],
        learning_rate=rsvd_params['learning_rate'],
        lambda_reg=rsvd_params['lambda_reg'],
        epochs=epochs
    )
    rsvd_model.fit(train_matrix)

    bias_matrix = float(rsvd_model.mu) + rsvd_model.b_u[:, np.newaxis] +
    rsvd_model.b_i[np.newaxis, :]
    latent_matrix = np.dot(np.dot(rsvd_model.U, rsvd_model.Sigma),
    rsvd_model.V.T)

```

```

pred_matrix = bias_matrix + latent_matrix

del rsvd_model, bias_matrix, latent_matrix
gc.collect()

return pred_matrix

## 7.4. Find Best Ensemble Weight
def find_best_ensemble_alpha(pred_vae_full, pred_rsvd_full,
                             val_inner_indices, val_inner_values):
    """
    Mencari alpha terbaik berdasarkan prediksi di val_inner (bukan test set).
    Formula: pred = (alpha * VAE) + ((1 - alpha) * RSVD)

    Returns:
        best_alpha, best_val_rmse
    """
    best_alpha = 0.0
    best_val_rmse = float('inf')

    val_pred_vae = np.clip(pred_vae_full[val_inner_indices] * MAX_RATING,
                           1.0, MAX_RATING)
    val_pred_rsvd = np.clip(pred_rsvd_full[val_inner_indices] *
                           MAX_RATING, 1.0, MAX_RATING)
    val_actual = val_inner_values * MAX_RATING

    for alpha in np.linspace(0, 1, 101):
        beta = 1.0 - alpha
        pred_ensemble = np.clip((alpha * val_pred_vae) + (beta * val_pred_rsvd),
                                1.0, MAX_RATING)
        rmse = np.sqrt(np.mean(np.square(val_actual - pred_ensemble)))

```

```

        if rmse < best_val_rmse:
            best_val_rmse = rmse
            best_alpha = alpha

    return best_alpha, best_val_rmse

## 7.5. Calculate Evaluation Metrics
def calculate_metrics(y_true, y_pred):
    """Menghitung MSE, RMSE, dan MAE sekaligus."""
    mse = np.mean(np.square(y_true - y_pred))
    rmse = np.sqrt(mse)
    mae = np.mean(np.abs(y_true - y_pred))
    return mse, rmse, mae

# 8. Execute K-Fold Cross Validation
# -----
cv_results = {
    'fold': [], 'test_size': [], 'best_alpha': [], 'best_val_rmse': [],
    'vae_mse': [], 'vae_rmse': [], 'vae_mae': [],
    'rsvd_mse': [], 'rsvd_rmse': [], 'rsvd_mae': [],
    'hybrid_mse': [], 'hybrid_rmse': [], 'hybrid_mae': [],
}

for fold_idx, test_fold_indices in enumerate(fold_index_groups):
    fold_num = fold_idx + 1
    print(f"\n{'='*60}")
    print(f" FOLD {fold_num} / {N_FOLDS}")
    print(f"{'='*60}")

    # Langkah 1: Buat matriks train_inner, val_inner, test

```

```

(train_inner_matrix, val_inner_indices, val_inner_values,
 test_matrix, test_indices) = create_fold_matrices(
    normalized_matrix, all_rows, all_cols, test_fold_indices,
    val_ratio=0.1, random_seed=RANDOM_SEED
)

actual_values = test_matrix[test_indices] * MAX_RATING
print(f" [INFO] Train_inner: {int(np.sum(train_inner_matrix > 0))}, | "
      f"Val_inner: {len(val_inner_values)}, | Test: {len(actual_values)}, |")

# Langkah 2: Training VAE
print(f" [VAE] Mulai training ({CV_VAE_EPOCHS} epoch)...")
pred_vae_full = train_vae_fold(train_inner_matrix, best_vae_params,
num_items, CV_VAE_EPOCHS)
print(f" [VAE] Selesai.")

# Langkah 3: Training RSVD
print(f" [RSVD] Mulai training ({CV_RSVD_EPOCHS} epoch)...")
pred_rsvd_full = train_rsvd_fold(train_inner_matrix, best_rsvd_params,
CV_RSVD_EPOCHS)
print(f" [RSVD] Selesai.")

# Langkah 4: Cari alpha terbaik di val_inner
print(f" [ENSEMBLE] Mencari alpha terbaik di val_inner...")
best_alpha, best_val_rmse = find_best_ensemble_alpha(
    pred_vae_full, pred_rsvd_full, val_inner_indices, val_inner_values
)
best_beta = 1.0 - best_alpha
print(f" [ENSEMBLE] Alpha={best_alpha:.2f}, RSVD={best_beta:.2f} (val
RMSE={best_val_rmse:.4f})")

```

```

# Langkah 5: Evaluasi di test set

pred_vae_test = np.clip(pred_vae_full[test_indices] * MAX_RATING, 1.0,
MAX_RATING)

pred_rsvd_test = np.clip(pred_rsvd_full[test_indices] * MAX_RATING, 1.0,
MAX_RATING)

pred_hybrid_test = np.clip(
    (best_alpha * pred_vae_test) + (best_beta * pred_rsvd_test), 1.0,
MAX_RATING
)

mse_v, rmse_v, mae_v = calculate_metrics(actual_values, pred_vae_test)
mse_r, rmse_r, mae_r = calculate_metrics(actual_values, pred_rsvd_test)
mse_h, rmse_h, mae_h = calculate_metrics(actual_values, pred_hybrid_test)

# Langkah 6: Simpan hasil fold
cv_results['fold'].append(fold_num)
cv_results['test_size'].append(len(actual_values))
cv_results['best_alpha'].append(float(best_alpha))
cv_results['best_val_rmse'].append(float(best_val_rmse))
cv_results['vae_mse'].append(float(mse_v));
cv_results['vae_rmse'].append(float(rmse_v));
cv_results['vae_mae'].append(float(mae_v))
    cv_results['rsvd_mse'].append(float(mse_r));
cv_results['rsvd_rmse'].append(float(rmse_r));
cv_results['rsvd_mae'].append(float(mae_r))
    cv_results['hybrid_mse'].append(float(mse_h));
cv_results['hybrid_rmse'].append(float(rmse_h));
cv_results['hybrid_mae'].append(float(mae_h))

print(f'\n  {'Metrik':<10} | {'VAE':^12} | {'RSVD':^12} | {'Hybrid':^12}")
print(f"  {'-'*10}-+{'-'*12}-+{'-'*12}-+{'-'*12}")

```

```

    print(f' {'MSE':<10} | {mse_v:^12.4f} | {mse_r:^12.4f} | {mse_h:^12.4f}")
    print(f'      {'RMSE':<10} |   {rmse_v:^12.4f}   |   {rmse_r:^12.4f}   |
{rmse_h:^12.4f}")
    print(f' {'MAE':<10} | {mae_v:^12.4f} | {mae_r:^12.4f} | {mae_h:^12.4f}")

    del train_inner_matrix, test_matrix, val_inner_indices, val_inner_values
    del actual_values, pred_vae_full, pred_rsvd_full
    del pred_vae_test, pred_rsvd_test, pred_hybrid_test
    gc.collect()

print(f'\n{'='*60}")
print(f' SEMUA FOLD SELESAI")
print(f'{'='*60}")

# 9. Cross Validation Result
## 9.1. Calculate Mean and Std
metric_keys = [
    'vae_mse', 'vae_rmse', 'vae_mae',
    'rsvd_mse', 'rsvd_rmse', 'rsvd_mae',
    'hybrid_mse', 'hybrid_rmse', 'hybrid_mae',
]

cv_summary = {}
for key in metric_keys:
    values = np.array(cv_results[key])
    cv_summary[key] = {
        'mean': float(np.mean(values)),
        'std' : float(np.std(values)),
        'min' : float(np.min(values)),
        'max' : float(np.max(values))
    }

```



```

print("\n" + "="*70)
print(f'RINGKASAN CROSS VALIDATION — MEAN ± STD DEV
({N_FOLDS} FOLD)')
print("="*70)
print(f'  {Model':<10} | {'MSE':^20} | {'RMSE':^20} | {'MAE':^20}')
print(f'  {'-'*10}-+-{'-'*20}-+-{'-'*20}-+-{'-'*20}')

for model, label in zip(['vae', 'rsvd', 'hybrid'], ['VAE', 'RSVD', 'Hybrid']):
    mse_str      =      f'{cv_summary[f'{model}_mse']['mean']:.4f}      ±
{cv_summary[f'{model}_mse']['std']:.4f}'
    rmse_str     =      f'{cv_summary[f'{model}_rmse']['mean']:.4f}      ±
{cv_summary[f'{model}_rmse']['std']:.4f}'
    mae_str      =      f'{cv_summary[f'{model}_mae']['mean']:.4f}      ±
{cv_summary[f'{model}_mae']['std']:.4f}'
    print(f'  {label:<10} | {mse_str:^20} | {rmse_str:^20} | {mae_str:^20}')

print("="*70)

## 9.2. Save to JSON
timestamp = datetime.now().strftime('%Y%m%d_%H%M')
cv_run_dir = os.path.join(cv_output_dir, f'cv_{timestamp}')
os.makedirs(cv_run_dir, exist_ok=True)

final_cv_output = {
    'timestamp' : timestamp,
    'configuration': {
        'n_folds'      : N_FOLDS,
        'cv_vae_epochs' : CV_VAE_EPOCHS,
        'cv_rsvd_epochs': CV_RSVD_EPOCHS,
        'random_seed'  : RANDOM_SEED,
    }
}

```

```

        'vae_params' : best_vae_params,
        'rsvd_params' : best_rsvd_params,
    },
    'per_fold_results': cv_results,
    'summary' : cv_summary,
}

cv_json_path = os.path.join(cv_run_dir, f'cv_results_{timestamp}.json')
with open(cv_json_path, 'w') as f:
    json.dump(final_cv_output, f, indent=4)

print(f'[SUCCESS] Hasil CV tersimpan di: {cv_json_path}')

# 10. Visualization
## 10.1. Metric per Fold
fold_labels = [f'Fold {i}' for i in cv_results['fold']]
x_positions = np.arange(N_FOLDS)
bar_width = 0.25
alpha_values = cv_results['best_alpha']

metric_bar_configs = [
    ('rmse', 'RMSE', 'cv_rmse_per_fold'),
    ('mse', 'MSE', 'cv_mse_per_fold'),
    ('mae', 'MAE', 'cv_mae_per_fold'),
]

for metric_key, metric_label, file_prefix in metric_bar_configs:
    fig, ax = plt.subplots(figsize=(14, 7), dpi=100)

    bars_vae = ax.bar(x_positions - bar_width, cv_results[f'vae_{metric_key}'],
bar_width, label='VAE', color='#2ca02c', alpha=0.85)

```

```

bars_rsvd = ax.bar(x_positions, cv_results[f'rsvd_{metric_key}'],
bar_width, label='RSVD', color='#1f77b4', alpha=0.85)

bars_hybrid = ax.bar(x_positions + bar_width,
cv_results[f'hybrid_{metric_key}'], bar_width, label='Hybrid', color='#d62728',
alpha=0.85)

ax.axhline(cv_summary[f'vae_{metric_key}']['mean'], color='#2ca02c',
linestyle='--', linewidth=1.5, alpha=0.7, label=f'Mean VAE =
{cv_summary[f'vae_{metric_key}']['mean']:.4f}')
ax.axhline(cv_summary[f'rsvd_{metric_key}']['mean'], color='#1f77b4',
linestyle='--', linewidth=1.5, alpha=0.7, label=f'Mean RSVD =
{cv_summary[f'rsvd_{metric_key}']['mean']:.4f}')
ax.axhline(cv_summary[f'hybrid_{metric_key}']['mean'], color='#d62728',
linestyle='--', linewidth=1.5, alpha=0.7, label=f'Mean Hybrid =
{cv_summary[f'hybrid_{metric_key}']['mean']:.4f}')

for bar in [*bars_vae, *bars_rsvd, *bars_hybrid]:
    height = bar.get_height()
    ax.text(bar.get_x() + bar.get_width() / 2.0, height + 0.002,
            f'{height:.4f}', ha='center', va='bottom', fontsize=7.5, rotation=90)

for fold_idx, alpha_val in enumerate(alpha_values):
    ax.text(x_positions[fold_idx], ax.get_ylim()[0] - (ax.get_ylim()[1] -
ax.get_ylim()[0]) * 0.04,
            f' $\alpha$ ={alpha_val:.2f}', ha='center', va='top', fontsize=8,
color='#555555', fontstyle='italic')

ax.set_title(f'Perbandingan {metric_label} per Fold — VAE vs RSVD vs
Hybrid', fontsize=14, fontweight='bold', pad=15)
ax.set_xlabel('Fold ( $\alpha$  = bobot VAE dalam ensemble)', fontsize=12)
ax.set_ylabel(f'{metric_label} (lebih rendah lebih baik)', fontsize=12)

```

```

ax.set_xticks(x_positions)
ax.set_xticklabels(fold_labels)
ax.legend(fontsize=9, loc='center right')
ax.grid(True, axis='y', linestyle='--', alpha=0.5)
ax.yaxis.set_major_formatter(ticker.FormatStrFormatter("%.4f"))

ymin, ymax = ax.get_ylim()
ax.set_ylim(ymin - (ymax - ymin) * 0.07, ymax)

plt.tight_layout()
plot_path = os.path.join(cv_run_dir, f'{file_prefix}_{timestamp}.png')
plt.savefig(plot_path, dpi=300, bbox_inches='tight')
plt.show()
print(f'[SUCCESS] Grafik {metric_label} per fold: {plot_path}')

## 10.2. Mean  $\pm$  Std Dev
metric_display = ['MSE', 'RMSE', 'MAE']
metric_keys_display = ['mse', 'rmse', 'mae']

mean_alpha = float(np.mean(cv_results['best_alpha']))
std_alpha = float(np.std(cv_results['best_alpha']))

fig, axes = plt.subplots(1, 3, figsize=(16, 6), dpi=100)

model_names = ['VAE', 'RSVD', 'Hybrid']
model_keys = ['vae', 'rsvd', 'hybrid']
model_colors = ['#2ca02c', '#1f77b4', '#d62728']
x_pos = np.arange(len(model_names))

for ax_idx, (metric_label, metric_key) in enumerate(zip(metric_display,
metric_keys_display)):

```

```

ax = axes[ax_idx]

means = [cv_summary[f'{m}_{metric_key}']['mean'] for m in model_keys]
stds = [cv_summary[f'{m}_{metric_key}']['std'] for m in model_keys]

bars = ax.bar(x_pos, means, color=model_colors, alpha=0.85, width=0.5,
              yerr=stds, capsize=8, error_kw={'linewidth': 2, 'ecolor': 'black'})

for bar, mean_val, std_val in zip(bars, means, stds):
    ax.text(bar.get_x() + bar.get_width() / 2.0,
            mean_val + std_val + (mean_val * 0.01),
            f'{mean_val:.4f}\n±{std_val:.4f}',
            ha='center', va='bottom', fontsize=9, fontweight='bold')

ax.set_title(f'Mean {metric_label} ± Std Dev', fontsize=13, fontweight='bold',
pad=12)
ax.set_ylabel(metric_label, fontsize=11)
ax.set_xticks(x_pos)
ax.set_xticklabels(model_names, fontsize=11)
ax.grid(True, axis='y', linestyle='--', alpha=0.5)
ax.yaxis.set_major_formatter(ticker.FormatStrFormatter("%.4f"))

fig.suptitle(
    f'Ringkasan {N_FOLDS}-Fold Cross Validation — Mean ± Std Dev\n'
    f'Rata-rata Bobot Ensemble: VAE  $\alpha$  = {mean_alpha:.3f} ± {std_alpha:.3f} | '
    f'RSVD  $\beta$  = {1 - mean_alpha:.3f} ± {std_alpha:.3f}',
    fontsize=13, fontweight='bold', y=1.04
)
plt.tight_layout()

```

```

summary_plot_path = os.path.join(cv_run_dir,
f'cv_summary_errorbar_{timestamp}.png')
plt.savefig(summary_plot_path, dpi=300, bbox_inches='tight')
plt.show()
print(f'[SUCCESS] Grafik ringkasan: {summary_plot_path}')

## 10.3. Metric Trend Over Folds
fig, axes = plt.subplots(1, 3, figsize=(16, 5), dpi=100)

fold_x = cv_results['fold']
alpha_values = cv_results['best_alpha']

for ax_idx, (metric_label, metric_key) in enumerate(zip(metric_display,
metric_keys_display)):
    ax = axes[ax_idx]

    ax.plot(fold_x, cv_results[f'vae_{metric_key}'], marker='o', linewidth=2,
color='#2ca02c', label='VAE', markersize=7)
    ax.plot(fold_x, cv_results[f'rsvd_{metric_key}'], marker='s', linewidth=2,
color='#1f77b4', label='RSVD', markersize=7)
    ax.plot(fold_x, cv_results[f'hybrid_{metric_key}'], marker='^', linewidth=2,
color='#d62728', label='Hybrid', markersize=7)

    ax.set_title(f'Tren {metric_label} Antar Fold', fontsize=13, fontweight='bold',
pad=12)
    ax.set_xlabel('Fold ke-', fontsize=11)
    ax.set_ylabel(metric_label, fontsize=11)
    ax.set_xticks(fold_x)
    ax.grid(True, linestyle='--', alpha=0.5)
    ax.yaxis.set_major_formatter(ticker.FormatStrFormatter('%0.4f'))

```

```

# Axis kanan: tren alpha VAE per fold
ax_right = ax.twinx()
ax_right.plot(fold_x, alpha_values, marker='D', linewidth=1.5,
              color='#ff7f0e', linestyle='--', label='Alpha VAE ( $\alpha$ )', markersize=6,
              alpha=0.85)
ax_right.set_ylabel('Alpha VAE ( $\alpha$ )', fontsize=10, color='#ff7f0e')
ax_right.set_ylim(0, 1)
ax_right.tick_params(axis='y', labelcolor='#ff7f0e')
ax_right.yaxis.set_major_formatter(ticker.FormatStrFormatter('%0.2f'))

lines_left, labels_left = ax.get_legend_handles_labels()
lines_right, labels_right = ax_right.get_legend_handles_labels()
ax.legend(lines_left + lines_right, labels_left + labels_right, fontsize=9,
          loc='center right')

fig.suptitle('Tren Metrik Evaluasi per Fold — Konsistensi Model', fontsize=14,
            fontweight='bold', y=1.02)
plt.tight_layout()

trend_plot_path = os.path.join(cv_run_dir,
                                f'cv_trend_per_fold_{timestamp}.png')
plt.savefig(trend_plot_path, dpi=300, bbox_inches='tight')
plt.show()
print(f'[SUCCESS] Grafik tren: {trend_plot_path}')

# 11. Result Interpretation
# -----
hybrid_rmse_mean = cv_summary['hybrid_rmse']['mean']
hybrid_rmse_std = cv_summary['hybrid_rmse']['std']
vae_rmse_mean = cv_summary['vae_rmse']['mean']
rsvd_rmse_mean = cv_summary['rsvd_rmse']['mean']

```

```

best_model_name = min(
    [('VAE',    vae_rmse_mean),    ('RSVD',    rsvd_rmse_mean),    ('Hybrid',
hybrid_rmse_mean)],
    key=lambda x: x[1]
)[0]

# Koefisien variasi (CV%) sebagai ukuran konsistensi: <10% sangat konsisten
cv_percent    = (hybrid_rmse_std / hybrid_rmse_mean) * 100
consistency_label = ("sangat konsisten" if cv_percent < 10
                     else ("cukup konsisten" if cv_percent < 20 else "perlu ditinjau
ulang"))

print("\n" + "="*65)
print(" INTERPRETASI HASIL CROSS VALIDATION")
print("="*65)
print(f"\n Model terbaik berdasarkan mean RMSE: {best_model_name}")
print(f"\n Hybrid RMSE: {hybrid_rmse_mean:.4f} ± {hybrid_rmse_std:.4f}")
print(f" CV%      : {cv_percent:.2f}% → Model {consistency_label}")
print(f"\n Rata-rata bobot ensemble per fold:")
print(f"  VAE (alpha) = {np.mean(cv_results['best_alpha']):.2f}")
print(f"  RSVD (beta)  = {1 - np.mean(cv_results['best_alpha']):.2f}")
print("\n" + "="*65)

# 12. Paired t-test
# Uji signifikansi statistik untuk membuktikan perbedaan performa
# antar model bukan karena kebetulan.
#
# H0: Tidak ada perbedaan signifikan (mean RMSE sama)
# H1: Ada perbedaan signifikan (mean RMSE berbeda)
# Kriteria: tolak H0 jika p-value < 0.05

```



```
import os
import json
import numpy as np
from scipy import stats

cv_output_dir = os.path.abspath(os.path.join('.', 'Data', 'CrossValidation'))

# Ganti nama file sesuai sesi CV yang ingin diuji
CV_RESULT_FILE = 'cv_20260313_2116/cv_results_20260313_2116.json'
cv_json_path = os.path.join(cv_output_dir, CV_RESULT_FILE)

try:
    with open(cv_json_path, 'r') as f:
        cv_data = json.load(f)
        print(f'[SUCCESS] Load: {cv_json_path}')
        print(f'[INFO] Timestamp : {cv_data['timestamp']}')
        print(f'[INFO] N Folds : {cv_data['configuration']['n_folds']}')
except FileNotFoundError:
    print(f'[ERROR] File tidak ditemukan: {cv_json_path}')
    raise

per_fold = cv_data['per_fold_results']
N_FOLDS = cv_data['configuration']['n_folds']
rmse_vae = np.array(per_fold['vae_rmse'])
rmse_rsvd = np.array(per_fold['rsvd_rmse'])
rmse_hybrid = np.array(per_fold['hybrid_rmse'])
mae_vae = np.array(per_fold['vae_mae'])
mae_rsvd = np.array(per_fold['rsvd_mae'])
mae_hybrid = np.array(per_fold['hybrid_mae'])
```

```

# Paired t-test untuk RMSE dan MAE
t_rmse_h_r, p_rmse_h_r = stats.ttest_rel(rmse_hybrid, rmse_rsvd)
t_rmse_h_v, p_rmse_h_v = stats.ttest_rel(rmse_hybrid, rmse_vae)
t_rmse_r_v, p_rmse_r_v = stats.ttest_rel(rmse_rsvd, rmse_vae)

t_mae_h_r, p_mae_h_r = stats.ttest_rel(mae_hybrid, mae_rsvd)
t_mae_h_v, p_mae_h_v = stats.ttest_rel(mae_hybrid, mae_vae)
t_mae_r_v, p_mae_r_v = stats.ttest_rel(mae_rsvd, mae_vae)

alpha_level = 0.05

for metric_label, comparisons in [
    ('RMSE', [
        ('Hybrid vs RSVD', t_rmse_h_r, p_rmse_h_r),
        ('Hybrid vs VAE', t_rmse_h_v, p_rmse_h_v),
        ('RSVD vs VAE', t_rmse_r_v, p_rmse_r_v),
    ]),
    ('MAE', [
        ('Hybrid vs RSVD', t_mae_h_r, p_mae_h_r),
        ('Hybrid vs VAE', t_mae_h_v, p_mae_h_v),
        ('RSVD vs VAE', t_mae_r_v, p_mae_r_v),
    ]),
]:
    print("\n" + "="*65)
    print(f"    PAIRED t-TEST — {metric_label} (N_FOLDS={N_FOLDS},
alpha={alpha_level})")
    print("="*65)
    print(f"    {'Perbandingan':<25} | {'t-statistic':^14} | {'p-value':^12} |
Kesimpulan")
    print(f"    {'-'*25}-+{'-'*14}-+{'-'*12}-+{'-'*20}")
    for label, t_stat, p_val in comparisons:

```

```

        kesimpulan = 'Signifikan *' if p_val < alpha_level else 'Tidak Signifikan'
        print(f' {label:<25} | {t_stat:^14.4f} | {p_val:^12.4f} | {kesimpulan}')

print("\n Catatan: * = H0 ditolak, perbedaan signifikan pada alpha = 0.05")
print("      t negatif = model pertama memiliki nilai lebih rendah (lebih baik)")
print("="*65)

# Simpan hasil t-test ke file JSON hasil CV
ttest_results = {
    'alpha' : alpha_level,
    'n_folds': N_FOLDS,
    'rmse': {
        'hybrid_vs_rsvd': {'t_statistic': float(t_rmse_h_r), 'p_value':
float(p_rmse_h_r), 'significant': bool(p_rmse_h_r < alpha_level)},
        'hybrid_vs_vae' : {'t_statistic': float(t_rmse_h_v), 'p_value':
float(p_rmse_h_v), 'significant': bool(p_rmse_h_v < alpha_level)},
        'rsvd_vs_vae' : {'t_statistic': float(t_rmse_r_v), 'p_value':
float(p_rmse_r_v), 'significant': bool(p_rmse_r_v < alpha_level)},
    },
    'mae': {
        'hybrid_vs_rsvd': {'t_statistic': float(t_mae_h_r), 'p_value':
float(p_mae_h_r), 'significant': bool(p_mae_h_r < alpha_level)},
        'hybrid_vs_vae' : {'t_statistic': float(t_mae_h_v), 'p_value':
float(p_mae_h_v), 'significant': bool(p_mae_h_v < alpha_level)},
        'rsvd_vs_vae' : {'t_statistic': float(t_mae_r_v), 'p_value': float(p_mae_r_v),
'significant': bool(p_mae_r_v < alpha_level)},
    },
}

cv_data['paired_ttest'] = ttest_results
with open(cv_json_path, 'w') as f:

```

```
json.dump(cv_data, f, indent=4)

print(f"\n[SUCCESS] Hasil t-test ditambahkan ke: {cv_json_path}")
```

Lampiran 6. Program model.py

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.layers import Layer, Dense, Dropout
from tensorflow.keras.models import Model

class Sampling(Layer):
    """
    Kelas ini mengimplementasikan 'Reparameterization Trick'.
    Tujuannya agar operasi pengambilan sampel acak (random sampling) dari
    ruang laten tetap bisa diturunkan secara matematis (differentiable),
    sehingga proses Backpropagation pada jaringan saraf tiruan bisa berjalan.
    """
    def call(self, inputs):
        z_mean, z_log_var = inputs
        batch = tf.shape(z_mean)[0]
        dim = tf.shape(z_mean)[1]

        # Mengambil noise acak (epsilon) dari distribusi normal standar (mean=0,
        std=1)
        epsilon = tf.keras.backend.random_normal(shape=(batch, dim))

        # Menghitung standar deviasi (sigma) dari log_var
        sigma = tf.exp(0.5 * z_log_var)

        # Mengembalikan titik sampel laten: z = mean + (sigma * epsilon)
```

```
return z_mean + (sigma * epsilon)
```

```
class Encoder(Model):
```

```
    """
```

```
    Encoder bertugas memetakan data rating asli dari pengguna (dimensi besar)
    menjadi representasi selera laten (dimensi kecil/kompak).
    """
```

```
    def __init__(self, hidden_dims=[512, 256], latent_dim=50, dropout_rate=0.5,
    **kwargs):
```

```
        super(Encoder, self).__init__(**kwargs)
```

```
        # Dropout digunakan untuk mencegah overfitting dengan mematikan
        sebagian neuron secara acak
```

```
        self.dropout = Dropout(dropout_rate)
```

```
        # Membangun lapisan tersembunyi (Hidden Layers) secara dinamis
```

```
        self.hidden_layers = [Dense(dim, activation='relu') for dim in hidden_dims]
```

```
        # Output layer dari Encoder: Nilai Rata-rata (Mean) dan Log Varians dari
        distribusi ruang laten
```

```
        self.z_mean = Dense(latent_dim, name='z_mean')
```

```
        self.z_log_var = Dense(latent_dim, name='z_log_var')
```

```
    def call(self, inputs):
```

```
        x = self.dropout(inputs)
```

```
        for layer in self.hidden_layers:
```

```
            x = layer(x)
```

```
        return self.z_mean(x), self.z_log_var(x)
```

```
class Decoder(Model):
```

```
    """
```

Decoder bertugas membangun ulang (rekonstruksi) matriks laten kembali menjadi bentuk tebakan rating asli (dengan dimensi awal).

"""

```
def __init__(self, hidden_dims=[256, 512], output_dim=1682, **kwargs):  
    super(Decoder, self).__init__(**kwargs)  
    self.hidden_layers = [Dense(dim, activation='relu') for dim in hidden_dims]
```

Layer output menggunakan Sigmoid karena data input sudah dinormalisasi ke rentang 0 - 1

```
self.reconstruction = Dense(output_dim, activation='sigmoid',  
name='decoder_output')
```

```
def call(self, inputs):  
    x = inputs  
    for layer in self.hidden_layers:  
        x = layer(x)  
    return self.reconstruction(x)
```

```
class VAE(Model):
```

"""

Model utama yang menggabungkan Encoder, Sampling, dan Decoder. Termasuk penyesuaian perhitungan Loss Function menggunakan Beta-VAE untuk mencegah masalah Posterior Collapse.

"""

Tambahkan parameter beta di init (default 1.0 seperti VAE biasa)

```
def __init__(self, encoder, decoder, beta=1.0, **kwargs):  
    super(VAE, self).__init__(**kwargs)  
    self.encoder = encoder  
    self.decoder = decoder
```

Simpan beta sebagai tf.Variable agar nilainya bisa diperbarui secara

```

        # dinamis oleh KLANnealingCallback di setiap epoch tanpa perlu rebuild
model

        self.beta = tf.Variable(float(beta), trainable=False, dtype=tf.float32,
name='beta')

        self.sampling = Sampling()

        # Pelacak nilai metrik selama proses training
        self.total_loss_tracker = tf.keras.metrics.Mean(name="total_loss")
                                self.reconstruction_loss_tracker =
tf.keras.metrics.Mean(name="reconstruction_loss")
        self.kl_loss_tracker = tf.keras.metrics.Mean(name="kl_loss")

        @property
        def metrics(self):
            return [self.total_loss_tracker, self.reconstruction_loss_tracker,
self.kl_loss_tracker]

        def call(self, inputs):
            # Forward pass biasa untuk tahap evaluasi/testing
            z_mean, z_log_var = self.encoder(inputs)
            z = self.sampling([z_mean, z_log_var])
            return self.decoder(z)

        def train_step(self, data):
            """
            Modifikasi proses training: VAE HANYA belajar dari rating yang ada
nilainya (> 0),
            dan mengabaikan (masking) film yang belum diberi rating oleh pengguna.
            """
            if isinstance(data, tuple):

```

```

data = data[0] # ambil x saja, y dibuang

with tf.GradientTape() as tape:
    z_mean, z_log_var = self.encoder(data)
    z = self.sampling([z_mean, z_log_var])
    reconstruction = self.decoder(z)

# A. Reconstruction Loss (Masked MSE)
# Membuat mask bernilai 1 jika rating ada, dan 0 jika kosong
mask = tf.cast(data > 0, tf.float32)
squared_diff = tf.square(data - reconstruction)
masked_se = squared_diff * mask

# Menghitung rata-rata error murni
mse_loss = tf.reduce_sum(masked_se) / (tf.reduce_sum(mask) + 1e-8)
num_items = tf.cast(tf.shape(data)[1], tf.float32)
reconstruction_loss = mse_loss * num_items

# B. KL Divergence Loss
# Berfungsi meregulasi agar distribusi ruang laten mendekati normal
Gaussian
kl_loss = -0.5 * (1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var))
kl_loss = tf.reduce_mean(tf.reduce_sum(kl_loss, axis=1))

# C. Total Loss dengan Hyperparameter Beta
# Gunakan self.beta yang didapat dari inisialisasi model
total_loss = reconstruction_loss + (self.beta * kl_loss)

# Backpropagation: Menghitung gradien dan memperbarui bobot (weights)
grads = tape.gradient(total_loss, self.trainable_weights)
self.optimizer.apply_gradients(zip(grads, self.trainable_weights))

```



```

# Memperbarui history loss untuk ditampilkan di progress bar
self.total_loss_tracker.update_state(total_loss)
self.reconstruction_loss_tracker.update_state(reconstruction_loss)
self.kl_loss_tracker.update_state(kl_loss)

return {
    "loss": self.total_loss_tracker.result(),
    "reconstruction_loss": self.reconstruction_loss_tracker.result(),
    "kl_loss": self.kl_loss_tracker.result(),
}

class KLANnealingCallback(tf.keras.callbacks.Callback):
    """
    Callback untuk menaikkan nilai beta VAE secara bertahap (linear annealing)
    dari 0.0 menuju beta_target selama fase annealing.
    """
    def __init__(self, beta_target, annealing_epochs):
        """
        Args:
            beta_target : nilai beta akhir yang ingin dicapai,
                        diambil dari hasil hyperparameter tuning
            annealing_epochs : jumlah epoch untuk menaikkan beta dari 0 ke
            beta_target
        """
        super(KLANnealingCallback, self).__init__()
        self.beta_target = beta_target
        self.annealing_epochs = annealing_epochs

    def on_epoch_begin(self, epoch, logs=None):
        """

```

Dipanggil otomatis oleh Keras di awal setiap epoch.

Menghitung dan menetapkan nilai beta baru secara linear.

"""

Hitung proporsi kemajuan annealing (0.0 hingga 1.0)

if self.annealing_epochs > 0:

 progress = min(epoch / self.annealing_epochs, 1.0)

else:

 progress = 1.0

Hitung nilai beta saat ini secara linear

beta_now = progress * self.beta_target

Perbarui tf.Variable beta di model tanpa perlu rebuild

self.model.beta.assign(beta_now)

def on_epoch_end(self, epoch, logs=None):

"""

Tampilkan nilai beta aktif di akhir setiap epoch untuk monitoring.

"""

current_beta = float(self.model.beta.numpy())

print(f" [KL Annealing] Epoch {epoch + 1}: beta = {current_beta:.6f} /
{self.beta_target}")

class RSVD:

"""

Implementasi algoritma faktorisasi matriks tradisional yang diperkaya dengan:

1. Perhitungan Bias (Global, User, Item)

2. Identitas Inisialisasi pada matriks Sigma untuk menghindari Vanishing Gradient

"""

```

def __init__(self, n_factors=50, learning_rate=0.001, lambda_reg=0.001,
epochs=100, patience=5):
    self.k      = n_factors    # Jumlah dimensi laten
    self.eta    = learning_rate # Kecepatan belajar (Learning rate)
    self.lam    = lambda_reg   # Penalti regularisasi untuk mencegah
overfitting
    self.epochs = epochs
    self.patience = patience    # Jumlah epoch tanpa perbaikan val MSE
sebelum dihentikan
    self.loss_history = []      # Menyimpan nilai Total Loss (MSE + L2 Penalty)
setiap epoch

def fit(self, Z, val_ratio=0.1, random_seed=42):
    """
    Melatih model RSVD dengan validation split untuk early stopping.

    Args:
        Z      : np.ndarray matriks rating (users x items), nilai 0 = belum dirating
        val_ratio : float proporsi rating yang dipakai sebagai validation set
        (default 10%)
        random_seed : int seed untuk reproduksibilitas pembagian validation split
    """
    m, n = Z.shape # m = Jumlah pengguna, n = Jumlah film

    # -----
    # Validation Split
    # Pisahkan sebagian rating sebagai validation set
    # agar early stopping bisa memantau performa di data
    # yang tidak dilihat selama training
    # -----
    np.random.seed(random_seed)

```

```

# Ambil koordinat semua rating yang ada nilainya
rated_indices = np.argwhere(Z > 0)
n_val = int(len(rated_indices) * val_ratio)

# Pilih indeks validation secara acak tanpa penggantian
val_chosen = np.random.choice(len(rated_indices), n_val, replace=False)
val_idx = rated_indices[val_chosen]
val_rows = val_idx[:, 0]
val_cols = val_idx[:, 1]

# Simpan nilai rating asli untuk evaluasi validation
val_true = Z[val_rows, val_cols].copy()

# Buat training matrix: salin Z lalu hapus rating validation
Z_train = Z.copy()
Z_train[val_rows, val_cols] = 0.0

print(f'[INFO] Total rating : {len(rated_indices)}')
print(f'[INFO] Training ratings : {len(rated_indices) - n_val}')
print(f'[INFO] Validation ratings: {n_val:,}')

# Hitung rata-rata global HANYA dari rating training (val sudah dihapus)
nonzero_ratings = Z_train[Z_train > 0]
self.mu = np.mean(nonzero_ratings) if len(nonzero_ratings) > 0 else 0

# Bias user (pemberi rating) dan item (film)
self.b_u = np.zeros(m)
self.b_i = np.zeros(n)

# Inisialisasi U dan V dengan nilai kecil

```

```

self.U = np.random.normal(scale=0.1, size=(m, self.k))
self.V = np.random.normal(scale=0.1, size=(n, self.k))

# Inisialisasi Sigma dengan Matriks Identitas
# Mencegah error gradient terperangkap mendekati 0 di epoch awal
self.Sigma = np.eye(self.k)

# Variabel untuk early stopping — memantau validation MSE
best_val_mse = float('inf')
no_improve = 0

# Snapshot bobot terbaik (disimpan saat val MSE mencapai nilai terendah)
best_U, best_V, best_Sigma = None, None, None
best_b_u, best_b_i = None, None

for epoch in range(self.epochs):
    for u in range(m):
        for i in range(n):
            # KUNCI UTAMA: Hanya perbarui bobot jika user benar-benar
memberi rating
            # Gunakan Z_train (bukan Z) agar rating validation tidak ikut dilatih
            if Z_train[u, i] > 0:
                # Prediksi rating: Bias Global + Bias User + Bias Item + (U *
Sigma * V^T)
                dot_product = np.dot(np.dot(self.U[u, :], self.Sigma), self.V[i,
:].T)

                pred = self.mu + self.b_u[u] + self.b_i[i] + dot_product

                # Hitung jarak error (Asli dikurangi Prediksi)
                e_ui = Z_train[u, i] - pred

```

```

# Update Bias menggunakan Gradient Descent & L2 Regularization
self.b_u[u] += self.eta * (e_ui - self.lam * self.b_u[u])
self.b_i[i] += self.eta * (e_ui - self.lam * self.b_i[i])

# Update Matriks Laten
for k in range(self.k):
    U_uk = self.U[u, k]
    V_ik = self.V[i, k]
    Sigma_kk = self.Sigma[k, k]

    # Memperbarui bobot U, V, dan Sigma berdasarkan gradien
error
self.U[u, k] += self.eta * (e_ui * Sigma_kk * V_ik - self.lam
* U_uk)
self.V[i, k] += self.eta * (e_ui * Sigma_kk * U_uk - self.lam
* V_ik)
self.Sigma[k, k] += self.eta * (e_ui * U_uk * V_ik)

# Rekonstruksi matriks penuh
bias_matrix = self.mu + self.b_u[:, np.newaxis] + self.b_i[np.newaxis,
:]

latent_matrix = np.dot(np.dot(self.U, self.Sigma), self.V.T)
reconstructed_Z = bias_matrix + latent_matrix

# Hitung training MSE (hanya pada rating training, sel kosong dan val
diabaikan)
train_mask = (Z_train > 0)
current_mse = np.sum(np.square(Z_train[train_mask] -
reconstructed_Z[train_mask])) / np.sum(train_mask)

# Hitung validation MSE (pada rating yang disisihkan di awal)

```

```

val_pred = reconstructed_Z[val_rows, val_cols]
val_mse = np.mean(np.square(val_true - val_pred))

# Hitung penalti regularisasi L2 sesuai rumus (8) proposal:
#  $\lambda * (\|U\|_F^2 + \|V\|_F^2)$ 
# Catatan: Sigma tidak diregularisasi (lihat Catatan Desain di docstring)
l2_penalty = self.lam * (np.sum(np.square(self.U)) +
np.sum(np.square(self.V)))

# Total Loss = Training MSE + L2 Penalty (rumus 8 proposal)
current_loss = current_mse + l2_penalty
self.loss_history.append(current_loss)

# Print progress setiap epoch
print(f'Epoch {epoch+1:03d}/{self.epochs} | Train MSE:
{current_mse:.6f} | Val MSE: {val_mse:.6f} | L2: {l2_penalty:.6f} | Total Loss:
{current_loss:.6f}')

# Early Stopping
# Validation MSE dipakai karena training MSE selalu turun
# dan tidak mencerminkan performa di data yang belum dilihat

if val_mse < best_val_mse:
    best_val_mse = val_mse
    no_improve = 0

# Simpan snapshot bobot terbaik saat ini
best_U = self.U.copy()
best_V = self.V.copy()
best_Sigma = self.Sigma.copy()
best_b_u = self.b_u.copy()

```

```
        best_b_i = self.b_i.copy()
    else:
        no_improve += 1
        if no_improve >= self.patience:
            print(f"\n[Early Stopping] Tidak ada perbaikan Val MSE selama
{self.patience} epoch.")
            print(f"[Early Stopping] Berhenti di epoch {epoch+1}. Best Val
MSE: {best_val_mse:.6f}")

        # Kembalikan bobot ke snapshot terbaik
        self.U = best_U
        self.V = best_V
        self.Sigma = best_Sigma
        self.b_u = best_b_u
        self.b_i = best_b_i
        break
```